

Dédicace

Abréviations

API	Application Programming Interface
AHP	Analytic Hierarchy Process
AST	Abstract Syntax Tree
AUC	Area Under the Curve
CSV	Comma-Separated Values
DPG	Digital Public Goods
DPI	Digital Public Infrastructure
G2P	Government-to-Person
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
MLflow	Machine Learning Flow
NLP	Natural Language Processing
ORM	Object Relational Mapping
PIAF	Pipeline Intégré d'Analyse de Fraude
PMT	Proxy Means Test
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SQL	Structured Query Language
UML	Unified Modeling Language
XAI	Explainable Artificial Intelligence
XGBoost	eXtreme Gradient Boosting

Table des matières

Contexte général	1
1 Contexte du projet	2
1.1 Introduction :	2
1.2 Présentation de l'organisme d'accueil : EY Tunisie	2
1.2.1 AMC EY Tunisie	3
1.2.2 Services proposés par EY Tunisie	3
1.2.3 Présentation du sous service line Technology Strategy & Transformation :	4
1.3 Présentation du projet	5
1.3.1 Cadre du projet	5
1.3.2 Contexte du projet	5
1.3.3 Problématique	5
1.3.4 Objectif du projet	6
1.3.5 Étude de l'existant : la plateforme OpenG2P	6
1.3.5.1 Présentation d'OpenG2P	6
1.3.5.2 Processus actuel dans OpenG2P et les limites face au risque de fraude	7
1.4 Choix méthodologique et conceptuel	8
1.4.1 Méthodologie de conception	8
1.4.2 Méthodologie adoptée	8
1.4.2.1 Comparaison entre les méthodologies traditionnelles et agiles	9
1.4.3 Pilotage du projet	9
1.4.3.1 Les rôles de la méthodologie Scrum	9
1.4.3.2 Les concepts de base de la méthodologie Scrum	9
1.5 Diagramme de Gantt	10
2 État de l'art	12
2.1 Définitions et concepts clés	12
2.1.1 Concepts liés à la fraude et à la protection sociale :	12
2.1.2 Concepts d'Intelligence Artificielle et Machine Learning	13

2.1.3	Métriques d'Évaluation et Concepts de Validité	14
2.1.4	Explicabilité et génération d'explications	14
2.1.5	Analyse de graphe et détection de réseaux	15
2.1.6	Modèles de Machine Learning et Architectures de Détection	15
2.1.7	Architecture et plateforme	16
2.1.8	Identification des acteurs	17
2.1.9	Présentation des travaux existants	18
3	Spécifications fonctionnelles et choix technologiques	21
3.1	Introduction	21
3.2	Identification des besoins fonctionnelles	21
3.3	Identification des besoins non fonctionnels	22
3.4	Diagramme de cas d'utilisation général	24
3.4.1	Description du diagramme de cas d'utilisation	25
3.5	Diagrammes de séquence UML	26
3.6	Architectures des composants du moteur de détection de fraude	29
3.6.1	Pipeline global du moteur de détection (agrégation)	29
3.6.2	Architecture des composants du moteur de détection	30
3.6.3	Moteur de règles métier (Rule Engine)	32
3.6.4	Analyseur de réseau (Graph)	33
3.6.5	Explicabilité et génération du texte (SHAP + LLM)	33
3.6.6	Organisation de flux de données	34
3.6.7	Architecture physique et logique de la solution	36
3.6.8	Stack Technologique	39
3.6.8.1	Stack de développement et DevOps	41
3.6.8.2	Dashboards et visualisation des résultats	41
3.6.8.3	Collaboration et documentation	42
3.7	Conclusion	43
4	Réalisation	44
4.1	Déploiement et stabilisation de la plateforme	44
4.1.1	Configuration Docker Compose	44
4.1.2	Interface de la plateforme OpenG2P après la stabilisation	46
4.1.3	Interface de monitoring et de supervision	49

4.2	Développement des fonctionnalités	51
4.2.1	Construction du dataset	51
4.2.2	Modélisation et choix des scénarios de fraude	51
4.2.3	Exploration de la base et extraction SQL	54
4.2.3.1	Sélection des sources de données	54
4.2.3.2	Construction des features et tableau des 25 variables	55
4.2.4	Structure du dataset final	56
4.2.5	Génération du dataset synthétique et injection PostgreSQL	60
4.2.6	Un second banc d'essai : le dataset PaySim	61
4.2.7	Moteur de détection de fraude	62
4.2.8	Conception du moteur de détection de fraude par Machine Learning . . .	63
4.2.9	Analyse de réseau	66
4.2.10	Fusion des scores	67
4.2.11	Explicabilité SHAP	68
4.2.12	Explication LLM	70
4.3	Intégration dans OpenG2P	72
4.3.1	Synchronisation Odoo	72
4.3.2	Exposition via une API de services	76
4.4	Tableau de bord de suivi - Streamlit	78
4.4.1	Tableau de bord de suivi du moteur de détection de fraude	78
4.4.2	Analyse des dossiers de fraude	80
4.5	Conclusion partielle	85
5	Perspectives	87
5.1	Limites du travail réalisé	87
5.2	Améliorations à court terme	87
5.2.1	Apprentissage des poids de fusion	87
5.2.2	Intégration du score d'anomalie	87
5.2.3	Calibration par pays	88
5.2.4	Enrichissement des explications par RAG	88
5.3	Conditions d'un déploiement réel	88
5.3.1	Protection des données personnelles	88
5.3.2	Supervision du modèle en production	88

5.3.3	Montée en charge	88
5.4	Extensions envisageables	89

Table des figures

1.1	Logo d'EY	2
1.2	EY Global	2
1.3	AMC EY Tunisie	3
1.4	Services EY Tunisie	3
1.5	Les activités de la division « Technology Strategy & Transformation »	4
1.6	Logo de OpenG2P	6
1.7	Architecture fonctionnelle de OpenG2P	7
1.8	Processus actuel dans OpenG2P et les zones de vulnérabilité à la fraude	8
1.9	Déroulement général du processus Scrum	10
1.10	Diagramme de Gantt du projet	10
3.1	Diagramme de cas d'utilisation général	24
3.2	Diagramme de séquence des interactions entre les composants du moteur intelligent de détection de fraude	27
3.3	Diagramme de séquence : processus d'acceptation ou de refus d'un dossier par l'analyste fraude	28
3.4	Pipeline global d'orchestration du moteur de détection de fraude	29
3.5	Diagramme de classe des principaux composants du moteur de détection de fraude	31
3.6	Processus d'évaluation du moteur de règles métier	32
3.7	Processus d'analyse du réseau relationnel des bénéficiaires	33
3.8	Processus d'explicabilité SHAP et de génération d'explication par LLM	34
3.9	Diagramme de flux de données du moteur intelligent de détection de fraude	35
3.10	Architecture logique du moteur intelligent de détection de fraude	37
3.11	Architecture logique du moteur intelligent de détection de fraude	38
3.12	Langages et outils utilisés pour le développement du moteur intelligent de détection de fraude	39
4.1	Démarrage de l'ensemble du système via Docker Compose	46
4.2	Écran de connexion à la plateforme OpenG2P	46
4.3	Tableau de bord principal après authentification	47

4.4	Liste des bénéficiaires enregistrés dans le registre social	47
4.5	Gestion des groupes de bénéficiaires	48
4.6	Gestion des programmes sociaux et de leurs inscriptions	48
4.7	Gestion des paiements et des factures	49
4.8	Monitoring de l'état des conteneurs et des métriques système via Grafana	49
4.9	Monitoring de l'état du moteur anti-fraude via Grafana	50
4.10	Suivi de l'état des services via Prometheus	50
4.11	Processus de construction du dataset de détection de fraude à partir des données OpenG2P	51
4.12	Scénarios de fraude simulés	52
4.13	Vue d'ensemble du schéma relationnel d'OpenG2P et des colonnes des sept tables sélectionnées pour l'extraction des données	54
4.14	Colonnes et types du dataset OpenG2P généré (59 colonnes, dont les 28 features ML)	57
4.15	Processus de génération du dataset OpenG2P synthétique	60
4.16	Distribution du score PMT selon la classe réelle dans le dataset OpenG2P syn- thétique	61
4.17	Distribution du montant des transactions selon la classe réelle dans le dataset PaySim	62
4.18	Répartition des classes dans les données d'entraînement	64
4.19	Courbes ROC comparées des trois modèles de base et de l'ensemble	65
4.20	Distribution du score d'anomalie (Isolation Forest) par classe réelle	66
4.21	Pipeline de l'analyse de réseau : extraction des relations, construction du graphe, pondération des arêtes et calcul du PageRank	67
4.22	Importance globale des features (SHAP beeswarm) sur le jeu de test	69
4.23	Facteurs explicatifs SHAP pour un cas de fraude individuel	70
4.24	Explication LLM détaillée générée pour un bénéficiaire intégrée dans le tableau de bord	71
4.25	Explication LLM simple générée pour un bénéficiaire intégrée dans le tableau de bord	71
4.26	Intégration du module de détection de fraude dans Odoo	72
4.27	Intégration du module de notification d'alertes synchronisées dans Odoo	72
4.28	Intégration du module de notification : détails d'une alerte Odoo	73
4.29	Tableau de bord : vue des alertes actives avec niveaux de risque intégré dans Odoo	74
4.30	Tableau de bord Odoo : vue du scanner en temps-réel de la base de données . .	74

4.31	Tableau de bord Odoo : outil de prise de décision	75
4.32	Historique des alertes de fraude sur la fiche partenaire du bénéficiaire	76
4.33	Documentation Swagger de l'API exposant le pipeline de scoring	76
4.34	Documentation Swagger de l'API exposant le pipeline de scoring	77
4.35	Tableau de bord de suivi des cas de fraude	79
4.36	Résultat du scoring d'un bénéficiaire	80
4.37	Scoring des bénéficiaire au format CSV	81
4.38	Génération de rapport de fraude d'un bénéficiaire	82
4.39	Explication détaillée d'une décision de fraude	82
4.40	Carte des clusters de fraude géographiques détectés par DBSCAN	83
4.41	Tableau de monitoring du système et feedback des analystes	84
4.42	Interface de gestion des règles de détection de fraude	85

Table des tableaux

1.1	Services professionnels d'EY Tunisie	4
1.2	Méthodologies Traditionnelles vs Agiles	9
2.1	Acteurs internes du système	17
2.2	Acteurs externes et systèmes	18
2.3	Comparaison des méthodes de détection de fraude : avantages et limites	18
2.4	Solutions commerciales de détection de fraude sur le marché	19
2.5	Limites des travaux existants face à la philosophie DPI / DPG	20
3.1	Spécification des besoins fonctionnels du moteur intelligent de détection de fraude par acteur	21
3.2	Rôle des principaux composants du moteur de détection	30
3.3	Éléments clés du moteur de règles	32
3.4	Technologies de base du système de détection de fraude	40
3.5	Technologies d'explicabilité et de gestion des modèles	40
3.6	Outils de développement et d'infrastructure	41
3.7	Dashboards et plateformes de visualisation	42
3.8	Outils de collaboration et de documentation	43
4.1	Conteneurs Docker du système	45
4.2	Description des neuf scénarios de fraude simulés	53
4.3	Principales sources de données exploitées pour la construction du dataset	55
4.4	Les 25 features extraites pour le modèle ML	56
4.5	Fiche technique (datacard) du dataset synthétique	58
4.6	Statistiques descriptives des features par catégorie	59
4.7	Comparaison des modèles de base et de l'ensemble (jeu de test, seuil 0,5)	64
4.8	Rôles et permissions d'accès au système	78
4.9	Fonctionnalités du tableau de bord pour l'analyste fraude	80

Introduction Générale

Dans un monde où **1,3 milliard de personnes** vivent encore sous le seuil de pauvreté [1], la protection sociale n'est plus un luxe, c'est un impératif de survie. Chaque jour, des millions de familles dans les pays émergents et en développement dépendent d'un transfert monétaire pour nourrir leurs enfants, payer une consultation médicale ou maintenir un enfant à l'école. Ces programmes de transferts monétaires directs, désignés sous le terme **Government-to-Person (G2P)**, constituent aujourd'hui le mécanisme le plus efficace pour réduire la pauvreté et les inégalités dans les pays les plus vulnérables.

Pour rendre ces programmes possibles à grande échelle, les gouvernements se tournent vers un concept transformateur : **les Infrastructures Publiques Numériques (DPI)**.

Les DPI (Digital Public Infrastructure) désignent l'ensemble des systèmes numériques partagés, tels que les systèmes d'identification biométrique, les registres sociaux centralisés ou les plateformes de paiement, qui constituent le socle technologique nécessaire à la fourniture de services publics modernes, transparents et inclusifs.

Afin de développer et d'opérer efficacement ces infrastructures numériques, les gouvernements peuvent s'appuyer sur les **Biens Publics Numériques (Digital Public Goods)**, des solutions logicielles open source conçues pour être réutilisées, adaptées et déployées à grande échelle dans différents contextes nationaux.

OpenG2P est une plateforme open source conçue pour digitaliser la gestion des programmes de protection sociale. Déployée dans plusieurs pays tels que le Kenya, le Malawi et la Sierra Leone, OpenG2P constitue une solution robuste pour les institutions engagées dans la modernisation de leurs infrastructures numériques et l'amélioration de la gouvernance des aides sociales sans dépendance à un éditeur privé.

Cependant, les programmes G2P font face à un adversaire silencieux : **la fraude**.

Les tactiques sont multiples : multi-inscription sous des identités différentes, ménages fictifs etc. Selon la Banque mondiale, **jusqu'à 27 % du budget destiné à la protection sociale** est perdu à cause de ces fuites [3].

Le problème est aggravé par le fait que les contrôles actuels interviennent trop tard. Les audits manuels sont lents, coûteux, et ne couvrent qu'un échantillon limité. Les solutions commerciales, quant à elles, sont coûteuses et incompatibles avec la philosophie open-source des DPG.

C'est dans ce cadre que notre stage effectué au sein d'EY Tunisie s'inscrit : concevoir un moteur intelligent de détection de fraude libre, proactif et explicable, intégré à OpenG2P, pour protéger les budgets de protection sociale avant même le versement.

Ce rapport détaille la conception, le développement et l'évaluation de ce projet, en mettant l'accent sur la rigueur scientifique, l'adaptabilité à différents contextes nationaux et la capacité à protéger efficacement les budgets d'aide sociale.

Chapitre 1 Contexte du projet

1.1 Introduction :

Ce chapitre expose le contexte général des travaux menés, nous allons présenter le contexte global du projet, en débutant par l'introduction de l'organisme d'accueil qui est « EY Tunisie », décrire ses activités et ses différentes équipes. Par la suite, nous procédons à la description de notre problématique qui a mené au développement de ce projet. Enfin, nous expliquons la méthodologie choisie pour accomplir ce travail.

1.2 Présentation de l'organisme d'accueil : EY Tunisie



FIGURE 1.1 – Logo d'EY

Fondé en 1849, Ernst and Young est un cabinet d'audit financier et de conseil multinational implanté dans plus de 152 pays organisés en trois régions : Amérique ; Europe, Moyen Orient, Inde et Afrique (EMEIA) ; Asie-Pacifique. EY fait partie des « Big Four », soit les quatre plus grands groupes d'audit à l'échelle mondiale. EY est une organisation de services professionnels approfondie avec plus de 700 bureaux et 350 000 associés et collaborateurs dans le monde entier.

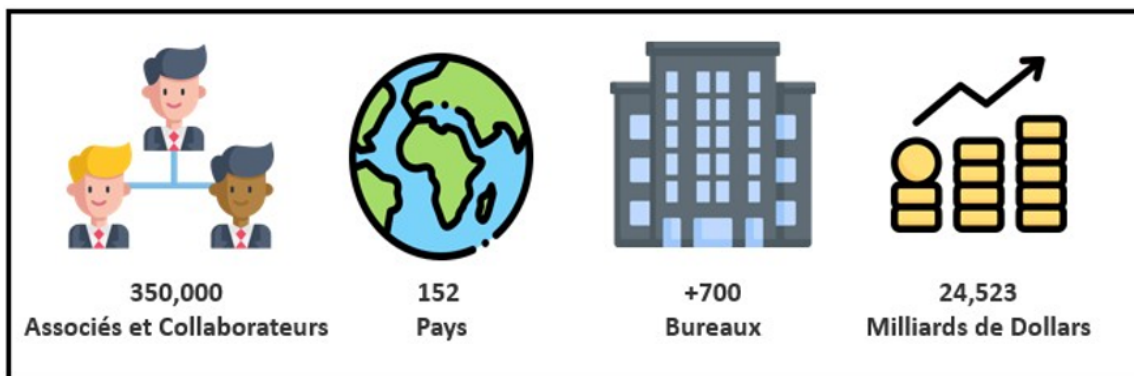


FIGURE 1.2 – EY Global

EY est une entreprise de premier plan à l'échelle mondiale évoluant dans le domaine d'assurance, de fiscalité, de transaction, et de conseil. EY bénéficie d'une équipe mondiale en expansion constante.

1.2.1 AMC EY Tunisie

Établie sur le territoire depuis 1987, AMC EY Tunisie se positionne comme l'une des pionnières dans le domaine des services professionnels du pays. Elle fait partie de la région EMEIA et compte plus de 1000 employés avec plus de 900 consultants en service de conseil. Grâce à plus de 620 missions réalisées annuellement, l'entreprise a atteint un chiffre d'affaires de 50 Millions de dinars tunisiens en 2022 avec plus de 50% réalisées auprès de clients internationaux.



FIGURE 1.3 – AMC EY Tunisie

EY Tunisie intervient dans différents secteurs économiques en Tunisie, notamment dans le secteur des technologies de l'information et de la communication, méritant la confiance tant du secteur public local (ministères, agences de développement local) que du secteur privé grâce à la proposition de solutions adaptées et pertinentes.

1.2.2 Services proposés par EY Tunisie

EY Tunisie structure son offre autour de quatre grands métiers complémentaires qui permettent à l'entreprise de répondre aux besoins variés de ses clients résumés dans le tableau suivant :

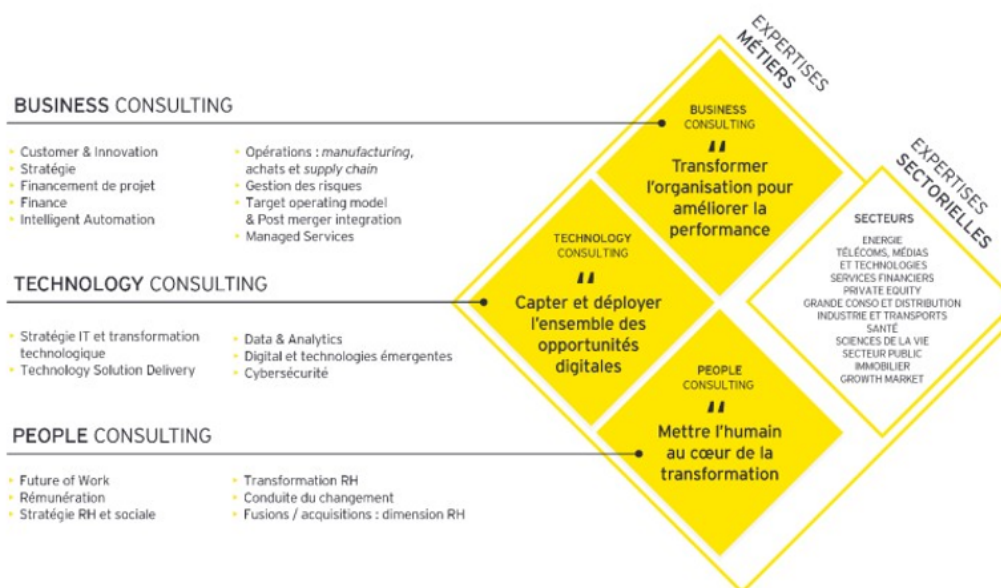


FIGURE 1.4 – Services EY Tunisie

Service	Description
Conseil	EY aide ses clients à créer de la valeur à long terme pour toutes les parties prenantes. En s'appuyant sur les données et la technologie, ses services et ses solutions orientent les clients et les aident à prendre des décisions sur le plan stratégique et opérationnel afin de parvenir favorablement à leurs attentes.
Audit	EY fournit des renseignements et cherche à offrir aux clients (investisseurs) la meilleure gestion et l'adéquate structuration de ses rapports et de ses opérations dans le but d'améliorer voire de valoriser leurs tâches et d'accroître leurs opportunités envers les autres entreprises.
Fiscalité et droit (Tax)	EY possède une expertise en matière d'impôt sur les sociétés, d'impôt international, d'impôt sur les transactions et de ressources humaines, de conformité et de déclaration, et d'impôt statutaire.
Stratégies et Transactions	EY aide ses clients à surmonter la crise et à stabiliser leurs activités à court terme. EY aide également les entreprises à surmonter les ralentissements et à se préparer à la reprise, ou à regarder au-delà pour permettre la transformation par le biais de la stratégie d'entreprise, des transactions, des fusions et acquisitions et des désinvestissements.

TABLEAU 1.1 – Services professionnels d'EY Tunisie

1.2.3 Présentation du sous service line Technology Strategy & Transformation :

Le département « **Technology Strategy & Transformation** » qui fait partie du service line Consulting occupe une position centrale dans la transformation d'entreprise et propose des possibilités d'optimisation des rendements, de supervision des risques et de promotion de l'innovation des entreprises.

En s'appuyant sur une gamme diversifiée de projets technologiques, la division « **Technology Strategy & Transformation** » offre à ses clients la possibilité d'améliorer leurs pratiques et d'acquérir un avantage compétitif. L'équipe offre cinq (5) principales activités pour cinq (5) secteurs intersectoriels, comme l'illustre la figure ci-dessous.

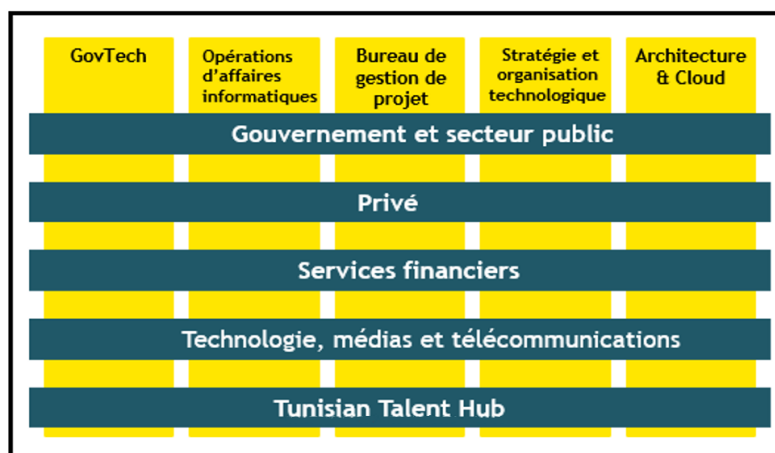


FIGURE 1.5 – Les activités de la division « Technology Strategy & Transformation »

1.3 Présentation du projet

1.3.1 Cadre du projet

Notre projet s'inscrit dans le cadre du stage de fin d'études pour l'obtention du Diplôme National d'Ingénieur en Informatique, réalisé au sein du cabinet EY Tunisie. Il s'inscrit dans une dynamique de transformation numérique des services publics, portée par l'adoption des **Digital Public Infrastructures (DPI)** et des **Digital Public Goods (DPG)**. Cette démarche rejoint plusieurs initiatives menées dans des pays francophones, où l'implémentation de solutions DPI/DPG bénéficie du soutien de bailleurs de fonds internationaux, dans une stratégie de transformation vers infrastructure numérique publique mutualisée et réutilisable au-delà d'un seul programme ou d'un seul pays.

1.3.2 Contexte du projet

La transformation numérique des administrations publiques s'appuie aujourd'hui sur les *Digital Public Infrastructures (DPI)* et les *Digital Public Goods (DPG)*, qui favorisent le développement de services publics numériques interopérables, sécurisés et accessibles. Dans ce contexte, *OpenG2P* constitue un Digital Public Good dédié à la gestion des programmes de protection sociale, en couvrant l'ensemble du cycle de vie des prestations, depuis l'enregistrement des bénéficiaires jusqu'à la distribution des aides.

C'est dans cette dynamique que s'inscrit notre projet, réalisé au sein d'EY Tunisie. Il consiste à concevoir et développer un moteur intelligent de détection des risques et des fraudes intégré à la plateforme OpenG2P, afin de renforcer la fiabilité et la transparence des programmes de protection sociale. Une étude préalable de l'architecture et du fonctionnement de la plateforme a permis d'identifier plusieurs limites en matière de détection de la fraude, qui constituent la problématique abordée dans ce mémoire.

1.3.3 Problématique

Les programmes de protection sociale gèrent chaque année des flux financiers considérables destinés aux populations vulnérables. Cependant, selon la Banque mondiale, **jusqu'à 27% de ce budget** est perdu à cause de la fraude et des erreurs de ciblage [3]. Les principaux problèmes identifiés sont les suivants :

- **La fraude prend des formes variées et évolutives** : faux bénéficiaires, ménages fictifs, identités partagées, manipulation de paiements.
- **Les contrôles actuels interviennent après le versement** : quand l'argent a déjà quitté les caisses de l'État.
- **Les vérifications manuelles** sont lentes, coûteuses, et ne couvrent qu'un échantillon limité.
- **Les règles de contrôle figées** ne détectent que les fraudes connues et ne s'adaptent pas aux nouvelles tactiques.
- **Les solutions commerciales** existantes sont **coûteuses, opaques, et incompatibles**

avec la philosophie open-source d'OpenG2P.

Il devient donc indispensable de disposer d'une solution intelligente, proactive et explicable, capable de détecter la fraude **avant** le versement.

1.3.4 Objectif du projet

Face à ces constats, notre projet a pour ambition de transformer fondamentalement la manière dont les programmes de protection sociale luttent contre la fraude. L'objectif n'est pas simplement de détecter des cas suspects, mais de **changer le paradigme** : passer d'un système réactif, qui découvre la fraude après le versement, à un système intelligent qui la prévient avant qu'elle ne cause des dégâts. Pour y parvenir, nous avons structuré notre travail autour de cinq objectifs principaux :

- **Détection proactive** : identifier les bénéficiaires à risque avant le versement, au moment même de leur enregistrement dans le système.
- **Analyse multi-couches** : croiser plusieurs signaux complémentaires règles métier, apprentissage automatique, comportements inhabituels et liens entre bénéficiaires pour ne laisser aucun angle mort.
- **Explicabilité des décisions** : accompagner chaque alerte d'une explication claire en français, permettant à un agent social de comprendre et de justifier pourquoi un cas est signalé.
- **Amélioration continue** : créer un système qui apprend des retours des enquêteurs et devient plus performant au fil du temps.
- **Accessibilité et intégration** : offrir une solution 100% open-source, gratuite et intégrée directement à la plateforme OpenG2P que les gouvernements utilisent déjà.

1.3.5 Étude de l'existant : la plateforme OpenG2P

Avant de proposer une solution, il est essentiel de comprendre comment fonctionne la plateforme sur laquelle elle sera intégrée. Nous avons donc mené une étude approfondie d'OpenG2P, son architecture, ses modules, et ses processus métier, afin d'identifier précisément où se situent les faiblesses face à la fraude.

1.3.5.1 Présentation d'OpenG2P



FIGURE 1.6 – Logo de OpenG2P

OpenG2P est une plateforme open-source bâtie sur Odoo 17, conçue pour centraliser la gestion des programmes de protection sociale. Elle est déployée dans plusieurs pays (Kenya, Malawi, Sierra Leone etc) et couvre l'ensemble du cycle de vie d'un programme social, depuis l'enregistrement des bénéficiaires jusqu'au versement des aides. La plateforme s'organise autour de plusieurs modules interconnectés :

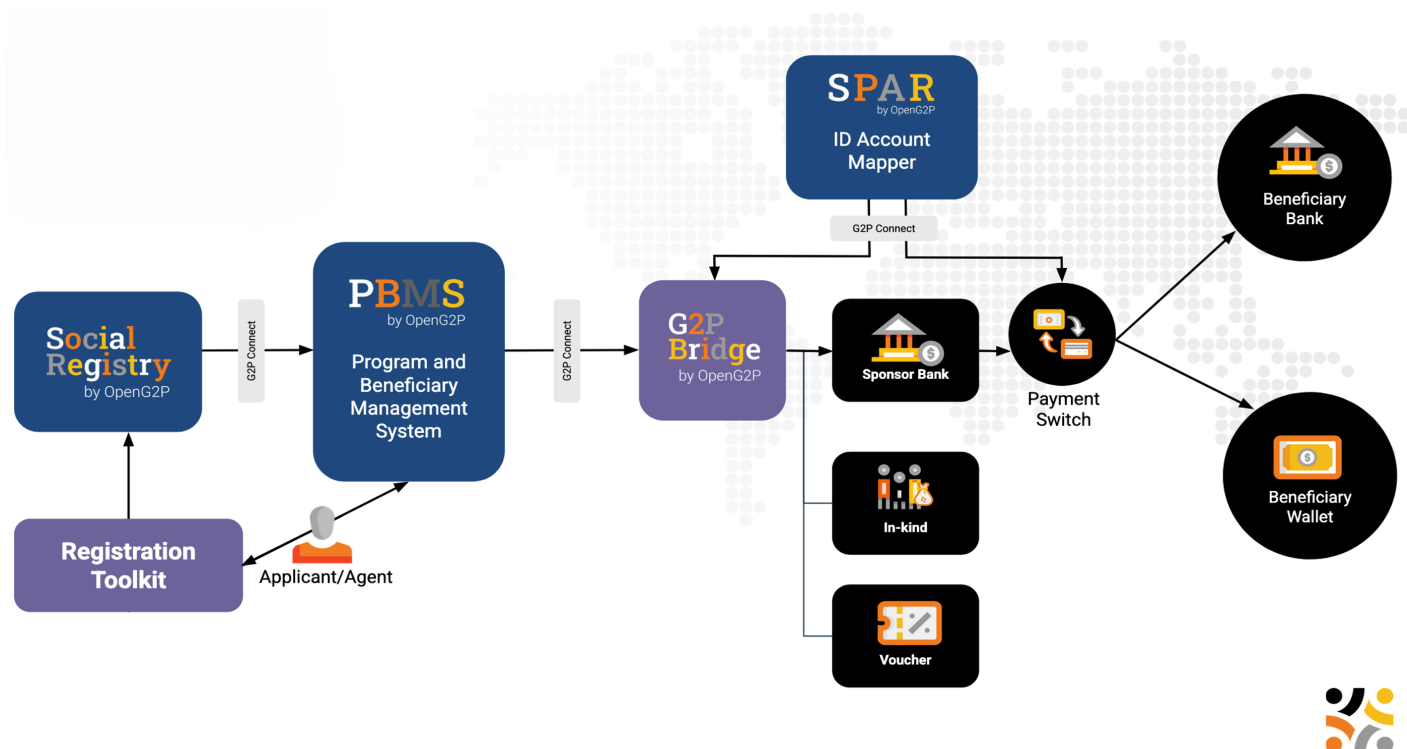


FIGURE 1.7 – Architecture fonctionnelle de OpenG2P

- **Registre social (Social Registry)** : base de données centralisée des bénéficiaires avec leurs informations personnelles, démographiques et socio-économiques
- **Gestion des programmes (PBMS)** : définition des programmes d'aide, organisation des cycles de versement (trimestriels, mensuels)
- **Droits et paiements (SPAR)** : calcul automatisé des montants dus à chaque bénéficiaire et suivi de chaque versement
- **Données transactionnelles (G2P Bridge)** : registre des coordonnées téléphoniques et bancaires utilisées pour les transferts
- **Groupes et ménages** : gestion des liens familiaux et des regroupements de bénéficiaires

1.3.5.2 Processus actuel dans OpenG2P et les limites face au risque de fraude

Le processus de contrôle anti-fraude dans OpenG2P repose actuellement sur un système minimal : quelques vérifications manuelles effectuées par les agents sociaux lors de l'enregistrement des bénéficiaires, complétées par des audits post-versement sur un échantillon limité. Aucun moteur automatisé de détection n'est intégré à la plateforme.

Afin de mieux comprendre les limites du système actuel, nous présentons ci-dessous une analyse des mécanismes de contrôle existants.

La Figure 1.6 illustre le processus actuel de vérification dans OpenG2P, où l'agent social

consulte manuellement les fiches des bénéficiaires et effectue des contrôles ponctuels sans outil d'aide à la décision :

Blocs OpenG2P et chaîne de distribution des prestations sociales

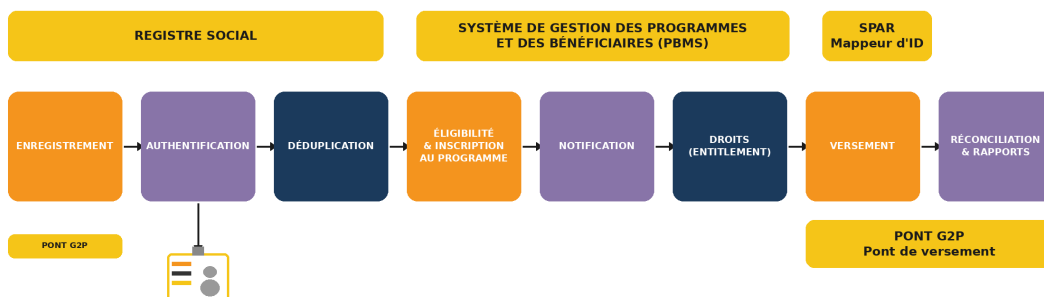


FIGURE 1.8 – Processus actuel dans OpenG2P et les zones de vulnérabilité à la fraude

Comme le montre cette figure, **aucun mécanisme de détection automatique** n'intervient entre l'enregistrement et le versement. La fraude peut s'infiltrer à chaque étape sans être détectée, et les contrôles manuels post-versement ne couvrent qu'une fraction des cas.

1.4 Choix méthodologique et conceptuel

1.4.1 Méthodologie de conception

Pour la conception de ce projet, nous avons adopté le langage de modélisation **UML** afin de représenter les différentes vues du système de manière standardisée. Nous avons retenu quatre diagrammes : cas d'utilisation, classes, séquence et déploiement, qui permettent de couvrir respectivement les interactions, la structure, le flux de scoring et l'architecture Docker du système.

1.4.2 Méthodologie adoptée

La nature exploratoire d'un projet de data science où les données évoluent, les modèles doivent être comparés et les hypothèses remises en question impose une méthodologie flexible. Nous avons donc adopté la méthode **SCRUM**, un cadre agile structuré en sprints de 1 à 2 semaines, permettant de livrer des incréments fonctionnels et d'intégrer les retours d'EY Tunisie de manière continue.

1.4.2.1 Comparaison entre les méthodologies traditionnelles et agiles

Critère	Traditionnelle	Agile
Approche	Linéaire et séquentielle	Itérative et incrémentale
Exigences	Figées au départ	Évolutives
Changement	Difficile et coûteux	Flexible et intégré
Client	Impliqué au début et à la fin	Impliqué en continu
Livraison	Une seule fois à la fin	Fréquente (cycles courts)
Risques	Identifiés tardivement	Maîtrisés en continu

TABLEAU 1.2 – Méthodologies Traditionnelles vs Agiles

1.4.3 Pilotage du projet

Le projet a été organisé en six phases : cadrage, conception, génération de données, modélisation ML, intégration et tests. L'équipe SCRUM est composée comme suit :

Product Owner : EY Tunisie

Scrum Master : [Cyrine Garram, Mariem Abcha]

Équipe de développement : [Halim Trabelsi]

1.4.3.1 Les rôles de la méthodologie Scrum

La méthode **Scrum** repose sur trois rôles principaux :

Product Owner : Représente les besoins fonctionnels du projet, définit les priorités, gère le *Product Backlog* et valide les livrables à la fin de chaque sprint.

Scrum Master : Garantit l'application de la méthodologie Scrum, facilite les échanges entre les membres de l'équipe et veille à la résolution des obstacles rencontrés.

Équipe projet : Réunit les membres chargés de la conception, du développement, des tests et de l'intégration de la solution afin d'atteindre les objectifs définis pour chaque sprint.

1.4.3.2 Les concepts de base de la méthodologie Scrum

La méthode **Scrum** s'appuie sur plusieurs concepts fondamentaux qui permettent d'organiser et de piloter efficacement le développement du projet :

Backlog produit : Il s'agit de la liste des besoins, des fonctionnalités et des objectifs du projet, classés par ordre de priorité. Ce backlog évolue tout au long du projet en fonction des besoins et des retours des parties prenantes.

Sprint Backlog : Il correspond à l'ensemble des tâches sélectionnées à partir du backlog produit pour être réalisées au cours d'un sprint donné.

Sprint : Il s'agit d'une période de travail de durée limitée, généralement comprise entre une et quatre semaines, durant laquelle l'équipe développe un incrément fonctionnel du produit.

Mêlée quotidienne (*Daily Scrum*) : C'est une courte réunion quotidienne permettant de suivre l'avancement des travaux, d'identifier les difficultés rencontrées et de coordonner les actions à entreprendre.

Incrément : Il représente le résultat obtenu à la fin d'un sprint. Chaque incrément doit constituer une version fonctionnelle et exploitable du produit, apportant une valeur ajoutée au projet.

La Figure 1.9 illustre le déroulement général du processus Scrum.

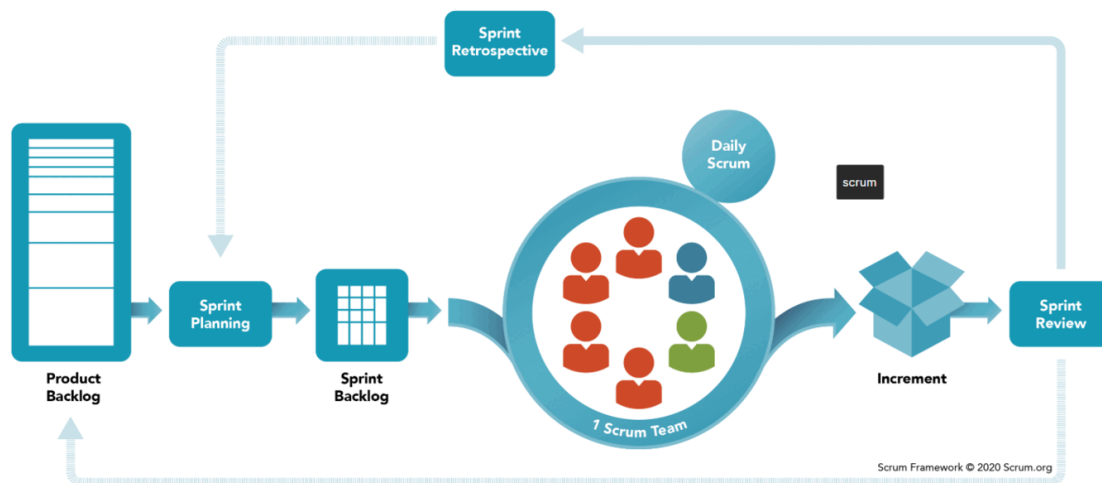


FIGURE 1.9 – Déroulement général du processus Scrum

1.5 Diagramme de Gantt

Le diagramme de Gantt ci-dessous présente la planification temporelle du projet, découpée en phases et en sprints sur la durée du stage. La Figure 1.10 illustre cette répartition en utilisant les mois comme unité de temps.

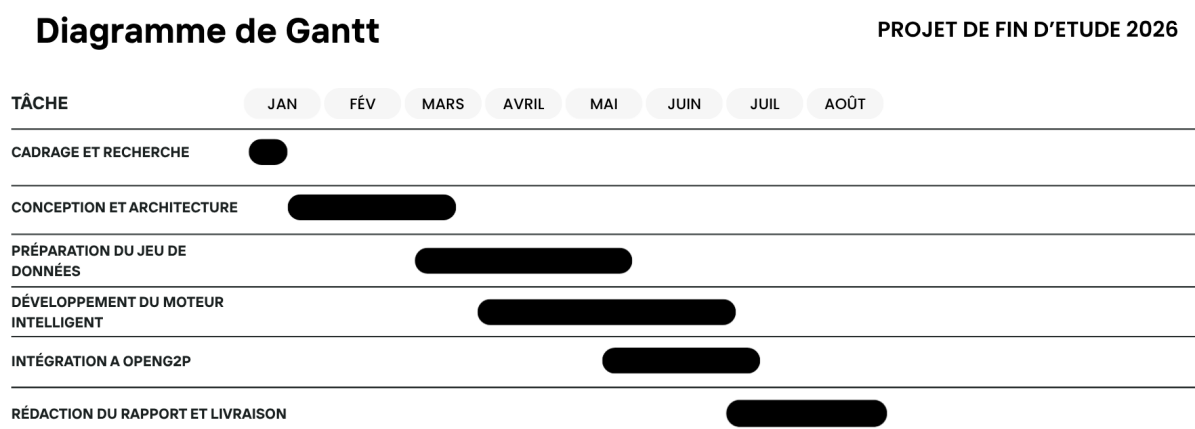


FIGURE 1.10 – Diagramme de Gantt du projet

Les principales phases du projet sont les suivantes :

Cadrage et recherche : Cette première phase a consisté à analyser le contexte du projet, identifier les besoins de l'entreprise, étudier OpenG2P ainsi que les travaux existants relatifs à la détection de fraude.

Conception et architecture : Cette étape a permis de définir l'architecture de la solution, les technologies à utiliser ainsi que les différents modules composant le moteur intelligent.

Préparation du jeu de données : Cette phase a été consacrée à l'identification des scénarios de fraude, à la conception des caractéristiques (*features*) et à la génération d'un jeu de données synthétique destiné à l'entraînement et à l'évaluation des modèles.

Développement du moteur intelligent : Cette étape correspond à l'implémentation du moteur de détection de fraude, incluant le moteur de règles, les modèles de Machine Learning, l'analyse des graphes et le calcul du score de risque.

Intégration à OpenG2P : Cette phase a consisté à intégrer la solution développée au sein de la plateforme OpenG2P, à assurer la communication entre les différents composants et à valider le bon fonctionnement de l'ensemble.

Rédaction du rapport et livraison : La dernière phase a porté sur la rédaction du mémoire, la préparation de la soutenance, la finalisation des livrables et la validation finale du projet.

Conclusion

Ce chapitre a posé les fondations du projet : après avoir présenté le cadre institutionnel, étudié les lacunes d'OpenG2P et du marché, et défini nos objectifs, nous avons adopté une méthodologie SCRUM adaptée à la nature itérative d'un projet de data science. Le chapitre suivant sera consacré à l'**état de l'art**, où nous examinerons les techniques de détection de fraude dans la littérature scientifique afin de justifier les choix techniques de notre solution.

Chapitre 2 État de l’art

Introduction

Avant de détailler la conception et la réalisation de notre solution, il est essentiel de poser les bases théoriques sur lesquelles elle repose. Ce chapitre a pour objectif de présenter les notions fondamentales nécessaires à la compréhension du projet et d’examiner les techniques existantes dans la littérature scientifique pour la détection de fraude.

2.1 Définitions et concepts clés

Il est indispensable de définir le vocabulaire et les notions fondamentales qui seront mobilisés tout au long de ce rapport. Cette section a pour but de rendre le reste du document accessible à tout lecteur, y compris ceux qui ne sont pas familiers avec les domaines de l’intelligence artificielle ou de l’analyse de données.

2.1.1 Concepts liés à la fraude et à la protection sociale :

- **Programme G2P (Government-to-Person)** : mécanisme par lequel un gouvernement verse des aides financières directement aux citoyens éligibles (allocations familiales, bourses, aides alimentaires). C’est le contexte métier de notre projet.
- **Détection de fraude** : ensemble des méthodes permettant d’identifier des comportements malhonnêtes ou irréguliers dans un système. Dans notre cas, il s’agit de repérer les bénéficiaires qui tentent de recevoir indûment des aides sociales.
- **Protection sociale** : ensemble des politiques et programmes publics visant à réduire la pauvreté et la vulnérabilité des populations, à travers des transferts monétaires, des aides alimentaires ou un accès facilité aux services essentiels (santé, éducation). Les programmes de protection sociale constituent le terrain d’application de notre système de détection de fraude.
- **Bénéficiaire** : individu ou ménage inscrit à un ou plusieurs programmes de protection sociale et destinataire des prestations versées. C’est l’entité sur laquelle porte l’analyse de risque de fraude dans notre système.
- **Fraude sociale** : manipulation intentionnelle des informations ou des processus d’un programme social dans le but d’obtenir un avantage indu. Elle se distingue de l’erreur involontaire (déclaration erronée non intentionnelle) par la présence d’une intention de tromper le système. Plusieurs typologies sont couvertes par notre moteur de règles :
 - *inscription multiple (multi-enrollment)* : un même bénéficiaire s’inscrit à un nombre anormalement élevé de programmes cumulables ;
 - *usurpation d’identité (identity fraud)* : utilisation d’une identité fictive ou empruntée, souvent révélée par le partage d’un même numéro de téléphone ou compte bancaire entre plusieurs bénéficiaires.

- *ménage fantôme (ghost household)* : déclaration d'un ménage fictif ou surdimensionné pour augmenter le montant de l'aide perçue.
- *sous-déclaration de revenus (income underreporting)* : dissimulation du revenu réel afin de paraître éligible à un programme destiné aux ménages les plus pauvres.
- *manipulation des paiements (payment manipulation)* : anomalies volontaires dans le cycle de versement (paiements dupliqués, écarts injustifiés entre montant émis et montant perçu).
- *fraude en réseau (collusion)* : coordination entre plusieurs bénéficiaires (partage de coordonnées bancaires ou téléphoniques) pour multiplier les gains frauduleux, détectable par analyse de graphe.

2.1.2 Concepts d'Intelligence Artificielle et Machine Learning

L'intelligence artificielle, et plus précisément le machine learning, est au cœur de notre solution. Pour comprendre comment notre système apprend à distinguer un bénéficiaire légitime d'un fraudeur, il est nécessaire de maîtriser les notions suivantes :

- **Machine Learning (apprentissage automatique)** : branche de l'intelligence artificielle où un programme apprend à prendre des décisions à partir d'exemples passés, sans être explicitement programmé pour chaque cas. Le programme identifie des régularités dans les données et les utilise pour faire des prédictions sur de nouveaux cas.
- **Apprentissage supervisé** : type de machine learning où le modèle apprend à partir d'exemples dont on connaît déjà la réponse (par exemple : « ce bénéficiaire est un fraudeur » ou « ce bénéficiaire est légitime »). Le modèle apprend la relation entre les caractéristiques d'un bénéficiaire et son statut, puis applique cette connaissance aux nouveaux cas.
- **Apprentissage non supervisé** : type de machine learning où le modèle ne dispose pas de réponses connues. Il cherche à identifier des comportements atypiques ou des groupes naturels dans les données, sans savoir à l'avance ce qui est normal ou anormal.
- **Feature engineering (extraction de caractéristiques)** : étape qui consiste à transformer les données brutes en variables exploitables par un modèle. Par exemple, à partir du nombre de téléphones partagés et du nombre de comptes bancaires partagés, on calcule un « score de risque réseau » qui résume l'information en un seul chiffre.
- **Data leakage (fuite de données)** : erreur dans la construction du modèle où des informations liées à la réponse (le label fraude/légitime) se retrouvent accidentellement dans les données d'entraînement. Le modèle semble alors parfait, mais ses performances sont artificiellement gonflées et ne se reproduisent pas en conditions réelles.
- **Collusion** : forme de fraude collective où plusieurs individus collaborent pour tromper le système — par exemple, en partageant un même compte bancaire ou un même numéro de téléphone pour multiplier les inscriptions.

2.1.3 Métriques d'Évaluation et Concepts de Validité

Pour évaluer la performance d'un modèle de détection de fraude, il est essentiel de comprendre les métriques suivantes, qui permettent de mesurer son efficacité et ses limitations :

- **Vrai positif** : cas où le système identifie correctement une transaction ou un bénéficiaire réellement frauduleux comme étant frauduleux.
- **Vrai négatif** : cas où le système identifie correctement une transaction ou un bénéficiaire légitime comme étant non frauduleux.
- **Faux positif** : cas où le système signale un bénéficiaire comme fraudeur alors qu'il est légitime. Un excès de faux positifs surcharge inutilement les enquêteurs et nuit à la confiance envers le système.
- **Faux négatif** : cas où le système laisse passer un vrai fraudeur sans le détecter. C'est la situation la plus coûteuse, car l'argent est versé à une personne qui n'y a pas droit.
- **Accuracy (exactitude)** : proportion de prédictions correctes (vrais positifs + vrais négatifs) parmi l'ensemble des prédictions effectuées.
- **Précision (precision)** : proportion de cas réellement frauduleux parmi tous les cas que le système a prédits comme frauduleux.
- **Rappel (recall)** : proportion de cas réellement frauduleux que le système a effectivement réussi à détecter parmi tous les cas frauduleux existants.
- **F1-score** : moyenne harmonique de la précision et du rappel, offrant une mesure équilibrée de la performance du modèle, surtout lorsque les classes sont déséquilibrées (comme c'est souvent le cas en détection de fraude).
- **AUC-ROC (Area Under the Receiver Operating Characteristic Curve)** : métrique d'évaluation qui mesure la capacité d'un modèle à distinguer les fraudeurs des légitimes, sur une échelle de 0 à 1. Un AUC de 0.5 signifie que le modèle ne fait pas mieux que le hasard ; un AUC de 1.0 signifie une détection parfaite. Dans la pratique, un AUC supérieur à 0.85 est considéré comme bon.

2.1.4 Explicabilité et génération d'explications

Un système de détection de fraude ne peut être utile que s'il est compréhensible par ses utilisateurs. Un agent social doit pouvoir expliquer à un bénéficiaire pourquoi son dossier a été signalé. C'est le rôle de l'explicabilité :

- **XAI (Explainable Artificial Intelligence)** : domaine de l'intelligence artificielle qui vise à rendre les décisions des modèles compréhensibles par des humains. Plutôt qu'une « boîte noire » qui dit simplement « fraude détectée », un système explicable indique *pourquoi* : quels facteurs ont pesé dans la décision.
- **SHAP (SHapley Additive exPlanations)** : méthode d'explicabilité qui attribue à chaque caractéristique du bénéficiaire une contribution précise au score final. Par exemple : « le partage de téléphone avec 4 autres bénéficiaires a augmenté le score de risque de +0.15 ». SHAP est fondé sur la théorie des jeux (valeurs de Shapley) et constitue aujourd'hui le standard de l'industrie.

- **LLM (Large Language Model)** : modèle d'intelligence artificielle capable de comprendre et de générer du texte en langage naturel. Dans notre projet, nous utilisons Mistral (via Ollama) pour transformer les facteurs SHAP et les drapeaux de règles en une **explication rédigée en français**, compréhensible par un agent social non-technique.
- **RAG (Retrieval-Augmented Generation)** : technique qui enrichit la réponse d'un LLM en lui fournissant des documents pertinents avant la génération. Dans notre contexte, le LLM reçoit les facteurs SHAP, les règles déclenchées et le score du bénéficiaire comme contexte, ce qui lui permet de produire une explication précise et fondée sur des faits réels, et non sur des connaissances génériques.

2.1.5 Analyse de graphe et détection de réseaux

La fraude n'est pas toujours l'acte d'un individu isolé. Souvent, plusieurs personnes collaborent en partageant des ressources (téléphones, comptes bancaires). L'analyse de graphe permet de révéler ces réseaux invisibles :

- **Graphe** : structure mathématique composée de **nœuds** (les entités — ici, les bénéficiaires) et d'**arêtes** (les relations entre eux — ici, le partage d'un téléphone ou d'un compte bancaire). Si deux bénéficiaires partagent le même numéro de téléphone, une arête les relie.
- **Détection de communautés** : technique qui identifie des groupes de nœuds fortement connectés entre eux. Une communauté dense de 5 à 15 bénéficiaires partageant plusieurs téléphones et comptes bancaires est un signal fort de fraude organisée.
- **Centralité (PageRank)** : mesure d'importance d'un nœud dans un réseau. Un bénéficiaire très connecté (lié à de nombreux autres bénéficiaires suspects) obtient un score de centralité élevé. Notre solution utilise l'algorithme PageRank, le même que celui inventé par Google pour classer les pages web adapté au contexte de la fraude sociale.

2.1.6 Modèles de Machine Learning et Architectures de Détection

Au-delà des principes fondamentaux du machine learning, plusieurs modèles et architectures spécialisés ont été développés pour traiter les problèmes complexes comme la détection de fraude. Voici les plus importants :

- **XGBoost (eXtreme Gradient Boosting)** : algorithme qui combine des centaines d'arbres de décision simples, chacun corrigeant les erreurs du précédent, pour obtenir des prédictions très précises. C'est le modèle principal retenu dans notre solution en raison de sa performance exceptionnelle sur les données déséquilibrées.
- **Random Forest (forêt aléatoire)** : algorithme qui construit plusieurs arbres de décision indépendants et combine leurs prédictions par vote majoritaire. Il est robuste au surapprentissage et sert de modèle de comparaison fiable.
- **Isolation Forest** : algorithme de détection d'anomalies qui isole les comportements atypiques en mesurant leur facilité à se séparer du reste des données. Plus un individu est isolé rapidement, plus il est suspect.
- **Régression logistique** : modèle supervisé qui estime la probabilité qu'un cas appar-

tienne à une classe (fraude ou légitime). Bien que simple, il offre une excellente interprétabilité — chaque coefficient explique directement son impact.

- **Sur-apprentissage (overfitting)** : situation où un modèle apprend les détails spécifiques des données d'entraînement au lieu des régularités générales. Un modèle surappris performe parfaitement sur les données vues mais échoue sur les nouvelles.
- **Sous-apprentissage (underfitting)** : situation inverse où le modèle est trop simple pour capturer les régularités des données. Un modèle sous-appris performe mal sur l'entraînement et ne généralise pas aux données nouvelles.
- **Validation croisée (cross-validation)** : technique qui divise les données en plusieurs sous-ensembles et entraîne le modèle sur chaque combinaison. Elle offre une évaluation robuste et fiable de la performance réelle du modèle.
- **SHAP (SHapley Additive exPlanations)** : technique d'interprétabilité basée sur la théorie des jeux qui assigne une « contribution » à chaque feature. Elle garantit que les explications de chaque prédiction sont équitables et théoriquement justifiées.
- **Analyse de réseau et PageRank** : approche qui construit un graphe où les nœuds sont des bénéficiaires et les arêtes représentent des connexions (téléphones ou comptes partagés). PageRank attribue un score de centralité qui révèle les réseaux de fraude organisée.
- **Stacking (méta-apprentissage)** : technique qui combine les prédictions de plusieurs modèles en entraînant un méta-modèle pour fusionner les scores optimalement. Elle tire parti des forces de chaque modèle et réduit leurs faiblesses individuelles.
- **Déséquilibre de classes (class imbalance)** : problème courant en détection de fraude où les fraudes sont rares (1-5%) et les données d'entraînement contiennent beaucoup plus d'exemples légitimes. On le résout par des techniques comme SMOTE, l'ajustement des poids de classe, ou l'évaluation sur des métriques appropriées.
- **Calibration du modèle** : processus qui garantit que la probabilité prédite par le modèle (par exemple 0.7) reflète la véritable probabilité empirique de fraude. Un modèle bien calibré fournit des seuils de décision fiables pour les systèmes de production.

2.1.7 Architecture et plateforme

Notre solution s'intègre dans un écosystème technique existant. Les termes suivants décrivent les briques d'architecture utilisées :

- **Moteur de règles (Rule Engine)** : composant logiciel qui évalue un ensemble de conditions prédéfinies par des experts métier (par exemple : « si le bénéficiaire partage son compte bancaire avec 2 autres personnes, déclencher une alerte »). Les règles sont transparentes, auditables et modifiables sans toucher au code.
- **Architecture hybride** : approche qui combine plusieurs techniques complémentaires : règles métier, apprentissage automatique et analyse de graphe, pour couvrir un maximum de scénarios de fraude. Aucune technique seule ne suffit, c'est leur combinaison qui fait la force du système.
- **OpenG2P / Odoo** : OpenG2P est la plateforme open-source de protection sociale sur laquelle notre solution est intégrée. Elle est bâtie sur Odoo 17, un progiciel de gestion intégré (ERP) modulaire et extensible.

- **API REST** : interface de communication standardisée qui permet à différents systèmes d'échanger des données via le protocole HTTP. Notre moteur de fraude expose une API REST pour que d'autres applications (dashboard, OpenG2P, outils de monitoring) puissent lui envoyer des requêtes et recevoir des scores.
- **Pipeline** : chaîne de traitement automatisée où les données passent successivement par plusieurs étapes : extraction des caractéristiques, évaluation par les règles, scoring ML, analyse de graphe, explicabilité, pour produire un résultat final. Le terme « pipeline » évoque l'idée d'un tuyau où les données entrent d'un côté et le score de risque sort de l'autre.

Ces définitions posées, nous disposons maintenant du vocabulaire nécessaire pour examiner en détail chacune des techniques de détection de fraude dans les sections suivantes.

2.1.8 Identification des acteurs

Notre système de détection de fraude interagit avec plusieurs acteurs, qu'ils soient des utilisateurs humains ou des systèmes informatiques. L'identification de ces acteurs est une étape essentielle pour définir les besoins fonctionnels de la solution. Le Tableau 2.1 présente les acteurs internes qui utilisent directement le système, tandis que le Tableau 2.2 décrit les acteurs externes et les systèmes avec lesquels notre solution communique.

Acteur	Rôle
Bénéficiaire	Personne inscrite au programme de protection sociale qui reçoit (ou tente de recevoir) les aides. C'est l'entité analysée par le moteur de détection de fraude.
Agent social	Personnel chargé de l'enregistrement des bénéficiaires dans OpenG2P et de la vérification des informations sur le terrain. Il est le premier point de contact avec le bénéficiaire.
Analyste fraude	Utilisateur qui consulte les alertes générées par le système, examine les explications fournies (SHAP + LLM) et décide des actions à entreprendre : validation, investigation ou blocage du versement.
Administrateur	Responsable de la configuration des règles métier, des seuils de détection et de la gestion technique de la plateforme (utilisateurs, droits d'accès, paramètres ML).

TABLEAU 2.1 – Acteurs internes du système

Acteur / Système	Rôle
OpenG2P / Odoo	Plateforme source qui gère le registre des bénéficiaires, les programmes et les paiements. Elle transmet les données au moteur de détection et reçoit les alertes de fraude en retour.
Moteur intelligent de détection de fraude	Système central qui reçoit les données, applique les quatre couches d'analyse (règles, ML, graphe, anomalie), produit un score de risque et génère une explication en langage naturel.
Organisme de protection sociale	Entité institutionnelle (ministère, agence gouvernementale) bénéficiaire des rapports de fraude, responsable des politiques publiques et des décisions budgétaires.

TABLEAU 2.2 – Acteurs externes et systèmes

2.1.9 Présentation des travaux existants

Avant de développer notre solution technologique, il est essentiel de réaliser un état de l'art des travaux existants dans le domaine de la détection de la fraude sociale. Les tableaux suivants présentent une synthèse de ce panorama sectoriel. Pour plus de clarté, la littérature académique, les solutions commerciales et les architectures DPI ont été séparées. Cette classification permet de mettre en lumière les leviers techniques ainsi que les méthodes analytiques qui caractérisent chaque approche.

TABLEAU 2.3 – Comparaison des méthodes de détection de fraude : avantages et limites

Approche	Principe	Avantages	Limites
Règles métier (rule-based)	Conditions explicites définies par des experts métier (seuils, combinaisons logiques)	+ Transparent et facile à auditer + Exécution rapide, sans entraînement + Pas besoin de données labellisées	– Rigide face aux nouvelles fraudes – Maintenance manuelle lourde – Ne détecte que les schémas connus
ML supervisé (XGBoost, RF)	Apprentissage à partir d'exemples étiquetés fraude / légitime	+ Détecte des schémas subtils et complexes + Bonne précision globale (AUC élevé) + S'améliore avec le volume de données	– Nécessite des données labellisées – Risque de fuite de données – Peu explicable par défaut
Détection d'anomalies (Isolation Forest)	Identification de comportements atypiques sans étiquettes, par isolement statistique	+ Détecte la fraude incon nue + Aucun label requis (non supervisé) + Complémentaire au ML supervisé	– Taux de faux positifs élevé – Difficile à interpréter – Sensible au paramétrage
Analyse de réseau (PageRank)	Graphe de liens entre bénéficiaires (téléphone, compte bancaire partagé)	+ Révèle la fraude organisée + Capture la collusion invisible au ML + Vision relationnelle unique	– Coûteux en calcul – Dépend de la qualité des liens – Complexe à mettre en place

TABLEAU 2.4 – Solutions commerciales de détection de fraude sur le marché

Solution	Principe / Utilité	Avantages	Limites
FICO Falcon Fraud Manager	Scoring en temps réel basé sur un consortium de données partagées entre banques et un moteur ML supervisé.	+ Consortium mondial (9 000+ institutions) + Détection temps réel à très haute échelle + AUC ~ 0.95 sur la fraude bancaire	– Coût : 100k–500k+\$/an – Conçu pour la fraude bancaire uniquement – Code propriétaire fermé
Feedzai (RiskOps)	Plateforme IA-native combinant ML supervisé et analyse de graphe pour le scoring de transactions en temps réel.	+ Analyse de graphe intégrée + Architecture API-first moderne + Adaptable à plusieurs canaux de paiement	– Coût : 150k–600k+\$/an – Ciblé paiements / fin-tech – Pas d'intégration G2P possible
SAS Fraud Management	Suite analytique avancée combinant des modèles ensemblistes, des alertes configurables et un moteur d'investigation.	+ Couverture multi-secteurs + Moteur d'investigation intégré + Plateforme cloud SAS Viya	– Coût : 200k–1M+\$/an – Intégration sur mesure complexe – Explicabilité limitée pour le terrain
NICE Actimize	Surveillance des transactions et conformité anti-blanchiment (AML) avec workflows d'enquête intégrés.	+ Leader en conformité réglementaire + Workflows d'investigation complets + Couverture AML et fraude combinée	– Coût : 300k–2M+\$/an – Aucune analyse de réseau (graphe) – Dépendance éditeur très forte
Featurespace (ARIC)	Analyse comportementale adaptative qui modélise le comportement normal de chaque utilisateur et détecte les déviations en temps réel.	+ Détection non supervisée innovante + S'adapte sans réentraînement + XAI avec reason codes	– Coût : 100k–400k\$/an – Ciblé banque et assurance – Pas d'analyse de graphe

TABLEAU 2.5 – Limites des travaux existants face à la philosophie DPI / DPG

Dimension	Ce qui manque dans les solutions actuelles	Problème non résolu face au DPI / DPG
Accessibilité financière	Les solutions commerciales coûtent de 100k\$ à plus de 2M\$ par an , hors de portée des budgets publics en développement.	Contredit l'objectif d'inclusion des DPG : une technologie réservée aux pays riches reproduit les inégalités qu'elle prétend combattre.
Ouverture du code	Les moteurs de détection sont propriétaires et fermés , impossible de les auditer, les adapter ou les réutiliser.	Va à l'encontre du principe fondamental des Biens Publics Numériques : code ouvert, réutilisable et vérifiable par tous.
Souveraineté des données	Les plateformes cloud propriétaires imposent que les données sensibles transitent par des serveurs externes .	Menace la souveraineté numérique des États : les données des bénéficiaires doivent rester sous contrôle national.
Intégration native OpenG2P	Aucune solution du marché n'est conçue pour s'intégrer nativement à OpenG2P ou aux registres sociaux.	Oblige des développements sur mesure coûteux : rompt l'interopérabilité modulaire promue par l'écosystème DPI.
Spécificité de la fraude sociale	Les outils ciblent la fraude bancaire (carte, paiement) et non les fraudes propres au social : multi-inscription, ménages fictifs.	Les schémas de fraude G2P (collusion, partage d'identité) restent mal couverts par des modèles pensés pour la finance.
Explicabilité pour le terrain	Les scores sont produits par des boîtes noires , sans justification compréhensible par un agent social non-technique.	Empêche la redevabilité exigée dans les services publics : une décision d'aide doit pouvoir être expliquée au citoyen.
Proactivité (avant versement)	Les contrôles traditionnels (audits manuels) interviennent après le versement, quand la perte est déjà subie.	Le modèle DPI vise une prévention en amont : l'absence de détection temps réel laisse fuir les budgets avant toute action.

Conclusion

L'examen des travaux et des outils actuels montre que chacune des approches explorées apporte une contribution utile mais reste limitée lorsqu'elle est appliquée isolément aux programmes de protection sociale du contexte des DPI et DPG, spécifiquement OpenG2P. À partir de ces constats, nous élaborons dans le chapitre suivant une spécification détaillée des exigences fonctionnelles et une architecture technologique hybride capable de surmonter ces limitations.

Chapitre 3 Spécifications fonctionnelles et choix technologiques

3.1 Introduction

Forts des constats du chapitre précédent, l'absence de solution satisfaisant simultanément les exigences d'accessibilité, de transparence et de souveraineté. Nous orientons notre conception vers une approche systématique et traçable. Ce chapitre traduit ces apprentissages en spécifications formelles qui guident la construction de notre moteur intelligent de détection de fraude.

3.2 Identification des besoins fonctionnelles

TABEAU 3.1 – Spécification des besoins fonctionnels du moteur intelligent de détection de fraude par acteur

ID	Acteur	Besoin fonctionnel
AGENT SOCIAL — Analyse unitaire		
BF01	Agent social	Déclencher un scan unitaire : lancer manuellement l'analyse de fraude sur un bénéficiaire et consulter le score de risque.
BF01	Agent social	Consulter le score de risque : Consulter si le bénéficiaire est classé LOW, MEDIUM, HIGH ou CRITICAL et visualiser les règles déclenchées.
ANALYSTE FRAUDE — Examiner et valider les alertes		
BF02	Analyste fraude	Consulter les alertes de fraude : visualiser liste des cas détectés avec score, niveau de risque et règles déclenchées.
BF03	Analyste fraude	Examiner les explications : consulter facteurs SHAP et explication en français générée par le LLM.
BF04	Analyste fraude	Donner un feedback : valider ou infirmer une alerte (fraude confirmée, faux positif, indéterminé) pour boucle d'amélioration.
BF05	Analyste fraude	Exporter un rapport d'investigation : générer rapport en PDF ou CSV pour un cas de fraude.
BF06	Analyste fraude	Versionner les modèles : consulter historique MLflow, comparer performances et effectuer un rollback.

ID	Acteur	Besoin fonctionnel
BF07	Analyste fraude	Consulter le tableau de bord : visualiser indicateurs (nombre cas, répartition par risque, tendances, taux faux positifs).
BF08	Analyste fraude	Bloquer ou authentifier les bénéficiaires : visualiser indicateurs (nombre cas, répartition par risque, tendances, taux faux positifs).
BF09	Analyste fraude	Suivre la performance du modèle : consulter métriques ML (AUC, précision, rappel) et détecter dégradation.
ADMINISTRATEUR — Configurer le moteur		
BF10	Admin	Gérer les règles métier : ajouter, modifier, activer/désactiver règles de détection (seuils, poids, conditions).
BF11	Admin	Recharger les règles en temps réel : appliquer modifications sans redémarrer le système.
BF12	Admin	Réentraîner le modèle ML : déclencher réentraînement avec nouvelles données et feedbacks collectés.
Traiter et générer automatiquement		
BF113	Système	Scanner en masse : analyser automatiquement l'ensemble des bénéficiaires en un seul appel (scoring batch).
BF14	Système	Scoring temps réel : déclencher automatiquement l'analyse à chaque nouvel enregistrement via webhook.
BF15	Système	Calculer le score hybride : combiner quatre sous-scores (règles, ML, anomalie, graphe) en score final unique.
BF16	Système	Générer l'explication : produire automatiquement facteurs SHAP et explication en français via LLM Mistral.
BF17	Système	Synchroniser avec Odoo : créer automatiquement cas de fraude dans module Odoo et mettre à jour leur statut.
BF18	Système	Envoyer des alertes : notifier analystes lorsqu'un cas CRITICAL ou HIGH est détecté.

3.3 Identification des besoins non fonctionnels

Les besoins non fonctionnels concernent les contraintes à prendre en considération pour mettre en place une solution adéquate aux attentes des clients. Notre plateforme doit nécessairement assurer ces besoins :

- **Sécurité** : La plateforme doit garantir la confidentialité et l'intégrité des données personnelles des bénéficiaires, en conformité avec les réglementations en vigueur (ex : RGPD)

- **Scalabilité** : La solution doit être capable de gérer un nombre croissant de bénéficiaires et de transactions sans dégradation des performances.
- **Accessibilité** : L'interface utilisateur doit être intuitive et accessible à des agents sociaux ayant des compétences techniques limitées.
- **Interopérabilité** : Le moteur de détection doit s'intégrer facilement avec d'autres systèmes utilisés par les organismes de protection sociale.
- **Maintenabilité** : Le code doit être bien structuré, documenté et modulaire pour faciliter les mises à jour et les évolutions futures.
- **Modularité** : Organisation claire des différentes fonctions.

3.4 Diagramme de cas d'utilisation général

Pour mettre ces besoins en perspective, nous présentons un diagramme de cas d'utilisation général qui illustre les interactions entre les différents acteurs et le système pour répondre aux besoins identifiés. Ce diagramme permet de visualiser les fonctionnalités principales du moteur de détection de fraude et la manière dont chaque acteur interagit avec le système pour accomplir ses tâches spécifiques.

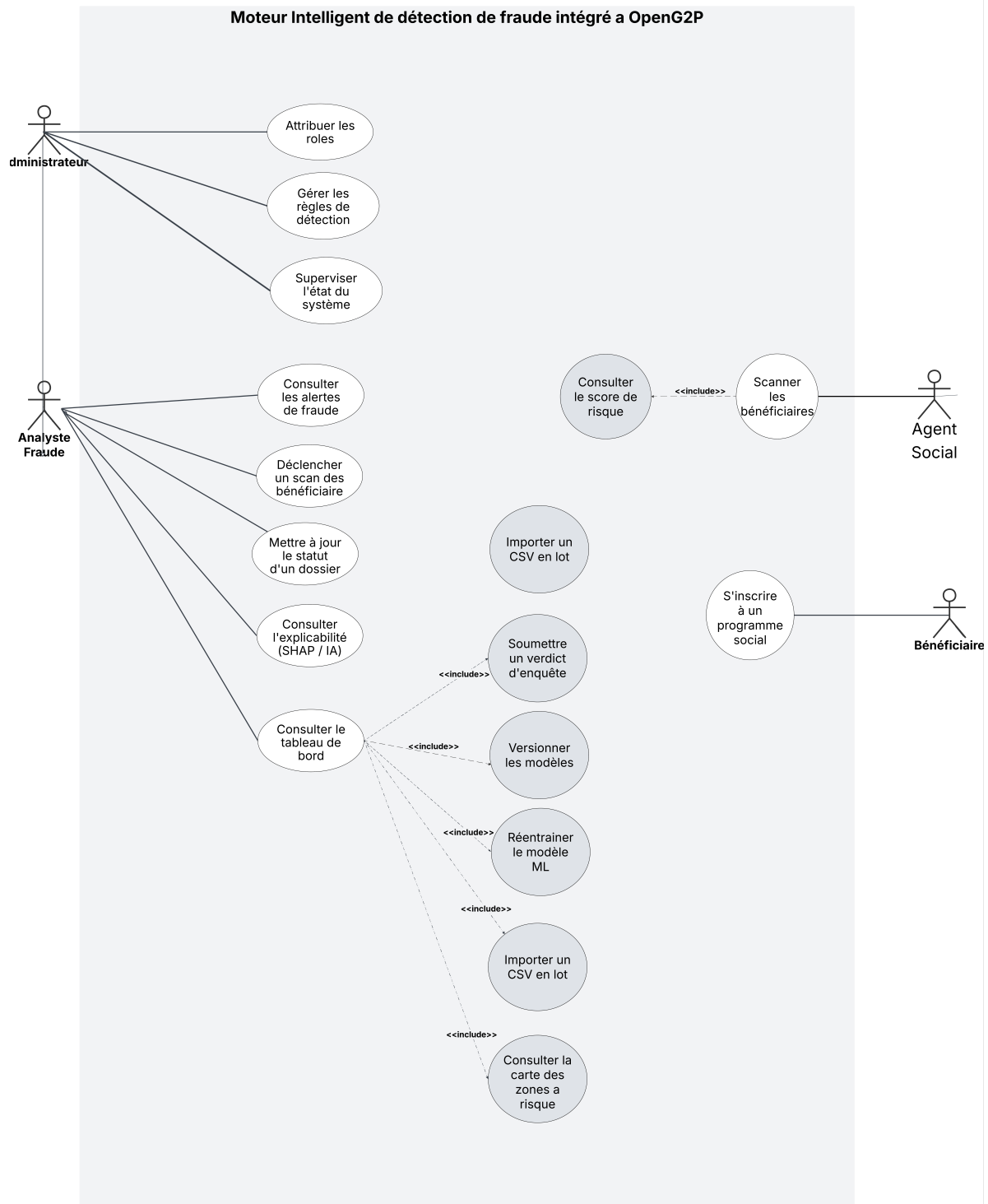


FIGURE 3.1 – Diagramme de cas d'utilisation général

3.4.1 Description du diagramme de cas d'utilisation

Ce diagramme présente les quatre acteurs qui interagissent avec notre système, ainsi que les actions que chacun peut réaliser. Pour bien comprendre leur articulation, il est utile de suivre le parcours naturel d'un bénéficiaire dans le système, de son inscription jusqu'à la détection éventuelle d'une fraude.

- **Bénéficiaire — le point de départ :**
 - s'inscrit à un programme social pour recevoir une aide financière ;
 - cette inscription est ce qui déclenche toute la chaîne : sans elle, il n'y a ni prestation à verser, ni fraude potentielle à détecter.
- **Agent social OU l'agent de terrain :**
 - scanne les bénéficiaires pour vérifier leur situation.
 - cette action déclenche automatiquement la consultation du score de risque calculé par le système.
 - dès qu'un dossier est contrôlé, l'agent voit immédiatement si le profil est suspect ou non.
- **Analyste fraude OU le superviseur :**
 - déclenche un scan complet des bénéficiaires pour lancer une analyse plus poussée.
 - consulte les alertes de fraude générées automatiquement.
 - comprend pourquoi un bénéficiaire a été signalé, grâce à l'explicabilité IA, une explication claire des facteurs ayant conduit au score.
 - met à jour le statut du dossier pour confirmer ou écarter le soupçon de fraude.
 - dispose d'un **tableau de bord** centralisant des fonctions avancées :
 - * importer des données en lot via un fichier **CSV**.
 - * soumettre le verdict final d'une enquête.
 - * consulter une **carte des zones à risque** pour repérer les régions les plus exposées.
 - * suivre l'évolution des modèles d'IA grâce au versionnage des modèles.
- **Administrateur :**
 - attribue les rôles aux différents utilisateurs (qui a le droit de faire quoi).
 - supervise l'état général du système pour s'assurer que tout fonctionne correctement.
 - consulter des statistiques détaillées sur les performances du moteur de détection.

En résumé, ce diagramme illustre une chaîne logique et progressive : le bénéficiaire déclenche le processus, l'agent social effectue un premier contrôle sur le terrain, l'analyste fraude approfondit l'enquête et prend les décisions, et l'administrateur veille en arrière-plan à la fiabilité globale du système.

3.5 Diagrammes de séquence UML

Après avoir présenté les acteurs et leurs fonctionnalités à travers le diagramme de cas d'utilisation, il est utile de montrer comment ces fonctionnalités se déroulent étape par étape, l'ordre exact des échanges entre les différents composants du système lorsqu'un scénario précis se produit. Nous en présentons deux, qui se complètent : le premier détaille le fonctionnement interne du moteur de scoring, tandis que le second décrit la manière dont l'analyste fraude traite ensuite le dossier généré.

- **Diagramme 1 — Calcul du score de fraude (figure 3.2) :**

- l'analyste fraude ouvre le dossier d'un bénéficiaire depuis OpenG2P, ce qui déclenche une requête vers le serveur FastAPI hébergeant le moteur de détection ;
- le **Feature Extractor** extrait les données brutes du bénéficiaire et les transforme en un vecteur de 25 caractéristiques exploitables ;
- ce vecteur est envoyé en parallèle au moteur de règles métier et au pipeline de Machine Learning, qui calculent chacun leur propre score (score de règles et score ML) ;
- ces deux scores sont ensuite fusionnés selon une pondération définie pour obtenir un score final unique ;
- le module SHAP identifie les variables les plus influentes dans ce score, puis le modèle de langage Mistral traduit ces facteurs techniques en une explication rédigée en langage naturel ;
- l'ensemble (score, niveau de risque, explication) est sauvegardé dans la base PostgreSQL, puis renvoyé jusqu'à l'interface Odoo pour affichage à l'analyste.

- **Diagramme 2 — Traitement du dossier par l'analyste (figure 3.3) :**

- une fois le score calculé par le moteur (diagramme précédent), l'analyste ouvre le dossier signalé et consulte les détails ainsi que les preuves associées ;
- après cette analyse manuelle, deux chemins sont possibles : si les données sont insuffisantes ou qu'aucune action n'est requise, l'analyste rejette directement le dossier, sans qu'aucun retour ne soit envoyé au moteur d'apprentissage ;
- si l'analyste choisit au contraire de poursuivre son investigation, le comportement du système dépend du niveau de risque du dossier :
 - * pour un risque **faible ou moyen**, l'analyste est notifié, consulte l'explication générée par l'IA, prend sa décision, puis peut accepter la demande — ce qui déclenche la poursuite du paiement au bénéficiaire ;
 - * pour un risque **élevé ou critique**, une alerte est envoyée à l'analyste, qui consulte à nouveau le dossier et son explication avant de prendre sa décision — ce qui entraîne le blocage ou la poursuite du paiement.
- dans les deux cas de poursuite de vérification, la base de données est mise à jour à chaque étape clé, et un retour est envoyé au moteur d'apprentissage afin d'améliorer les futures détections.

En résumé, ces deux diagrammes couvrent l'ensemble du cycle de vie d'un dossier de fraude.

Processus globale d'une demande

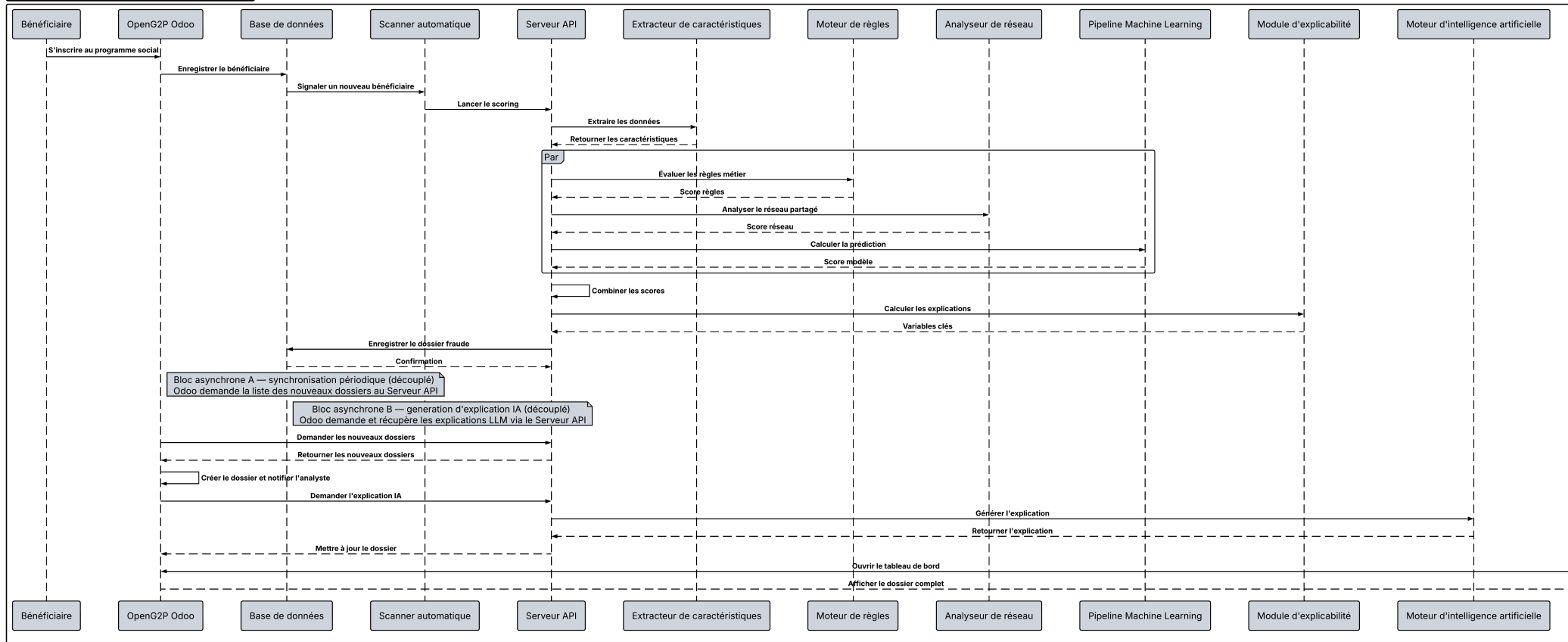


FIGURE 3.2 – Diagramme de séquence des interactions entre les composants du moteur intelligent de détection de fraude

Processus d'acceptation ou de refus d'un dossier de fraude

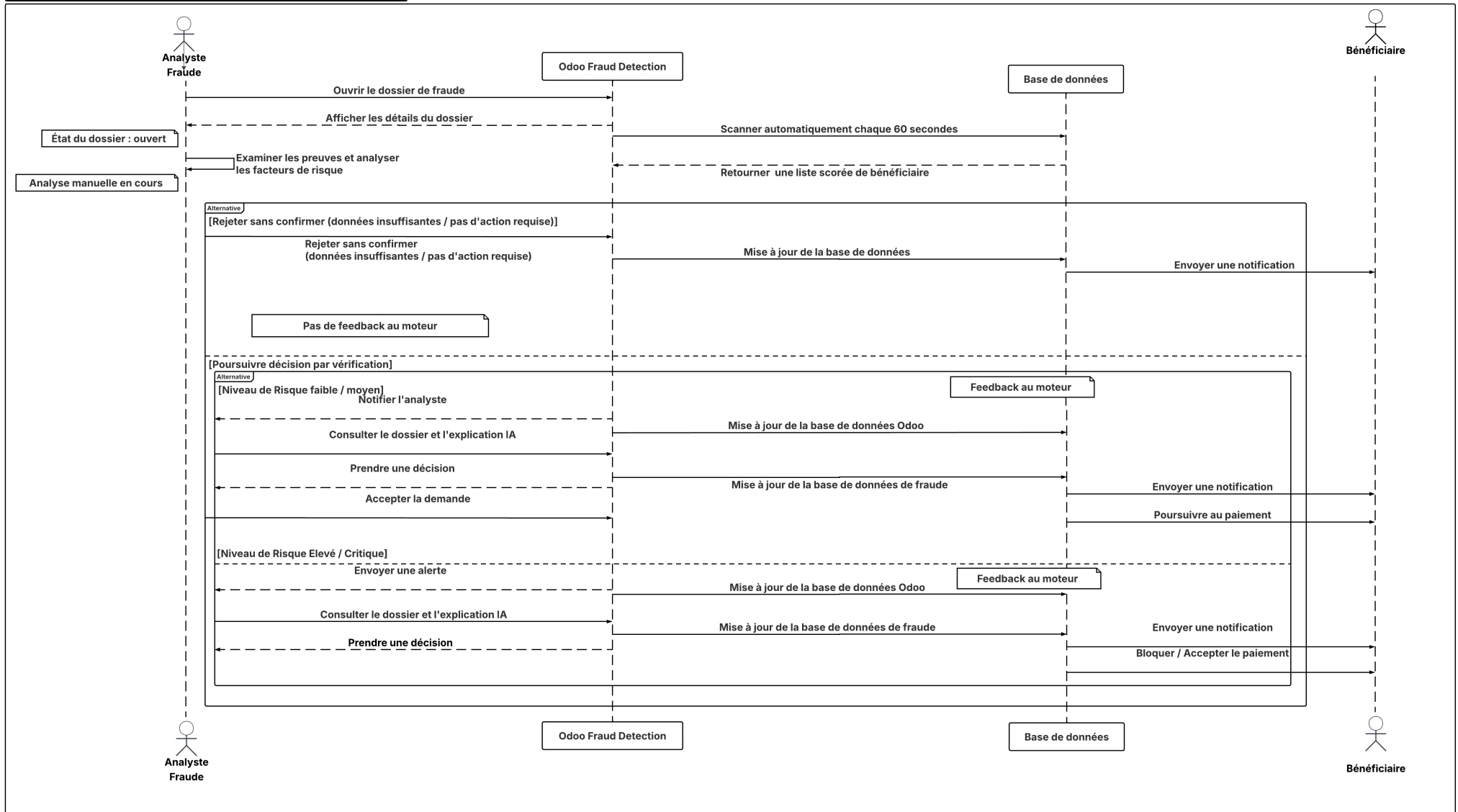


FIGURE 3.3 – Diagramme de séquence : processus d'acceptation ou de refus d'un dossier par l'analyste fraude

3.6 Architectures des composants du moteur de détection de fraude

Afin d'approfondir l'analyse technique, cette section détaille le fonctionnement interne de chaque agent.

3.6.1 Pipeline global du moteur de détection (agrégation)

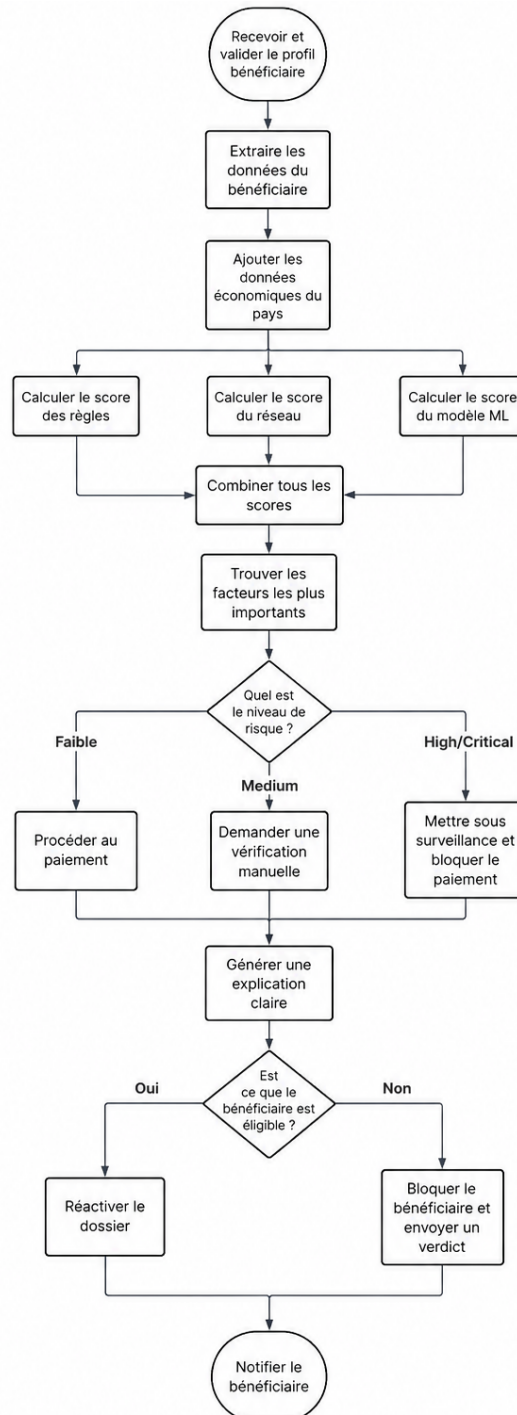


FIGURE 3.4 – Pipeline global d'orchestration du moteur de détection de fraude

Ce pipeline décrit le fonctionnement global du système. Les données du bénéficiaire sont extraites, puis trois scores (règles, réseau et modèle ML) sont calculés en parallèle avant d'être

fusionnés en un score de risque unique.

3.6.2 Architecture des composants du moteur de détection

Après avoir présenté le fonctionnement global du moteur de détection à travers le pipeline d'orchestration (figure 3.4), cette section montre comment ce fonctionnement est organisé dans le code. Le diagramme de classes présente les principaux composants du moteur, leurs responsabilités ainsi que les relations qui les relient. Chaque composant remplit une tâche précise et collabore avec les autres pour produire la décision finale. Le tableau 3.2 résume le rôle des principaux composants représentés dans le diagramme.

Composant	Rôle
DecisionOrchestrator	Coordonne tout le processus de détection. Il appelle les différents composants, rassemble leurs résultats et produit la décision finale.
FeatureEngineer	Prépare les données utilisées par le moteur en construisant les caractéristiques (features) du bénéficiaire.
BeneficiaryExtractor	Récupère les informations du bénéficiaire depuis la base de données OpenG2P.
RuleService	Calcule le score obtenu à partir des règles métier.
GraphAnalyzer	Analyse les relations entre bénéficiaires afin d'identifier des connexions ou des comportements suspects.
RelationshipExtractor	Récupère les liens entre bénéficiaires utilisés par le module d'analyse de réseau.
MultiModelScorer	Calcule le score de fraude à partir de plusieurs modèles de Machine Learning.
BaseModel	Interface commune utilisée par les modèles de prédiction.
RandomForestModel, LogisticRegressionModel, XGBoostModel	Produisent chacun une estimation de la probabilité de fraude.
IsolationForestModel	Détecte les comportements inhabituels grâce à un modèle de détection d'anomalies.
MetaModel	Combine les résultats des différents modèles afin d'obtenir un score unique (stacking).
Explainer	Explique la décision prise et met en évidence les facteurs qui ont le plus influencé le résultat.
FraudCaseRepository	Enregistre les décisions et conserve l'historique des cas analysés.

TABLEAU 3.2 – Rôle des principaux composants du moteur de détection

Cette architecture repose sur une séparation claire des responsabilités. Chaque composant réalise une tâche spécifique (extraction des données, règles métier, analyse du réseau, modèles de Machine Learning, explication et stockage des résultats). Cette organisation facilite la maintenance du système ainsi que l'ajout de nouveaux modèles ou de nouvelles règles sans modifier le reste du moteur.

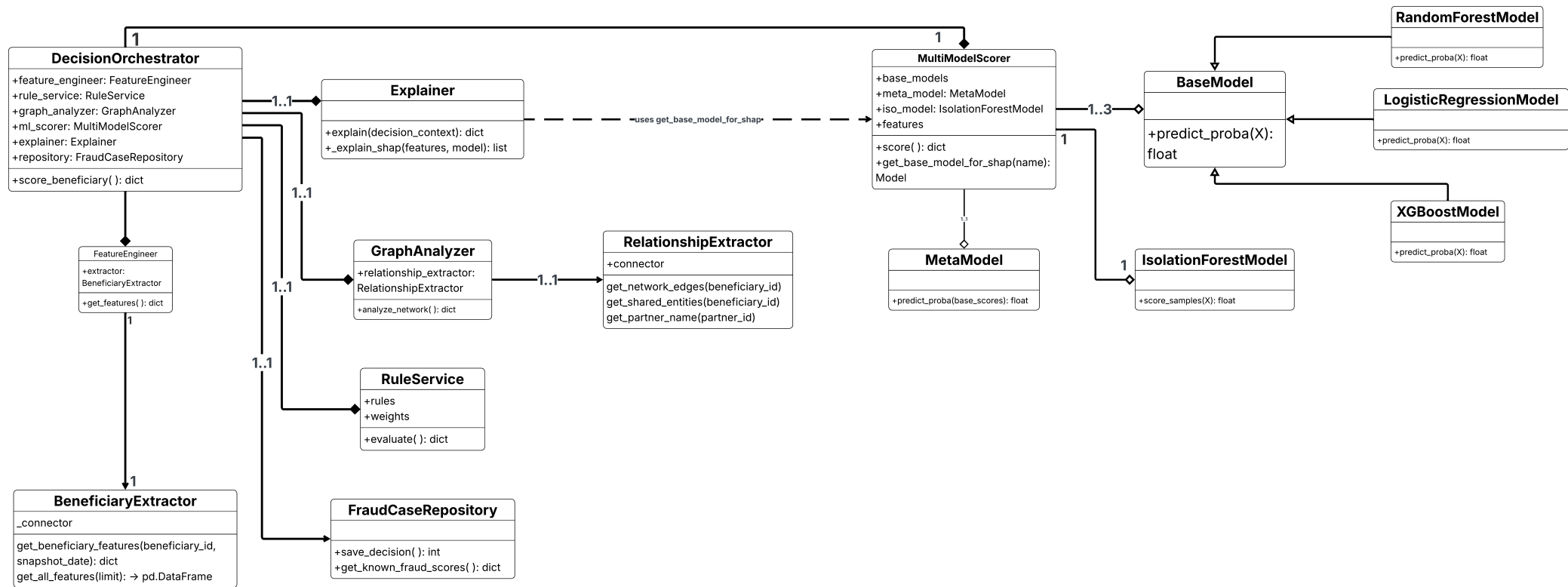


FIGURE 3.5 – Diagramme de classe des principaux composants du moteur de détection de fraude

3.6.3 Moteur de règles métier (Rule Engine)

Le moteur de règles constitue la première couche de détection. Il adapte son évaluation au contexte économique grâce au *GNI per capita* fourni par l'API de la Banque Mondiale. Les caractéristiques socio-économiques du bénéficiaire sont ensuite évaluées par dix règles métier pondérées afin de produire un niveau de risque.

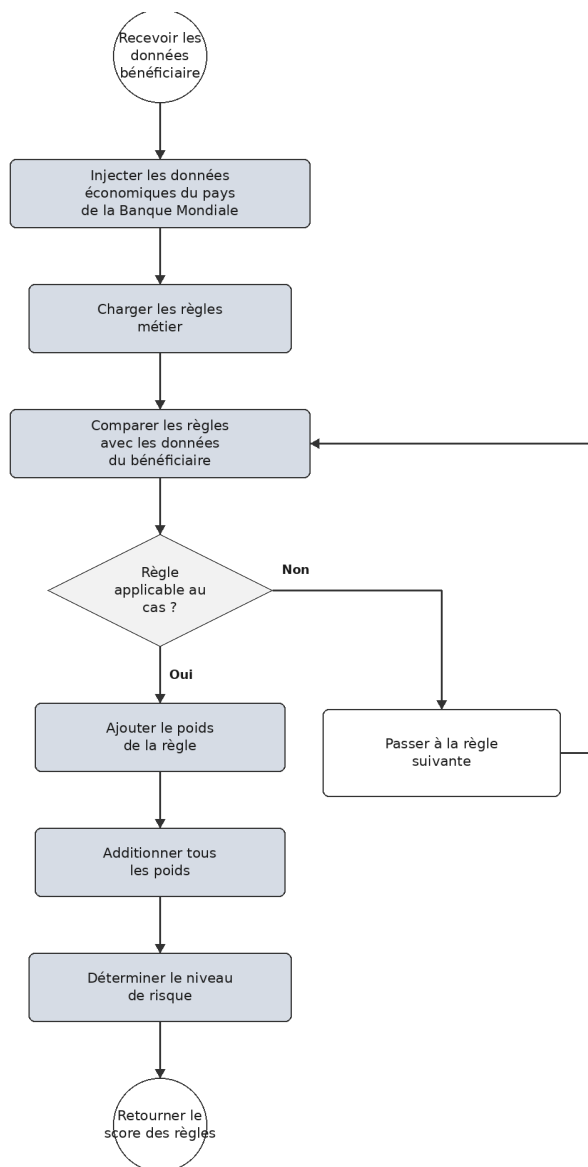


FIGURE 3.6 – Processus d'évaluation du moteur de règles métier

Élément	Rôle
API Banque Mondiale	Fournit le <i>GNI per capita</i> afin d'adapter l'analyse au contexte économique du pays.
Seuil de pauvreté	Référence utilisée pour détecter les sous-déclarations de revenus.
Revenu médian	Permet de contextualiser le revenu déclaré selon le pays de résidence.
Critères analysés	Revenu, taille du ménage, score PMT, nombre de programmes, comptes et téléphones partagés.
Score final	Somme pondérée des règles satisfaites, convertie en niveau de risque (Low, Medium, High ou Critical).

TABEAU 3.3 – Éléments clés du moteur de règles

3.6.4 Analyseur de réseau (Graph)

L'analyseur de réseau examine les liens entre bénéficiaires, notamment le partage de numéros de téléphone ou de comptes bancaires. Il construit un graphe relationnel et calcule l'importance de chaque nœud pour repérer les bénéficiaires fortement connectés à d'autres, un signe potentiel de fraude organisée en réseau.

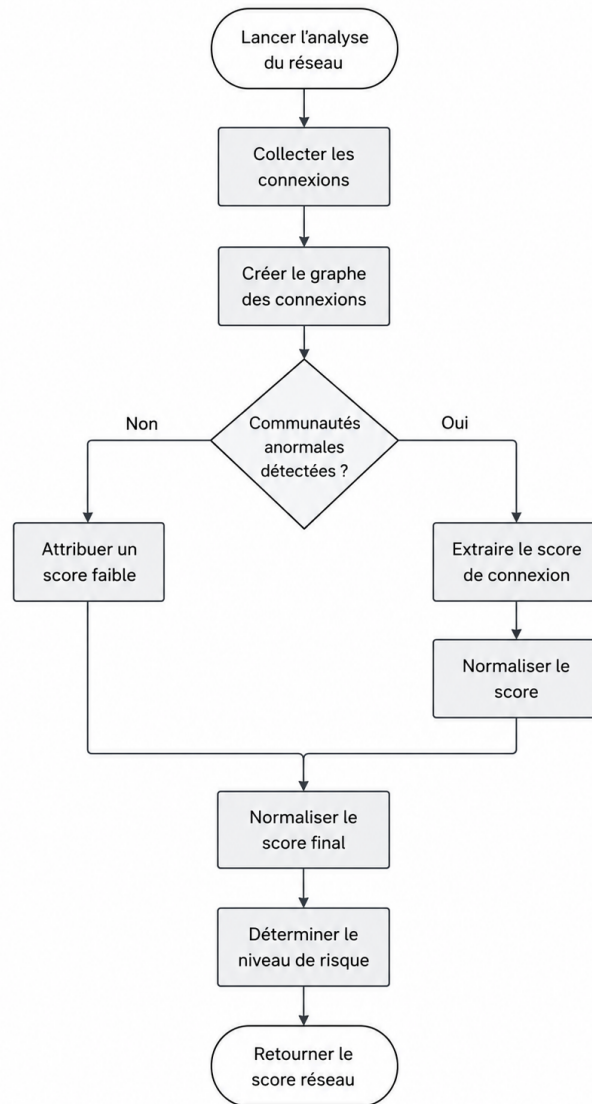


FIGURE 3.7 – Processus d'analyse du réseau relationnel des bénéficiaires

3.6.5 Explicabilité et génération du texte (SHAP + LLM)

Ce processus se déroule en deux temps. D'abord, le module SHAP analyse la contribution de chaque variable au score et en sélectionne les facteurs les plus déterminants. Ensuite, ces facteurs techniques sont simplifiés et transmis à un modèle de langage (Ollama), qui génère une explication en français compréhensible pour l'analyste.

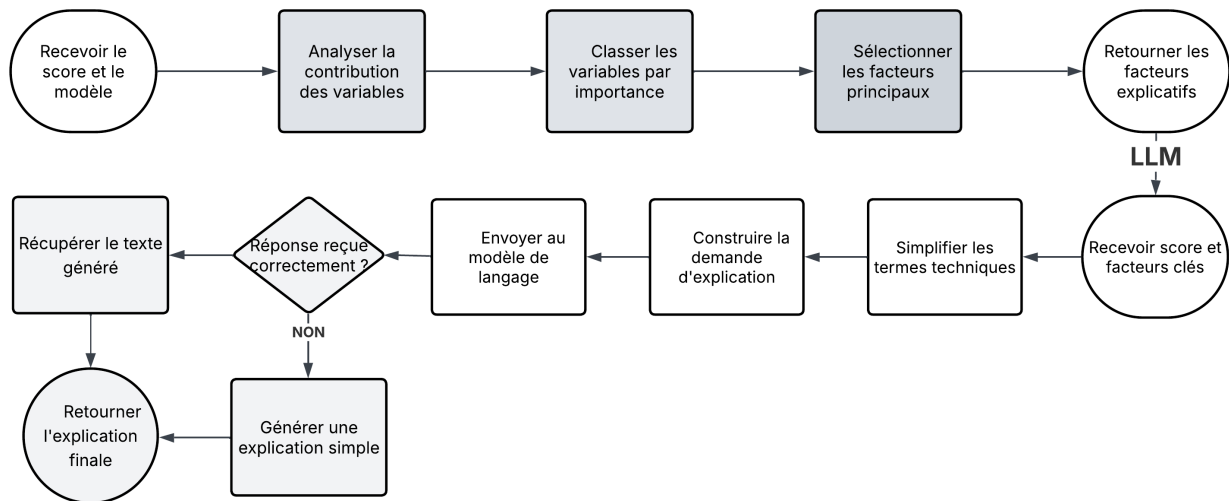


FIGURE 3.8 – Processus d’explicabilité SHAP et de génération d’explication par LLM

3.6.6 Organisation de flux de données

Pour visualiser convenablement l’échange entre ces classes et ces acteurs, nous présentons le diagramme de flux de données. Ce diagramme illustre comment les données circulent à travers le système, depuis la collecte initiale jusqu’à la génération des alertes et des rapports. Il met en évidence les points d’entrée et de sortie des données, ainsi que les transformations et traitements appliqués à chaque étape du processus.

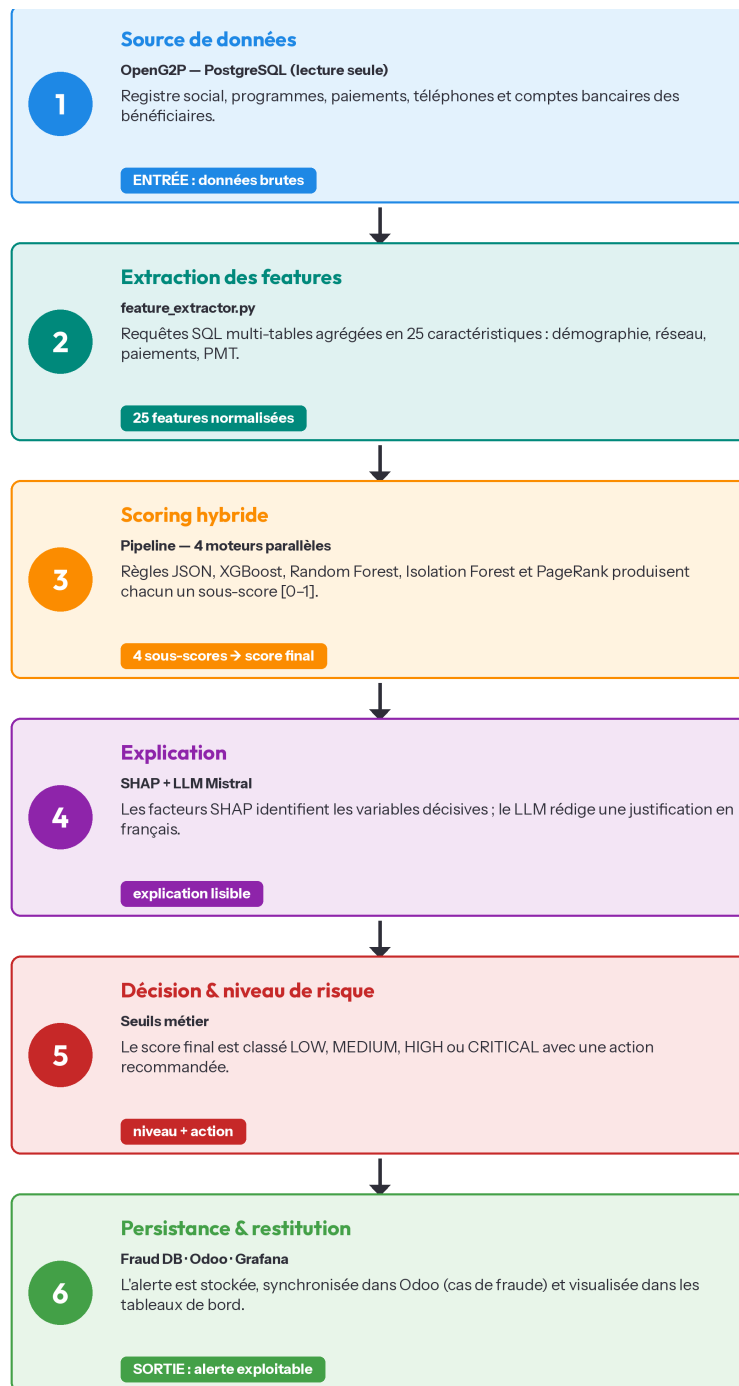


FIGURE 3.9 – Diagramme de flux de données du moteur intelligent de détection de fraude

3.6.7 Architecture physique et logique de la solution

Avant de détailler la mise en œuvre technique du système, il est utile de distinguer deux angles de lecture complémentaires. L'**architecture physique** répond à la question « où les composants s'exécutent-ils ? » 3.6 : elle décrit les machines, serveurs et services réels qui hébergent l'application, ainsi que les échanges réseau entre eux. L'**architecture logique**, quant à elle, répond à la question « comment les données circulent-elles et se transforment-elles ? » : elle décrit les modules fonctionnels du système et la façon dont ils collaborent, indépendamment de la machine sur laquelle ils tournent. Présenter ces deux vues permet au lecteur de comprendre à la fois *où* notre solution est déployée et *comment* elle traite l'information, depuis les données brutes d'OpenG2P jusqu'au verdict de fraude produit par le moteur d'analyse.

- **Architecture physique** : elle retrace le parcours matériel de la donnée, depuis la base PostgreSQL d'OpenG2P jusqu'au moteur de détection de fraude. Le poste de travail de l'utilisateur (**agent social ou analyste fraude**) communique avec le serveur d'application Odoo, lequel s'appuie sur le serveur de base de données pour stocker et interroger les informations des bénéficiaires. Ces données sont ensuite acheminées, via le réseau interne orchestré par Docker Compose, vers le serveur FastAPI hébergeant le moteur de fraude, qui sollicite à son tour les services de calcul (modèles ML, base vectorielle ChromaDB, serveur Ollama pour le LLM local). Cette vue matérielle permet d'identifier concrètement les machines impliquées, les protocoles de communication utilisés et les points de latence ou de défaillance potentiels du système.
- **Architecture logique** : elle décrit le cheminement fonctionnel des données, indépendamment de leur emplacement physique. Les informations brutes des bénéficiaires, extraites d'OpenG2P, sont transformées en un tableau de caractéristiques exploitable. Ce tableau alimente en parallèle quatre modules d'analyse : **moteur de règles métier**, **modèle de Machine Learning**, **détecteur d'anomalies** et **analyseur de graphe relationnel** dont les scores sont ensuite agrégés pour produire un score de risque final. Ce score est enrichi par un module d'explicabilité (**SHAP**) puis traduit en langage naturel par un modèle de langage local, avant d'être restitué à l'analyste fraude sous forme d'alerte exploitable. Cette vue logique met en évidence la logique métier du système et la manière dont chaque composant contribue à la décision finale.

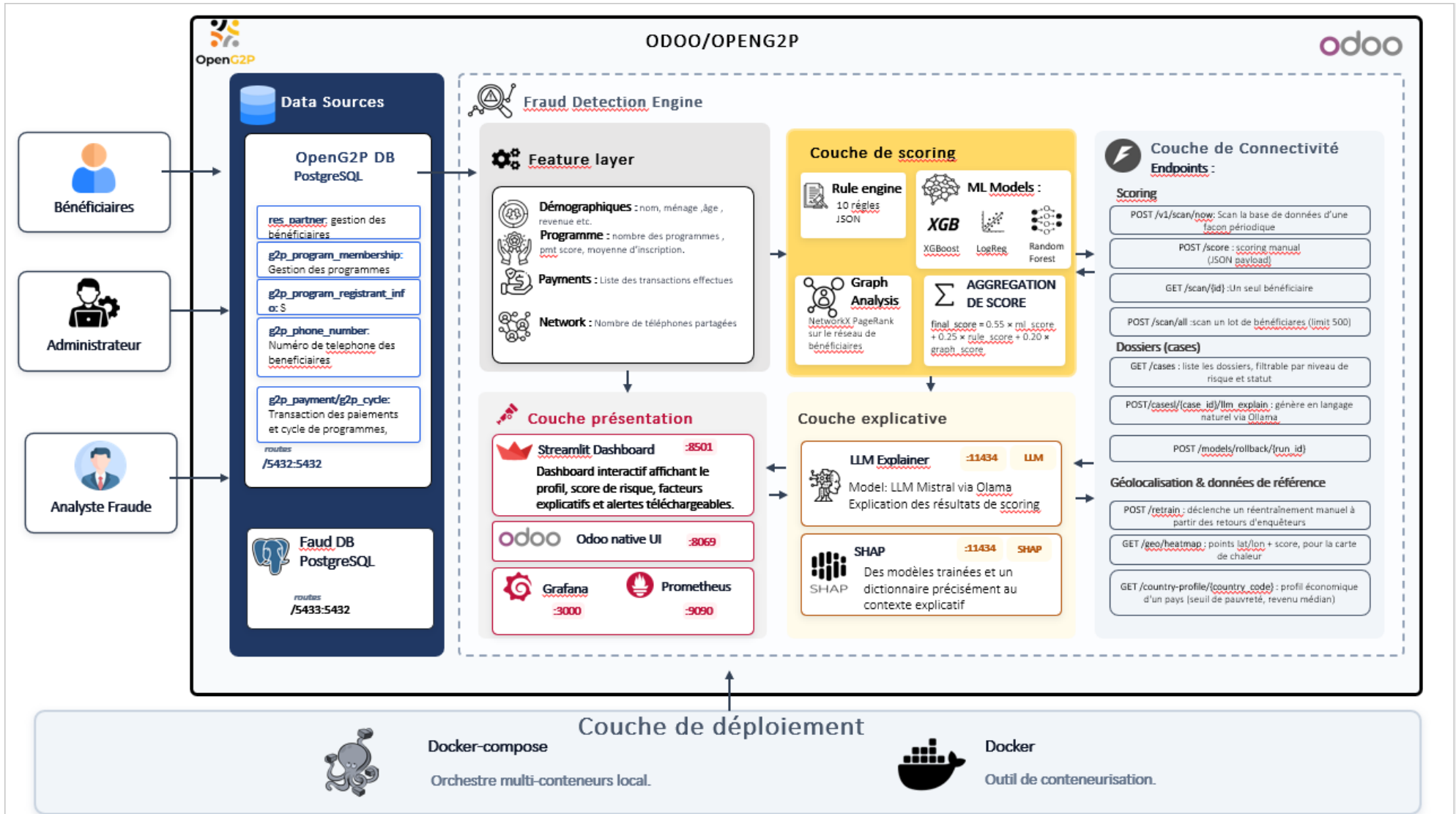


FIGURE 3.10 – Architecture logique du moteur intelligent de détection de fraude

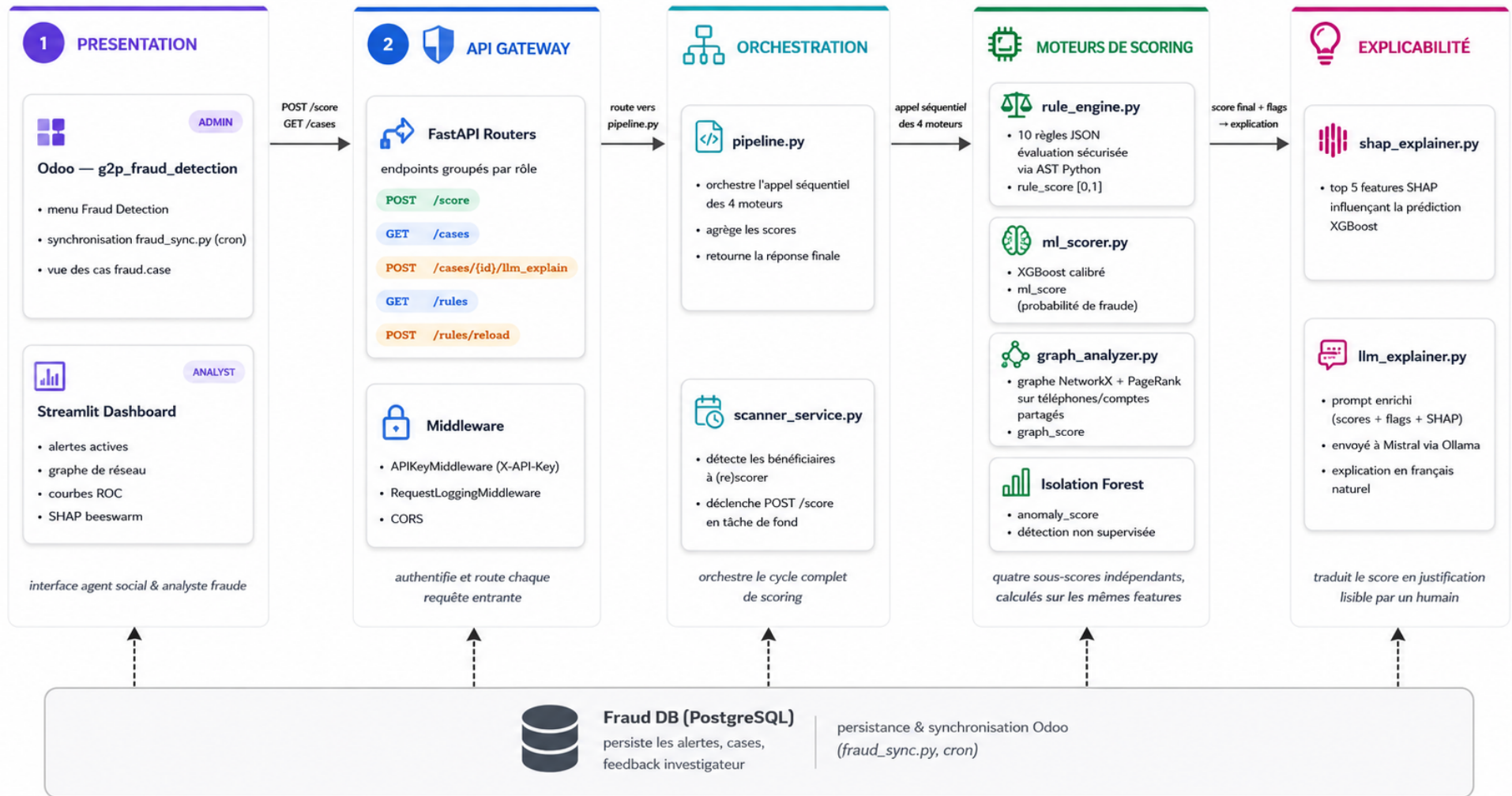


FIGURE 3.11 – Architecture logique du moteur intelligent de détection de fraude

Notre solution s'appuie sur une architecture hybride et modulaire, combinant des composants d'intelligence artificielle (apprentissage automatique, graphes, explicabilité), des règles métier paramétrables, et une infrastructure cloud-native. Cette approche multicouche nous permet de couvrir un maximum de scénarios de fraude tout en restant auditables et transparentes vis-à-vis des parties prenantes.

Après avoir présenté les différents composants du moteur de détection de fraude et leur rôle dans le processus de décision, il est nécessaire de s'intéresser aux technologies ayant permis leur développement. Chaque composant s'appuie sur des outils spécialisés pour assurer le traitement des données, l'apprentissage automatique, l'analyse des relations, l'explicabilité des résultats ainsi que le déploiement de la solution.

3.6.8 Stack Technologique

La Figure 3.12 illustre l'environnement technologique adopté, en regroupant les principaux langages, bibliothèques, frameworks et outils utilisés tout au long du projet.

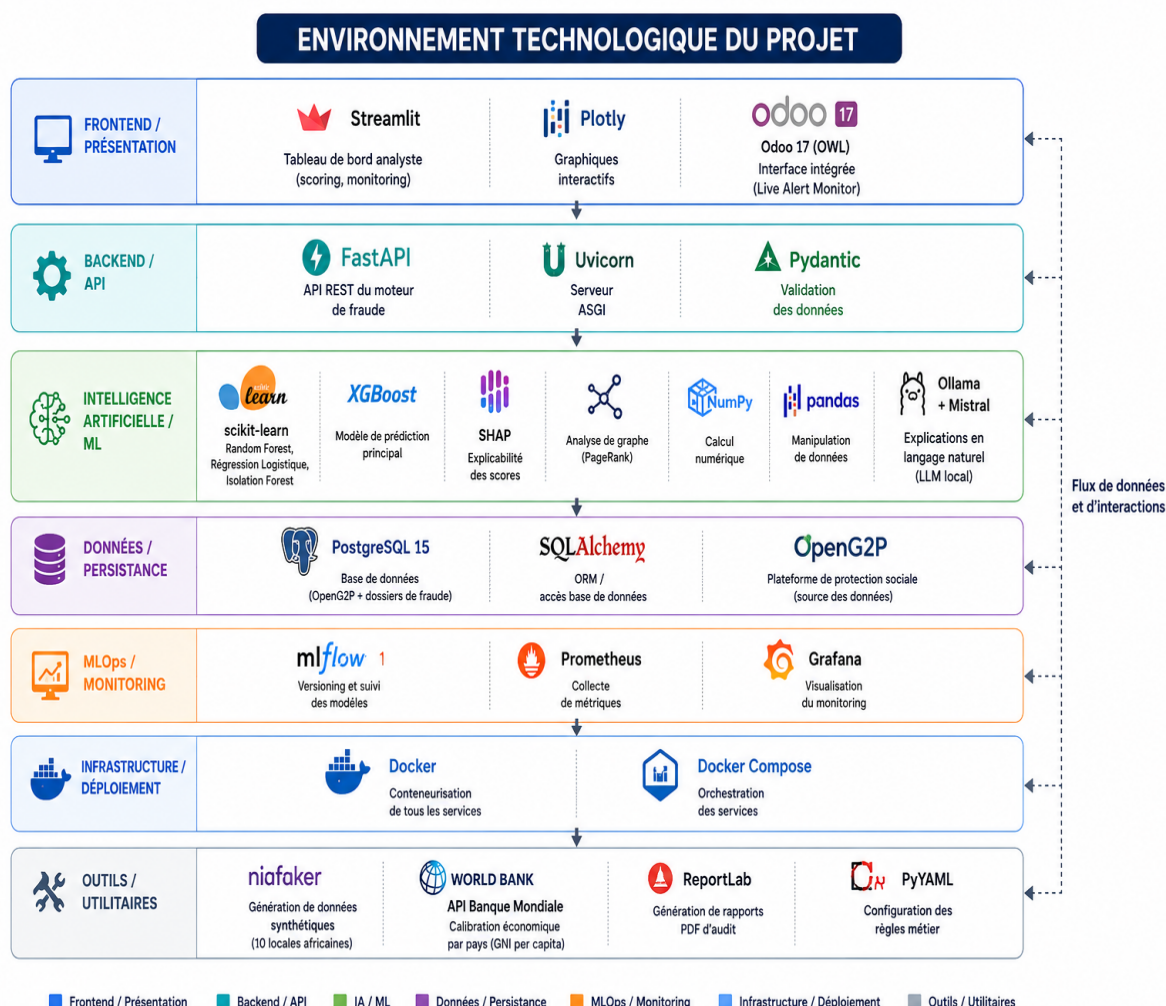


FIGURE 3.12 – Langages et outils utilisés pour le développement du moteur intelligent de détection de fraude

Composant	Technologie / Description
Base de données	PostgreSQL : Système de gestion de base de données relationnelle open source hautement fiable, avec support complet du SQL et des transactions ACID. Utilisé pour persister les données d'enregistrement des bénéficiaires, les alertes de fraude et l'historique des décisions.
Système intégré	OpenG2P / Odoo 17 : Plateforme open-source de gestion de programmes de protection sociale. Manage l'enregistrement des bénéficiaires, l'administration des programmes d'aide et l'intégration de notre moteur de détection de fraude.
ML et graphe	Python (scikit-learn, XGBoost, NetworkX, IsolationForest, RandomForest, Linear Regression) : Écosystème utilisé pour développer les modèles de machine learning, l'extraction de caractéristiques et l'analyse de graphe.

TABLEAU 3.4 – Technologies de base du système de détection de fraude

Couche	Outil / Technologie
Explicabilité	SHAP : Bibliothèque Python pour l'interprétabilité calculant la contribution de chaque caractéristique au score de fraude. Mistral via Ollama : Modèle de langue généraliste exécuté localement pour transformer les coefficients SHAP et règles en explications textuelles en français.
Analyse de graphe	Neo4j : Base de données graphe pour stocker et analyser les réseaux de bénéficiaires (partage de téléphones, comptes bancaires). Permet la détection de communautés frauduleuses et le calcul de centralité (PageRank).
Gestion des modèles	MLflow : Plateforme pour la gestion du cycle de vie des modèles ML. Enregistre les hyperparamètres, métriques de performance (AUC, précision, rappel) et permet un rollback facilité.

TABLEAU 3.5 – Technologies d'explicabilité et de gestion des modèles

3.6.8.1 Stack de développement et DevOps

Pour assurer la qualité, la maintenabilité et le déploiement continu de notre solution, nous avons intégré des outils modernes de développement et d'automatisation :

Catégorie	Outil / Framework
Éditeur de code	Visual Studio Code : Éditeur léger offrant le débogage intégré, autocomplétion intelligente, intégration native de Git et extensions pour Python, SQL et Docker.
Gestion de version	Git / GitHub : Système de contrôle de version pour la collaboration, suivi des changements et gestion des branches. GitHub héberge le dépôt avec workflows d'intégration continue.
Containerisation	Docker : Plateforme de containerisation pour packager l'application avec toutes ses dépendances. Assure la cohérence entre développement, test et production.
Orchestration	Docker Compose : Outil d'orchestration de conteneurs permettant de définir et de lancer l'ensemble de l'infrastructure applicative (backend, base de données, services auxiliaires) via un fichier de configuration YAML unique. Facilite les déploiements reproductibles et le développement local multi-conteneur.

TABLEAU 3.6 – Outils de développement et d'infrastructure

3.6.8.2 Dashboards et visualisation des résultats

Au-delà de la détection elle-même, il est essentiel de fournir aux différents utilisateurs (analystes fraude, superviseurs, administrateurs) des interfaces de visualisation intuitive et interactive. Notre solution intègre plusieurs couches de dashboards et de reporting :

Dashboard / Plateforme	Description et cas d'usage
Streamlit - Inter- face analytique	Plateforme web interactive développée en Python permettant aux analystes fraude et superviseurs de consulter les alertes détectées, visualiser les explications SHAP en temps réel, et accéder à des statistiques globales de fraude. Le dashboard Streamlit offre une réactivité immédiate et permet des filtres dynamiques (par zone géographique, programme, niveau de risque).
MLflow - Suivi des modèles	Tableau de bord dédié au versionnage et au monitoring des modèles ML. Permet de comparer les performances de différentes versions du modèle (AUC, précision, rappel), de visualiser les métriques d'entraînement, et de tracer la provenance de chaque artefact ML. Essential pour la gouvernance et la reproductibilité.
Heatmap géo- graphique	Visualisation cartographique interactive montrant la concentration géographique des alertes de fraude détectées. Chaque zone est colorée proportionnellement au nombre et à la sévérité des cas signalés, permettant aux superviseurs d'identifier rapidement les hotspots de fraude et d'adapter les stratégies d'investigation par région.
Tableau de bord superviseur	Vue synthétique affichant les KPI globaux : nombre total de cas analysés, taux de fraude détecté, proportion de vrais positifs, tendances temporelles, distribution par niveau de risque, et état des investigations en cours. Utilisé par les décideurs pour le monitoring stratégique du programme.
Système de re- porting automa- tisé	Génération automatique de rapports PDF ou CSV consolidant les résultats de détection, les explications SHAP, les analyses de réseau et les recommandations d'action pour chaque bénéficiaire signalé. Ces rapports sont exportables pour audit externe ou communication auprès du management.

TABLEAU 3.7 – Dashboards et plateformes de visualisation

Ces outils de visualisation sont complémentaires aux couches techniques : tandis que MLflow assure la traçabilité des modèles côté développement, Streamlit et les dashboards superviseur fournissent des interfaces métier accessibles aux non-techniciens. La heatmap géographique facilite le diagnostic spatial de la fraude, et le système de reporting garantit que chaque décision prise peut être justifiée et audité.

3.6.8.3 Collaboration et documentation

Enfin, pour faciliter la communication entre les membres de l'équipe et maintenir une documentation à jour :

Fonction	Outil
Tests et validation d'API	Postman / Swagger : Clients HTTP interactifs pour tester manuellement et automatiquement l'API REST du moteur de fraude. Permettent de vérifier les endpoints, les paramètres, les codes de retour et de générer de la documentation API automatique.
Monitoring opérationnel	ELK Stack (Elasticsearch, Logstash, Kibana) : Centralisation et visualisation en temps réel des logs de l'application. Permet de détecter rapidement les erreurs en production, d'analyser les tendances de performance et de diagnostiquer les problèmes.
Documentation technique	Confluence / Wiki GitHub : Dépôt centralisé pour les guides d'installation, les diagrammes d'architecture, les spécifications API complètes, les tutoriels d'utilisation et l'historique des décisions techniques.

TABLEAU 3.8 – Outils de collaboration et de documentation

Cet ensemble complet d'outils et de technologies forme un écosystème cohérent et bien articulé. Chaque composant remplit un rôle spécifique : les technologies de base assurent la performance et la fiabilité, les outils d'explicabilité garantissent la transparence, les dashboards et visualisations rendent les résultats accessibles aux utilisateurs métier, et enfin les outils de collaboration et documentation maintiennent la qualité et la traçabilité du projet.

3.7 Conclusion

Ce chapitre a consolidé les fondations techniques et conceptuelles de notre moteur de détection de fraude pour les programmes de protection sociale. En commençant par une analyse détaillée des acteurs internes et externes, nous avons défini les besoins fonctionnels et non fonctionnels qui doivent guider l'implémentation.

Chapitre 4 Réalisation

Introduction

Ce chapitre présente les différentes étapes de réalisation du moteur intelligent de détection de fraude. Le développement a été mené progressivement, depuis le déploiement de la plateforme OpenG2P jusqu'à l'intégration complète du moteur de détection. Les principales étapes sont les suivantes :

1. **Déploiement de la plateforme OpenG2P** : installation et configuration de l'environnement de travail.
2. **Définition des scénarios de fraude** : identification des principaux cas de fraude à détecter à partir de la littérature.
3. **Analyse de la base de données** : étude de la structure d'OpenG2P et extraction des variables pertinentes.
4. **Génération du dataset synthétique** : création d'un jeu de données réaliste et intégration dans la plateforme.
5. **Entraînement et évaluation des modèles** : apprentissage des modèles de détection et comparaison de leurs performances.
6. **Calcul du score de fraude** : combinaison des différents scores pour produire un score de risque unique.
7. **Intégration du moteur** : connexion du moteur à OpenG2P et présentation des interfaces développées.

Les sections suivantes détaillent chacune de ces étapes. Avant de commencer, la section suivante présente l'environnement technique utilisé pour développer et déployer la solution.

4.1 Déploiement et stabilisation de la plateforme

La toute première condition du projet était de disposer d'une plateforme OpenG2P fonctionnelle et stable : sans elle, aucune donnée à extraire, aucun bénéficiaire à scorer, aucun module de fraude à intégrer. Cette section décrit l'environnement conteneurisé mis en place pour obtenir ce socle reproductible.

4.1.1 Configuration Docker Compose

L'ensemble du système fonctionne via Docker, ce qui garantit un déploiement reproductible et isolé. Il est organisé en trois groupes de conteneurs complémentaires : le **cœur applicatif**, qui exécute la plateforme sociale et le moteur intelligent ; les **services de support**, qui outillent le machine learning et la génération de texte ; et la **pile d'observabilité**, qui surveille en continu la santé de l'ensemble. Le Tableau 4.1 détaille les treize conteneurs orchestrés, leur port d'exposition, l'image de base utilisée et leur rôle respectif dans l'architecture globale.

TABLEAU 4.1 – Conteneurs Docker du système

Conteneur	Port	Image / Base	Rôle
Cœur applicatif			
fraud-engine	8002	Python 3.11 (custom)	API FastAPI — moteur de détection de fraude
openg2p-odoo	8069	openg2p-odoo-sta	Plateforme sociale OpenG2P (Odoo 17)
openg2p-postgres	5432	postgres:15.6	Base de données de la plateforme OpenG2P
fraud-db	5433	postgres:15-alpir	Base de données dédiée au moteur de fraude (dossiers, alertes, historique)
spar-mapper-api	8000	Python (custom)	API de mapping entre les identifiants OpenG2P et les registres externes
spar-bene-portal	8001	Python (custom)	API du portail bénéficiaire (auto-inscription et consultation)
streamlit-dashbo	8501	Python 3.11 (custom)	Dashboard analyste — alertes, scoring, monitoring
Services de support			
mlflow-server	5000	ghcr.io/mlflow	Suivi et versionnage des expériences ML
ollama	11434	ollama/ollama	Serveur LLM local (Mistral) pour les explications en langage naturel
Pile d'observabilité			
prometheus	9090	prom/prometheus	Collecte et stockage des métriques de tous les services
grafana	3000	grafana/grafana	Visualisation : santé des conteneurs et métriques du moteur de fraude
cadvisor	8080	gcr.io/cadvisor	Métriques par conteneur (CPU, mémoire, réseau)
node-exporter	9100	prom/node-exporter	Métriques de la machine hôte (CPU, mémoire, disque)

Cette organisation répond à une exigence d'isolation : OpenG2P et le moteur de fraude disposent chacun de leur propre base de données (**openg2p-postgresql** et **fraud-db**), afin qu'une opération sur l'une n'affecte jamais la disponibilité de l'autre. La pile d'observabilité, quant à elle, ne participe à aucun flux métier : elle se contente d'interroger les autres conteneurs à distance, ce qui garantit qu'une panne de monitoring ne peut jamais faire tomber le système

qu'elle surveille.

Cette orchestration a été pensée pour minimiser la friction de déploiement : une seule commande suffit pour démarrer l'ensemble du système, comme le montre la Figure 4.1.

docker compose -f docker-compose.full.yml up -d

```
PS C:\Users\Mega Pc\Desktop\poc-v2\poc-v2> docker compose -f docker-compose.full.yml up -d
[+] up 11/11
✓ Container fraud-db           Healthy
✓ Container prometheus        Running
✓ Container mlflow-server      Running
✓ Container ollama             Running
✓ Container openg2p-postgresql Healthy
✓ Container openg2p-odoo       Running
✓ Container openg2p-spar-bene-portal-api Running
✓ Container openg2p-spar-mapper-api Running
✓ Container fraud-engine       Running
✓ Container grafana            Running
✓ Container streamlit-dashboard Running
PS C:\Users\Mega Pc\Desktop\poc-v2\poc-v2>
```

FIGURE 4.1 – Démarrage de l'ensemble du système via Docker Compose

4.1.2 Interface de la plateforme OpenG2P après la stabilisation

Une fois l'environnement Docker stabilisé, la plateforme OpenG2P est accessible via son interface web basée sur Odoo 17. Les captures suivantes présentent les principaux écrans utilisés au cours du projet.

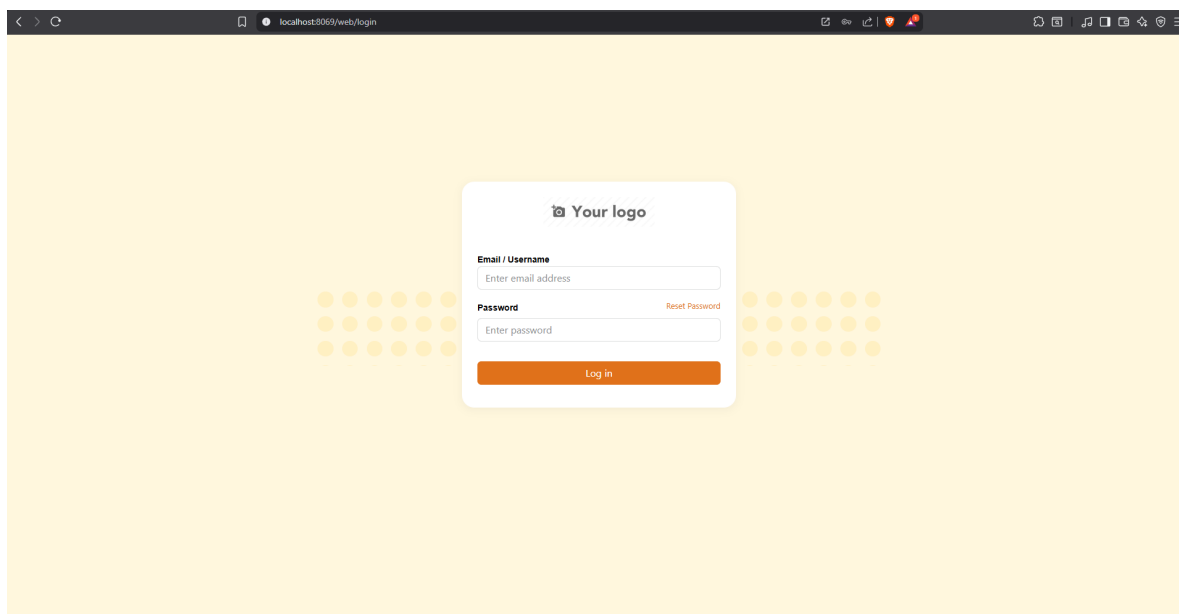


FIGURE 4.2 – Écran de connexion à la plateforme OpenG2P

Cet écran permet aux administrateurs de s'authentifier avant d'accéder aux fonctionnalités de la plateforme.

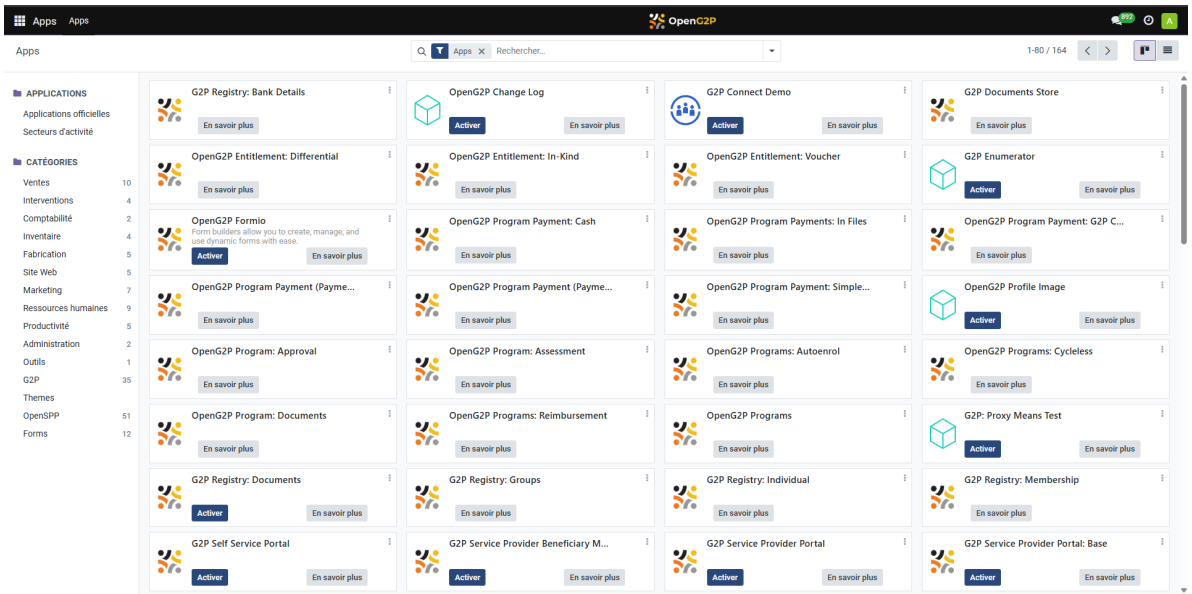


FIGURE 4.3 – Tableau de bord principal après authentification

Le tableau de bord regroupe les principaux modules disponibles pour la gestion de la protection sociale.

Enregistrement						
Nouvelles Personnes						
Rechercher...						
☐	Nom	Adresse	Téléphone	Mots clés	Date de naissance	Date d'inscription
☐	TRABELSI, HALIM	07/				17/07/2026
☐	Tsehay Ibrahim		+251417338484		03/08/1999	01/11/2023
☐	Uwirimana Ingabire		+250789130756		05/05/1963	01/11/2023
☐	Said El Ouazzani		+212697872982		15/09/1975	01/04/2020
☐	Robert Nakiganda		+256700479748		04/07/1982	01/02/2021
☐	Nakayiza Tumwesigye		+256766471027		15/03/1998	01/02/2022
☐	Isaac Kasazi		+256792680182		08/09/1999	01/12/2021
☐	Efua Darko		+216990023		09/04/1976	01/05/2022
☐	Yassine Ben Abdelkader		+212695820297		10/12/1973	01/10/2022
☐	Jibril Hango		+255764868740		14/02/1955	01/07/2021
☐	Claudine Uwera		+250728932460		02/01/1962	01/10/2020
☐	Mithunzi Williams		+27621247826		21/12/1952	01/04/2021
☐	Maame Asante		+233558805929		18/07/1976	01/11/2023
☐	Hilal Ally		+216990011		25/06/1982	01/11/2022
☐	Florence Akello		+256799955271		28/09/1972	01/11/2021
☐	Joseph Darko		+233266071596		24/09/1972	01/12/2022
☐	Fouad Bencheikroun		+216990033		20/02/1951	01/03/2022
☐	Sibongile Ndlovu		+27840494519		24/03/2003	01/06/2020
☐	Rasha Fathy		+201092124998		12/01/2003	01/10/2020
☐	Luthando Molefe		+27731839335		04/12/1990	01/04/2022

FIGURE 4.4 – Liste des bénéficiaires enregistrés dans le registre social

Cette interface permet de consulter et de gérer les informations des bénéficiaires enregistrés.

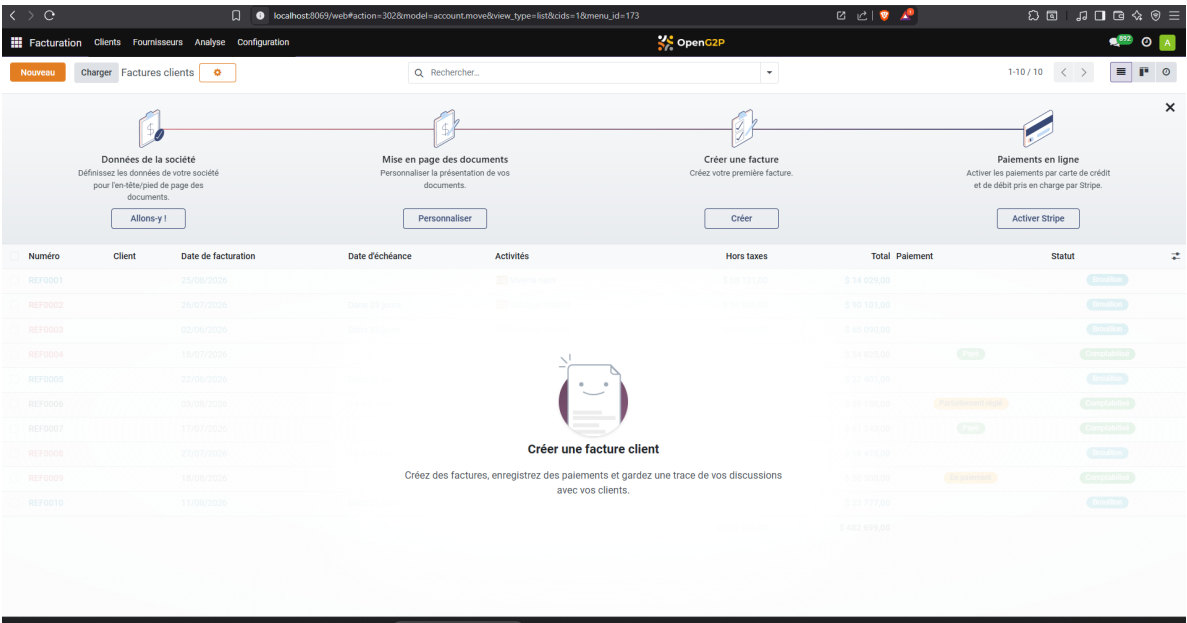


FIGURE 4.5 – Gestion des groupes de bénéficiaires

Les bénéficiaires peuvent être regroupés selon différents critères pour faciliter leur administration.

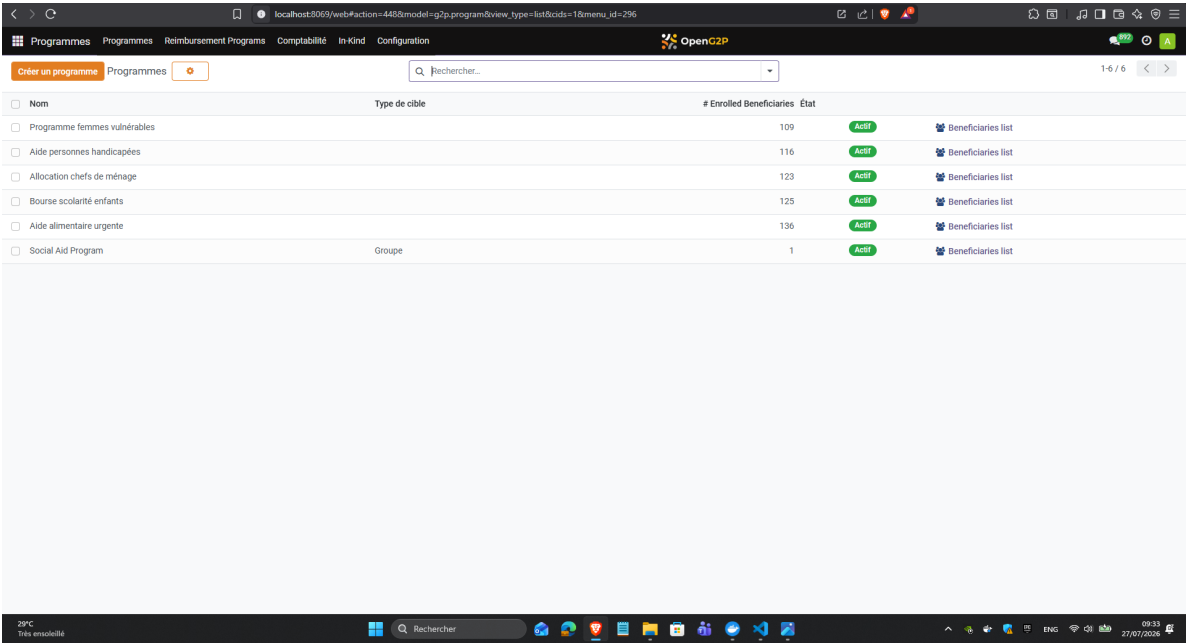


FIGURE 4.6 – Gestion des programmes sociaux et de leurs inscriptions

Cette vue permet de créer les programmes sociaux et de suivre les inscriptions des bénéficiaires.

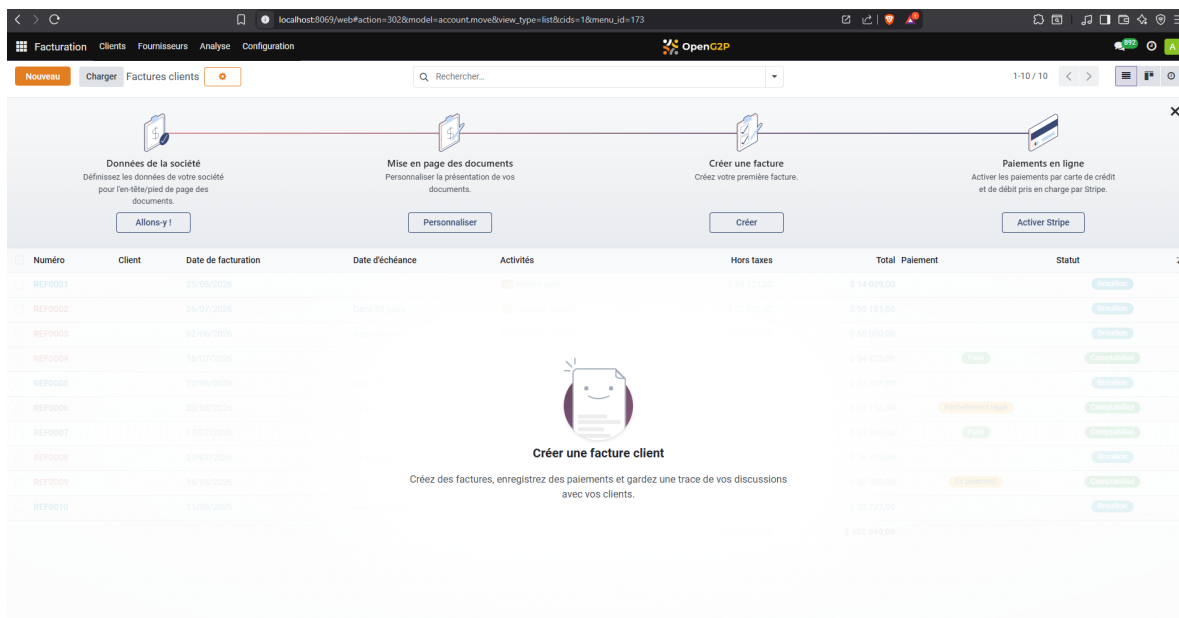


FIGURE 4.7 – Gestion des paiements et des factures

Les opérations de paiement et de facturation sont centralisées dans cette interface.

4.1.3 Interface de monitoring et de supervision

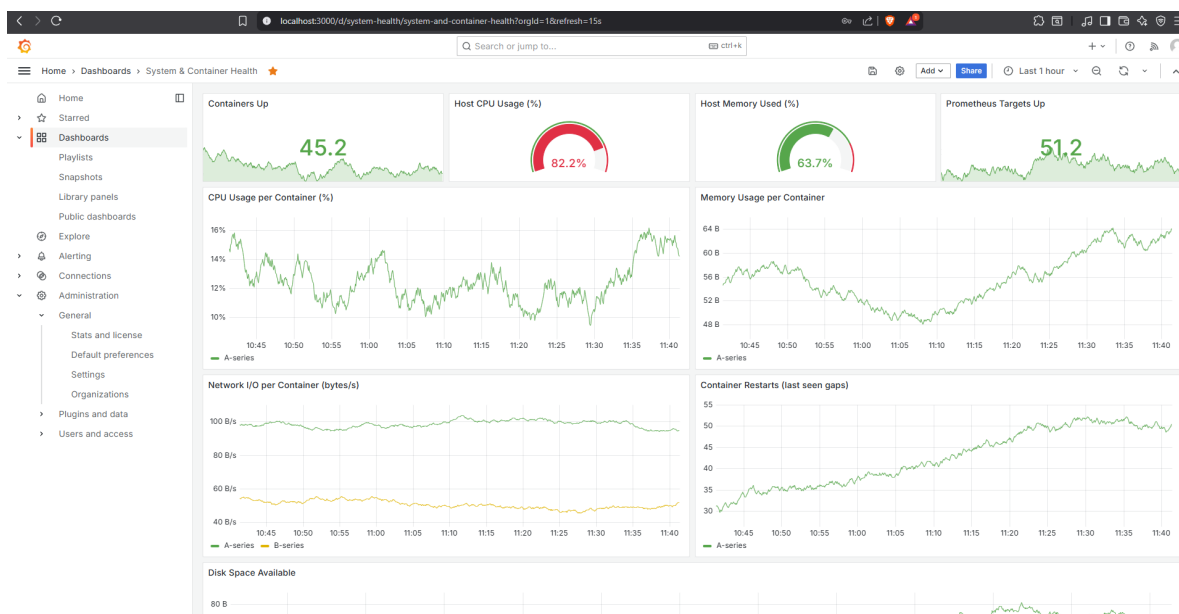


FIGURE 4.8 – Monitoring de l'état des conteneurs et des métriques système via Grafana

Grafana permet de visualiser en temps réel les performances et l'état des différents services de OpenG2p.

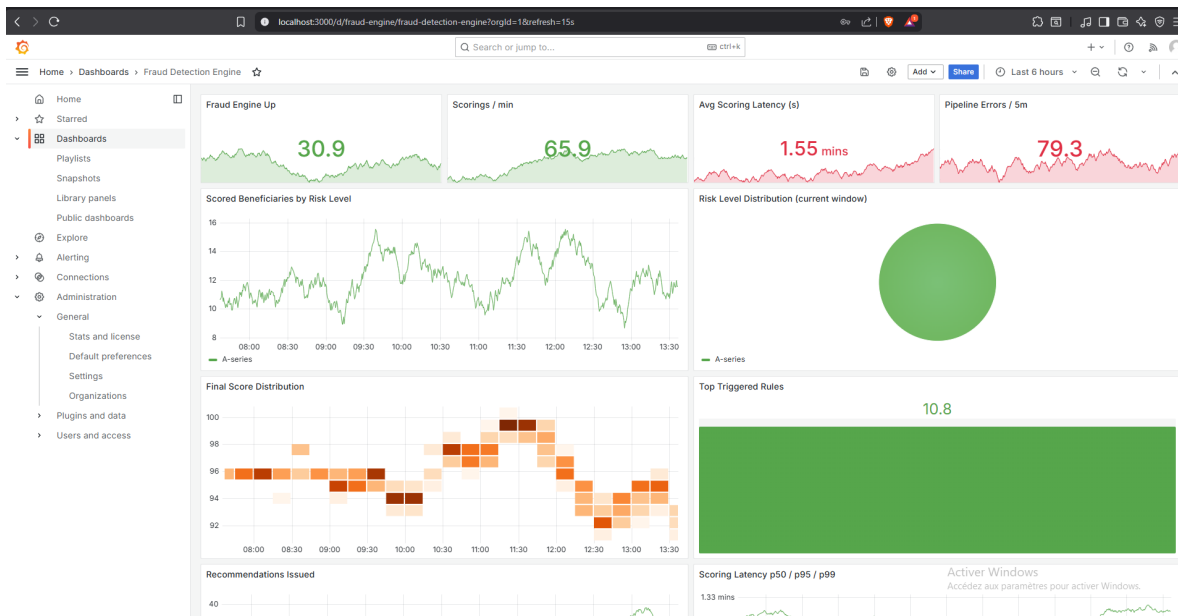


FIGURE 4.9 – Monitoring de l'état du moteur anti-fraude via Grafana

Aussi il permet de visualiser les métriques du système de détection de fraude.

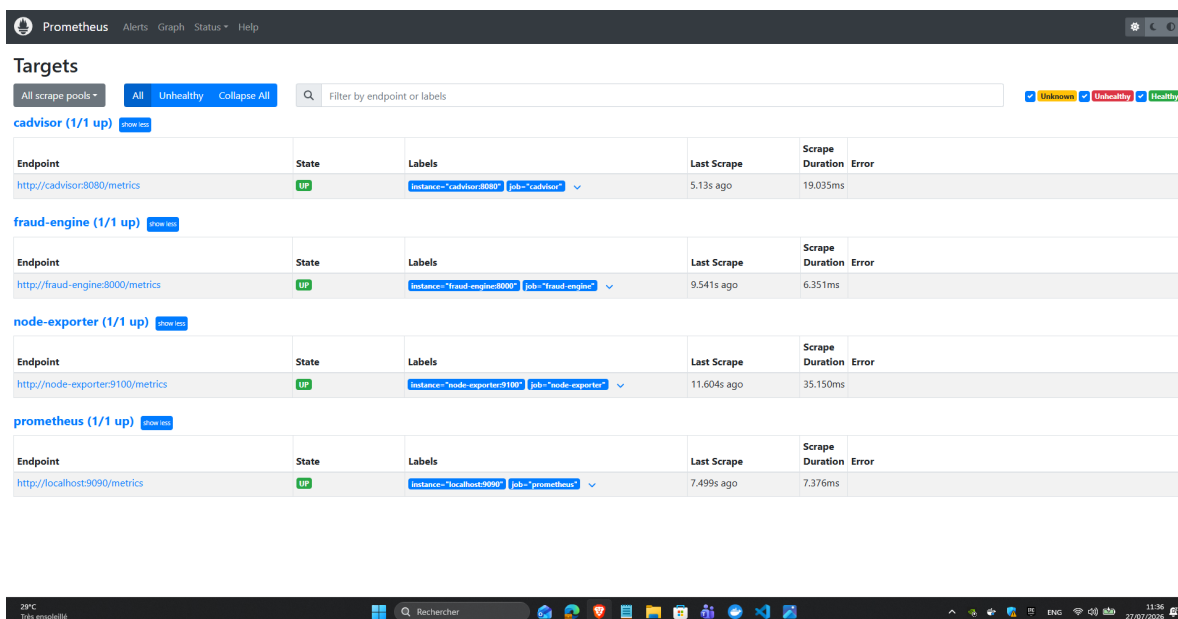


FIGURE 4.10 – Suivi de l'état des services via Prometheus

Prometheus collecte les métriques des conteneurs afin d'assurer le suivi de leur disponibilité.

4.2 Développement des fonctionnalités

4.2.1 Construction du dataset

Le développement du moteur de détection de fraude repose sur des modèles de Machine Learning, dont les performances dépendent directement de la qualité des données utilisées pour leur apprentissage. Or, la plateforme OpenG2P ne contient pas de cas de fraude étiquetés, puisqu'elle a été conçue pour gérer les programmes de protection sociale et non pour enregistrer des fraudes. Il a donc été nécessaire de construire un jeu de données adapté afin de permettre l'entraînement et l'évaluation des modèles.

Pour garantir un dataset représentatif, celui-ci a été élaboré à partir des informations disponibles dans OpenG2P. Les données des bénéficiaires ont été extraites, enrichies et organisées de manière à refléter leur situation et leurs interactions avec la plateforme. La Figure 4.11 présente les principales étapes de ce processus, qui seront détaillées dans les sections suivantes.

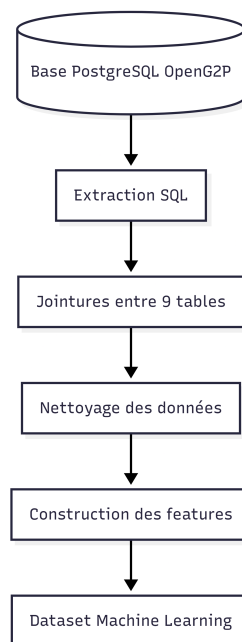


FIGURE 4.11 – Processus de construction du dataset de détection de fraude à partir des données OpenG2P

La construction du dataset débute par l'extraction des données depuis la base OpenG2P. Après les jointures entre les différentes tables et le nettoyage des données, les variables utiles sont construites afin d'obtenir un jeu de données prêt pour l'entraînement des modèles de Machine Learning.

4.2.2 Modélisation et choix des scénarios de fraude

Avant d'extraire la moindre donnée, il fallait d'abord définir précisément l'objectif de détection : quels comportements doivent être reconnus comme frauduleux ? Cette question a été tranchée en s'appuyant sur neuf scénarios de fraude reconnus dans la littérature sur la protection sociale, qui servent de référence pour étiqueter les exemples du dataset d'entraînement :

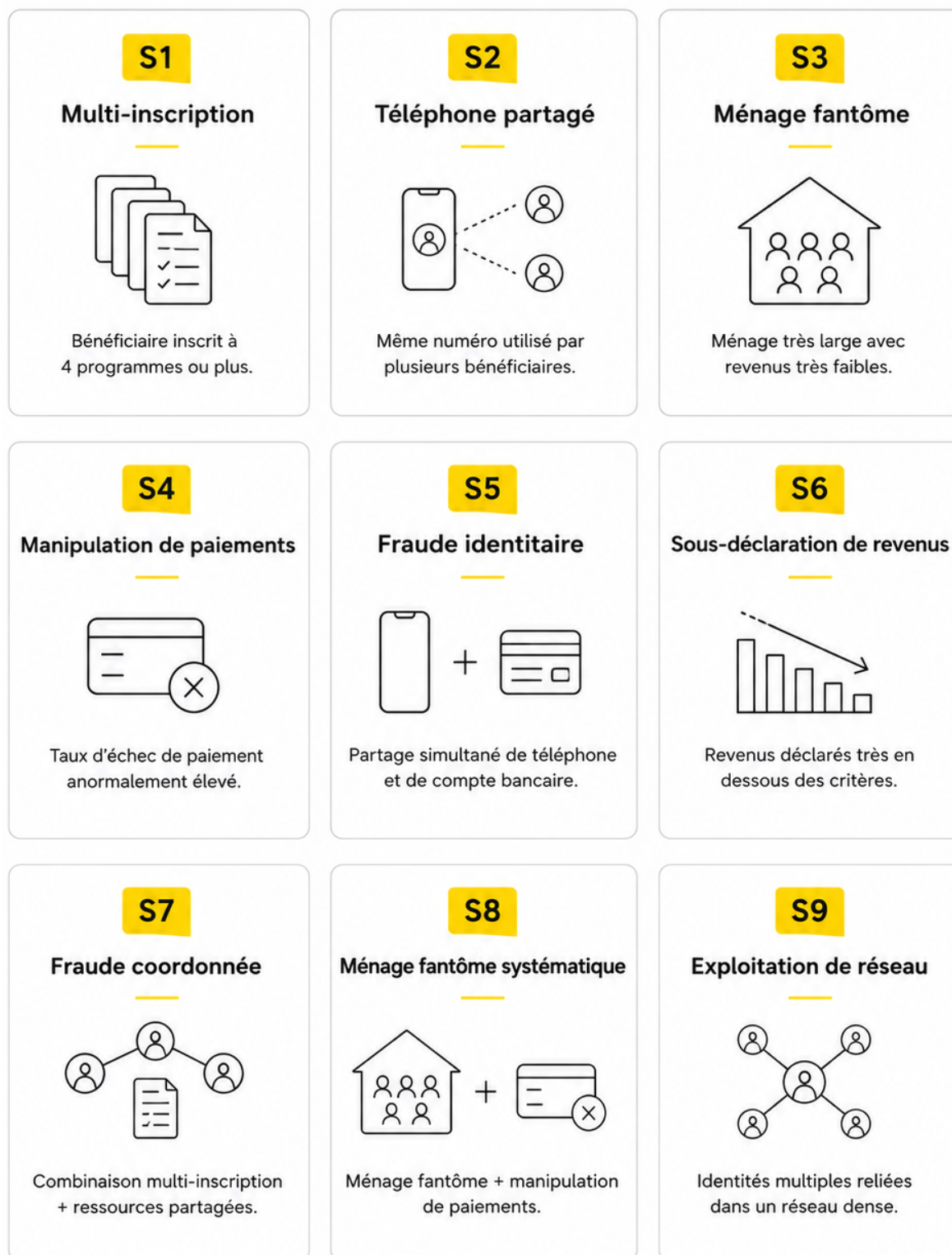


FIGURE 4.12 – Scénarios de fraude simulés

Le Tableau 4.2 décrit ces neuf scénarios en langage simple. Chacun correspond à une manière connue de tromper un programme d'aide sociale, allant de la fraude d'une seule personne jusqu'à la fraude organisée en réseau.

TABLEAU 4.2 – Description des neuf scénarios de fraude simulés

Scénario	En quoi consiste la fraude
Inscriptions multiples	Le bénéficiaire s'inscrit à plusieurs programmes d'aide en même temps afin de cumuler plusieurs aides.
Téléphone partagé	Un même numéro de téléphone est utilisé par plusieurs bénéficiaires, ce qui indique qu'une seule personne contrôle plusieurs dossiers.
Ménage fantôme	Le bénéficiaire déclare un ménage plus nombreux qu'en réalité pour toucher une aide plus élevée.
Manipulation des paiements	Le bénéficiaire reçoit un premier paiement puis disparaît des cycles suivants, un comportement anormal qui trahit une manipulation.
Usurpation d'identité	Une identité volée ou fabriquée est utilisée pour s'inscrire au programme.
Sous-déclaration du revenu	Le bénéficiaire déclare un revenu bien plus faible que son revenu réel pour paraître éligible à l'aide.
Fraude coordonnée	Un petit groupe de bénéficiaires partage plusieurs téléphones et comptes bancaires pour frauder ensemble.
Ménage fantôme systématique	De faux ménages sont créés de façon répétée et organisée, à grande échelle.
Exploitation du réseau	Un grand nombre de comptes reliés entre eux (téléphones et comptes partagés) exploitent le système de manière massive.

Pour rendre ce jeu de données réaliste, deux précautions ont été prises. D'abord, un léger bruit aléatoire est ajouté sur les variables sensibles (revenu, score PMT, nombre de programmes) : sans cela, les fraudeurs et les bénéficiaires honnêtes seraient trop faciles à séparer, et le modèle donnerait de bons résultats sur ces données mais échouerait en conditions réelles. Ensuite, environ 8 % des fraudeurs reçoivent des caractéristiques proches de celles des bénéficiaires honnêtes : ce sont des cas volontairement difficiles, qui imitent les vrais fraudeurs cherchant à passer inaperçus.

Ces neuf scénarios une fois définis, la question suivante devient celle des moyens : quelles variables, parmi celles disponibles dans OpenG2P, permettent réellement de les détecter ?

4.2.3 Exploration de la base et extraction SQL

La difficulté centrale de cette extraction tient à la taille et à la complexité de la base OpenG2P elle-même. En tant qu'écosystème Odoo complet, elle compte plusieurs **centaines de tables** couvrant de nombreux modules sans rapport avec la détection de fraude (comptabilité, CRM, notifications, etc.), au sein desquelles il faut identifier le sous-ensemble réellement pertinent. La Figure 4.13 quantifie cet écart entre le schéma disponible et le schéma effectivement exploité : sur les 587 tables du schéma **public**, seules 7 sont mobilisées par l'extraction, et parmi celles-ci, seule une fraction des colonnes disponibles (par exemple 93 colonnes dans **res_partner**) alimente réellement les 25 features du modèle.

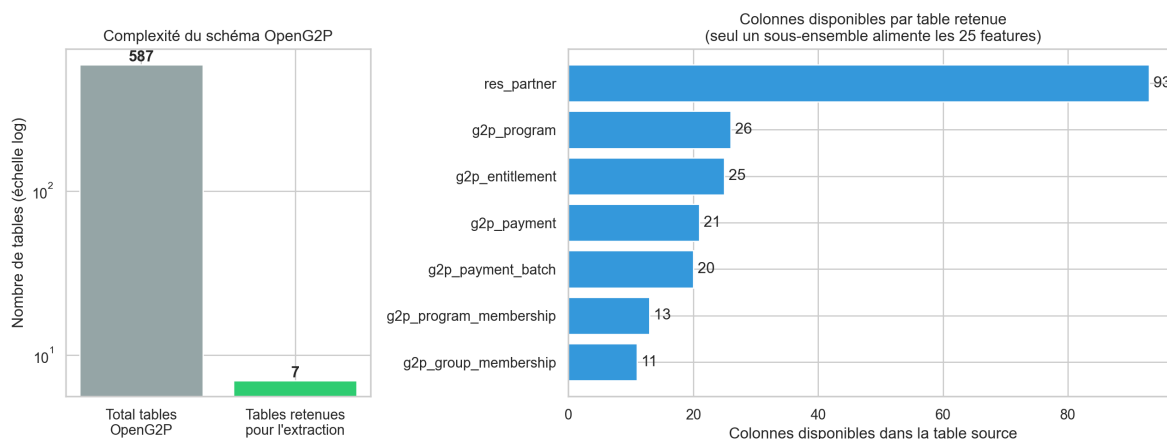


FIGURE 4.13 – Vue d'ensemble du schéma relationnel d'OpenG2P et des colonnes des sept tables sélectionnées pour l'extraction des données

Le module **Feature extractor** est responsable de l'extraction des informations depuis la base PostgreSQL d'OpenG2P. Afin de limiter les accès à la base de données, une requête SQL unique utilisant des Common Table Expressions (CTE) est exécutée. Cette requête réalise les jointures entre les tables retenues et agrège les informations relatives à chaque bénéficiaire en une seule ligne par profil, directement exploitable pour construire les variables d'entrée du moteur.

4.2.3.1 Sélection des sources de données

La solution retenue consiste à sélectionner, parmi l'ensemble des tables OpenG2P, celles qui contiennent un signal pertinent pour caractériser le profil socio-économique d'un bénéficiaire, sa participation aux programmes sociaux et son historique de paiements. L'analyse du schéma relationnel de la base PostgreSQL a permis d'identifier les modules et tables listés au Tableau 4.3, qui constituent les principales sources utilisées pour construire les variables d'entrée du moteur de détection de fraude.

TABLEAU 4.3 – Principales sources de données exploitées pour la construction du dataset

Module OpenG2P	Tables principales	Informations exploitées
Social Registry	res_partner	Informations d'identification du bénéficiaire (âge, sexe, téléphone, compte bancaire, revenu, etc.)
Program Management	g2p_program g2p_program_members	Programmes sociaux, inscriptions, statut d'adhésion et ancienneté du bénéficiaire.
Entitlements	g2p_entitlement	Prestations attribuées aux bénéficiaires, droits ouverts et historique des allocations.
Paielements	g2p_payment g2p_payment_batch	Historique des paiements, montants versés, statuts, cycles et fréquence des versements.
Registre des ménages	g2p_group g2p_group_members	Composition du ménage, liens familiaux et nombre de personnes à charge.
Évaluation socio-économique	PMT Assessment	Score PMT (Proxy Means Test) utilisé pour estimer le niveau de vulnérabilité des bénéficiaires.

Ces six sources couvrent l'identité du bénéficiaire, son inscription aux programmes, ses prestations, ses paiements, la composition de son ménage et son évaluation socio-économique — soit l'ensemble des dimensions nécessaires pour détecter aussi bien une fraude individuelle (revenus sous-déclarés) qu'une fraude organisée (réseau de comptes ou téléphones partagés). Ces variables sélectionnées, il reste à les transformer en un dataset structuré, directement exploitable par les algorithmes de Machine Learning.

4.2.3.2 Construction des features et tableau des 25 variables

À l'issue du prétraitement, chaque profil de bénéficiaire est traduit en un vecteur de 25 features numériques, résumées par catégorie au Tableau 4.4. Ces variables couvrent la démographie, la composition du ménage, la participation aux programmes, le score PMT, l'historique de paiements, les signaux réseau et des indicateurs dérivés (flags).

TABLEAU 4.4 – Les 25 features extraites pour le modèle ML

Catégorie	Features	Signification
Démographie	age, income, income_per_person	Âge, revenu total et revenu par personne
Ménage	household_size, nb_children, nb_elderly, dependency_ratio	Composition et vulnérabilité du ménage
Ménage	has_disabled, single_head	Présence d’handicap, chef de ménage seul
Programmes	nb_programs, nb_active_programs, avg_enrollment_days	Inscriptions actives et ancienneté
Score PMT	pmt_score, pmt_score_min	Score de ciblage proxy-means test
Paielements	payment_count, payment_gap_ratio, payment_success_rate	Volume et taux d’échec des paiements
Paielements	amount_variance, cycle_count	Variabilité des montants reçus
Réseau	shared_phone_count, shared_account_count, network_risk	Partage de ressources entre bénéficiaires
Groupe	group_membership_count	Appartenance à des groupes ménagers
Flags	high_amount_flag, income_program_inconsistent	Signaux dérivés de règles heuristiques

Au-delà de ce regroupement par catégorie métier, un audit technique du schéma réel du dataset confirme la cohérence des types de données : les 25 features sont toutes numériques (entiers, flottants ou booléens), sans valeur manquante, ce qui les rend directement exploitables par les modèles de Machine Learning sans étape d’encodage supplémentaire. Une fois ces variables construites, il reste à les assembler en un dataset tabulaire cohérent, avant de pouvoir les générer à grande échelle et les injecter dans OpenG2P.

4.2.4 Structure du dataset final

À l’issue des différentes étapes d’extraction, de nettoyage et de construction des features, les informations relatives à chaque bénéficiaire sont regroupées dans un dataset tabulaire destiné à l’entraînement et à l’évaluation des modèles de détection de fraude.

Le dataset obtenu est organisé sous forme tabulaire, où chaque ligne représente un bénéficiaire et chaque colonne une information descriptive ou une feature calculée. Il comporte 59 colonnes, dont 28 features utilisées par les modèles de Machine Learning, les autres étant dédiées aux identifiants et aux informations de supervision. La Figure 4.14 présente la structure complète de ce dataset.

Dataset OpenG2P — colonnes et types (36 colonnes, dont 25 features ML)

Colonne	Type	Valeurs non-nulles
age	int64	5,000
income	float64	5,000
income_per_person	float64	5,000
household_size	int64	5,000
nb_children	int64	5,000
nb_elderly	int64	5,000
dependency_ratio	float64	5,000
has_disabled	bool	5,000
single_head	bool	5,000
nb_programs	float64	5,000
nb_active_programs	int64	5,000
pmt_score	float64	5,000
pmt_score_min	float64	5,000
avg_enrollment_days	int64	5,000
payment_count	int64	5,000
payment_gap_ratio	float64	5,000
payment_success_rate	float64	5,000
amount_variance	float64	5,000
cycle_count	int64	5,000
shared_phone_count	float64	5,000
shared_account_count	float64	5,000
network_risk	float64	5,000
group_membership_count	int64	5,000
high_amount_flag	int64	5,000
income_program_inconsistency	int64	5,000
is_fraud	int64	5,000
synthetic_label	int64	5,000
partner_idx	int64	5,000
partner_id	int64	5,000
scenario	object	5,000
registration_age_days	int64	5,000
avg_entitlement_amount	float64	5,000
total_issued	float64	5,000
total_paid	float64	5,000
network_risk_score	float64	5,000
elderly_head	bool	5,000

FIGURE 4.14 – Colonnes et types du dataset OpenG2P généré (59 colonnes, dont les 28 features ML)

Au-delà de cette vue par colonnes, une *datacard* — fiche technique résumant la forme et les statistiques descriptives du dataset — a été générée automatiquement à partir du fichier réellement produit, de sorte qu'elle reste exacte à chaque régénération du dataset plutôt que d'être recopiée à la main. Le Tableau 4.5 en résume la forme globale, et le Tableau 4.6 détaille les statistiques descriptives (moyenne, écart-type, minimum, maximum) de chaque feature, regroupées par catégorie.

TABLEAU 4.5 – Fiche technique (datacard) du dataset synthétique

Caractéristique	Valeur
Nombre de lignes (bénéficiaires)	5 000
Nombre total de colonnes	59
Nombre de features ML	28
Types de colonnes	float64 : 22, int64 : 17, object : 17, bool : 3
Valeurs manquantes (sur les 28 features ML)	0
Bénéficiaires légitimes	4 341
Bénéficiaires frauduleux	659
Taux de fraude	13.18 %

TABLEAU 4.6 – Statistiques descriptives des features par catégorie

Catégorie	Feature	Moy.	Écart-t.	Min	Max
Démographie	age	49.303	15.73	23.0	76.0
	income	845.516	765.732	19.74	5917.09
	income_per_person	292.456	393.201	1.62	4862.12
Ménage	household_size	4.374	2.724	1.0	21.0
	nb_children	1.16	1.512	0.0	12.0
	nb_elderly	0.205	0.427	0.0	2.0
	dependency_ratio	0.72	1.077	0.0	11.0
	has_disabled	0.071	0.257	0.0	1.0
	single_head	0.156	0.363	0.0	1.0
Programmes	nb_programs	2.053	1.123	0.0	6.101
	nb_active_programs	1.937	0.874	1.0	4.0
	avg_enrollment_duration	1685.972	420.67	969.0	2399.0
Score PMT	pmt_score	0.579	0.242	0.0	1.357
	pmt_score_min	0.537	0.192	0.05	0.999
Paielements	payment_count	5.812	2.623	3.0	12.0
	payment_gap_ratio	0.331	0.213	0.0	1.206
	payment_success_ratio	0.676	0.139	0.0	0.917
	amount_variance	22934.443	14430.672	2034.12	61320.62
	cycle_count	3.0	0.0	3.0	3.0
Réseau	shared_phone_count	0.846	1.215	0.0	9.692
	shared_account_count	0.632	0.893	0.0	6.736
	network_risk	0.178	0.215	0.0	1.524
Groupe	group_membership	0.507	0.5	0.0	1.0
Signaux dérivés (flags)	high_amount_flag	0.05	0.218	0.0	1.0
	income_program_index	0.027	0.162	0.0	1.0
Calibration économique	income_ratio_to_income	1.46	1.323	0.034	10.22
	household_size_delta	0.374	2.724	-3.0	17.0
Identité	duplicate_nationality	0.005	0.072	0.0	1.0

Ce dataset structuré n'a de valeur que s'il peut être produit à l'échelle et alimenter réellement la plateforme : c'est l'objet de la dernière étape de cette construction.

4.2.5 Génération du dataset synthétique et injection PostgreSQL

Une fois les différentes features définies, un module dédié génère automatiquement un grand nombre de profils synthétiques reproduisant les principales caractéristiques observées dans les données OpenG2P : démographie, composition des ménages, historique des paiements et relations entre bénéficiaires. Ces profils sont ensuite injectés directement dans la base PostgreSQL d'OpenG2P afin d'obtenir un environnement de test réaliste, suffisamment riche pour entraîner et évaluer les modèles, sans utiliser de données personnelles réelles.

La Figure 4.15 présente ce processus. À partir de plusieurs scénarios de fraude et de profils légitimes définis lors de la phase de conception, le générateur crée automatiquement les bénéficiaires synthétiques puis les insère dans les différentes tables d'OpenG2P en respectant les relations et les contraintes de la base de données.

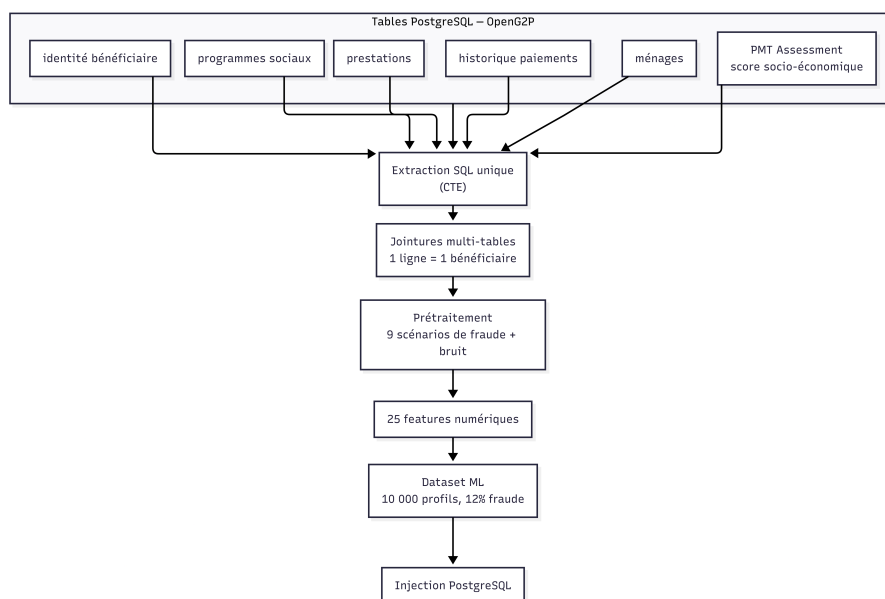


FIGURE 4.15 – Processus de génération du dataset OpenG2P synthétique

L'objectif n'est pas uniquement de produire des données réalistes, mais également d'éviter que les profils frauduleux soient trop faciles à distinguer des profils légitimes. Si les deux classes étaient parfaitement séparées, les modèles obtiendraient d'excellents résultats pendant l'entraînement sans pour autant être capables de généraliser sur de nouveaux cas. C'est pourquoi un bruit contrôlé et des cas limites ont été introduits lors de la génération des données.

La Figure 4.16 illustre cette approche à travers la distribution du score PMT. On observe un recouvrement important entre les bénéficiaires légitimes et frauduleux : les fraudes sont plus fréquentes pour les faibles valeurs de PMT, mais aucune frontière nette ne permet de les séparer. Cette distribution est volontairement recherchée afin de rapprocher le dataset des situations rencontrées en pratique.

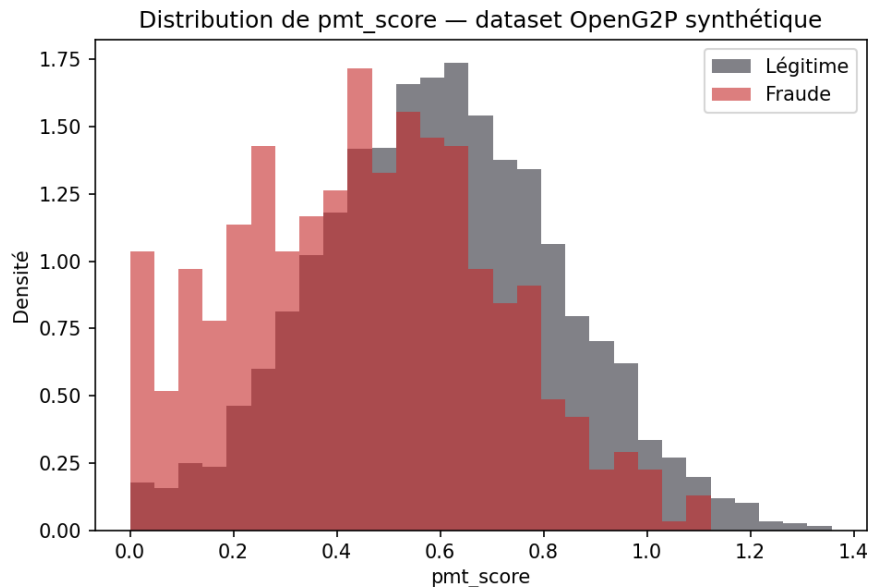


FIGURE 4.16 – Distribution du score PMT selon la classe réelle dans le dataset OpenG2P synthétique

Le dataset obtenu constitue désormais la base de travail du moteur de détection de fraude. Les 28 features construites sont utilisées comme variables d'entrée par les différents modules du système : moteur de règles, modèles de Machine Learning, analyse de réseau et moteur d'explicabilité.

4.2.6 Un second banc d'essai : le dataset PaySim

Bien que le dataset OpenG2P reproduise fidèlement le fonctionnement de la plateforme, il reste un jeu de données synthétique. Afin de vérifier que les modèles développés ne sont pas uniquement adaptés à ce contexte, un second jeu de données public, PaySim (Kaggle), a été utilisé lors de la phase d'entraînement et de validation.

Contrairement à OpenG2P, où chaque ligne correspond à un bénéficiaire, PaySim contient des transactions de paiement mobile. Les variables décrivent notamment le montant de la transaction, les soldes des comptes avant et après l'opération ainsi que le type de transaction. Avant l'entraînement, une phase de préparation a été nécessaire : certaines colonnes présentaient une fuite de données en révélant directement les transactions frauduleuses. Ces variables ont donc été supprimées afin d'obtenir un jeu de données plus représentatif et d'éviter des performances artificiellement élevées.

La Figure 4.17 présente la distribution des montants des transactions après nettoyage. Comme pour le score PMT du dataset OpenG2P, les distributions des transactions légitimes et frauduleuses se chevauchent largement. Le montant seul ne permet donc pas d'identifier une fraude, ce qui confirme la nécessité de combiner plusieurs variables pour prendre une décision fiable.

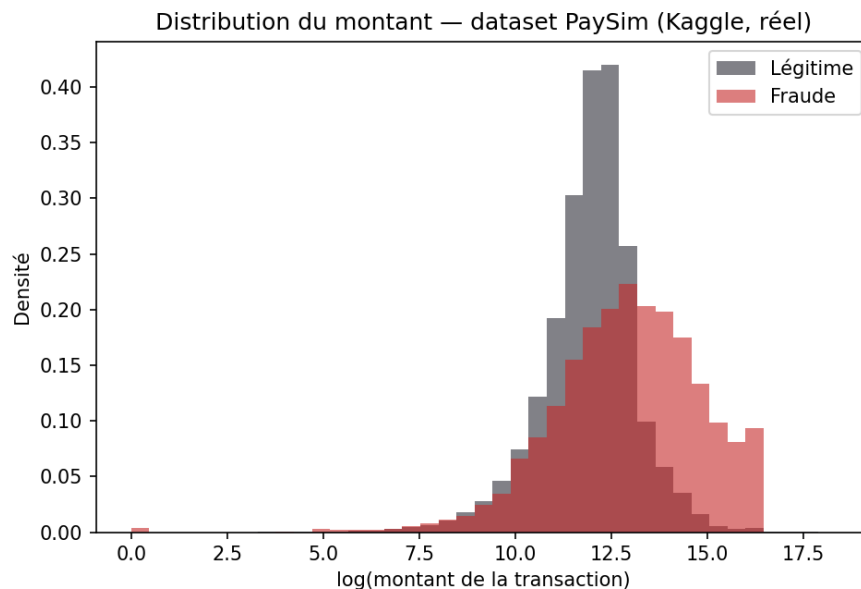


FIGURE 4.17 – Distribution du montant des transactions selon la classe réelle dans le dataset PaySim

Le modèle XGBoost entraîné sur les onze features retenues obtient d'excellentes performances (ROC-AUC de 1,000 et PR-AUC de 0,996). Ces résultats sont supérieurs à ceux obtenus sur OpenG2P, mais cette différence s'explique principalement par la nature des données. Les fraudes bancaires présentes dans PaySim laissent des indices plus marqués, tandis que les fraudes liées aux bénéficiaires d'aides sociales sont généralement plus discrètes et réparties sur plusieurs dimensions, comme le revenu, les relations entre bénéficiaires ou l'historique des paiements.

L'utilisation de PaySim a apporté plusieurs bénéfices. Elle a d'abord permis d'identifier et de corriger des fuites de données, puis de valider l'utilisation de métriques adaptées aux jeux de données déséquilibrés, comme le PR-AUC. Enfin, elle a confirmé que le pipeline d'entraînement développé n'est pas spécifique au seul dataset OpenG2P et reste performant sur un contexte totalement différent.

PaySim n'a toutefois pas vocation à remplacer le dataset OpenG2P. Il est uniquement utilisé comme jeu de validation complémentaire. Son rôle est de démontrer que les méthodes et les modèles retenus restent efficaces sur des données réelles, tout en conservant OpenG2P comme base principale du moteur de détection de fraude.

4.2.7 Moteur de détection de fraude

Avant même de faire appel à l'apprentissage automatique, le système applique une première ligne de détection : le moteur de règles. Il permet de capter des patterns de fraude connus sans avoir besoin d'un modèle ML — une approche déterministe et immédiatement interprétable, qui complète les modèles statistiques développés dans la suite du chapitre. Chaque règle est définie sous la forme d'une condition, d'un poids et d'un niveau de sévérité.

Les dix règles couvrent les scénarios suivants : partage de compte bancaire (poids 0,35), partage de téléphone (poids 0,30), trop de programmes (poids 0,30), score PMT suspect (poids

0,20), écart de paiements (poids 0,30), ménage fantôme (poids 0,25), combinaison téléphone et compte (poids 0,40), risque réseau élevé (poids 0,30), sous-déclaration de revenus (poids 0,20), et manipulation combinée au réseau (poids 0,35).

Les conditions de ces règles sont évaluées de façon sécurisée, sans jamais exécuter de code arbitraire — un choix d'implémentation déterminant puisque les règles peuvent, à terme, être modifiées par un administrateur métier sans revalidation du code source.

Le score de règles correspond à la somme des poids des règles déclenchées, plafonnée à 1. Une liste de signaux (par exemple compte partagé, ou combinaison réseau frauduleux) accompagne ce score pour expliquer les déclenchements.

Si le moteur de règles capture efficacement les patterns de fraude déjà répertoriés, il reste par nature limité aux scénarios anticipés lors de sa conception. La détection de patterns plus subtils ou non explicitement codifiés nécessite un second moteur, statistique cette fois : le pipeline de Machine Learning.

4.2.8 Conception du moteur de détection de fraude par Machine Learning

Le pipeline de Machine Learning ne repose pas sur un modèle unique, mais sur un **ensemble à deux étages** (*stacking*) : trois modèles supervisés de nature différente sont d'abord entraînés indépendamment sur les mêmes 28 features, puis un méta-modèle apprend à combiner leurs trois prédictions en un score unique, plus fiable que n'importe lequel des trois pris isolément. Un quatrième modèle, non supervisé, complète cet ensemble pour couvrir les fraudes qu'aucun exemple d'entraînement ne représente encore.

Le dataset utilisé pour l'entraînement est déséquilibré : comme le montre la Figure 4.18, seuls 13,2 % des bénéficiaires du jeu d'entraînement sont étiquetés comme frauduleux. Sans traitement, ce déséquilibre biaiserait tout modèle supervisé vers la classe majoritaire, au risque de manquer une part importante des vraies fraudes. Plutôt que de générer des exemples synthétiques de la classe minoritaire, la correction retenue agit directement sur la fonction de coût de chaque modèle : Random Forest et Régression Logistique pénalisent davantage une erreur sur la classe fraude via `class_weight="balanced"`, et XGBoost applique un facteur `scale_pos_weight` qui amplifie le gradient des exemples frauduleux mal classés. Cette approche a l'avantage de ne jamais introduire de données artificielles dans l'entraînement : chaque exemple vu par les modèles est un profil réellement généré, seule l'importance qui lui est accordée pendant l'apprentissage varie.

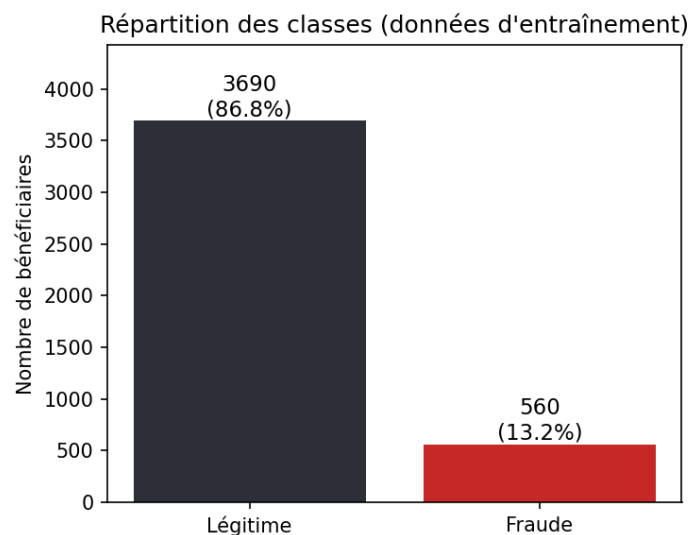


FIGURE 4.18 – Répartition des classes dans les données d’entraînement

Les trois modèles de base sont évalués individuellement sur un jeu de test (*holdout*) de 750 bénéficiaires jamais vus à l’entraînement, avant que leurs prédictions ne soient combinées. Le Tableau 4.7 compare leurs performances respectives à celles de l’ensemble final, et la Figure 4.19 illustre leurs courbes ROC.

TABLEAU 4.7 – Comparaison des modèles de base et de l’ensemble (jeu de test, seuil 0,5)

Modèle	AUC-ROC	F1 (fraude)	Précision	Rappel
Random Forest	0.905	0.704	0.933	0.566
XGBoost	0.913	0.807	0.890	0.737
Régression Logistique	0.874	0.561	0.448	0.747
Ensemble (stacking, retenu)	0.912	0.721	0.658	0.798

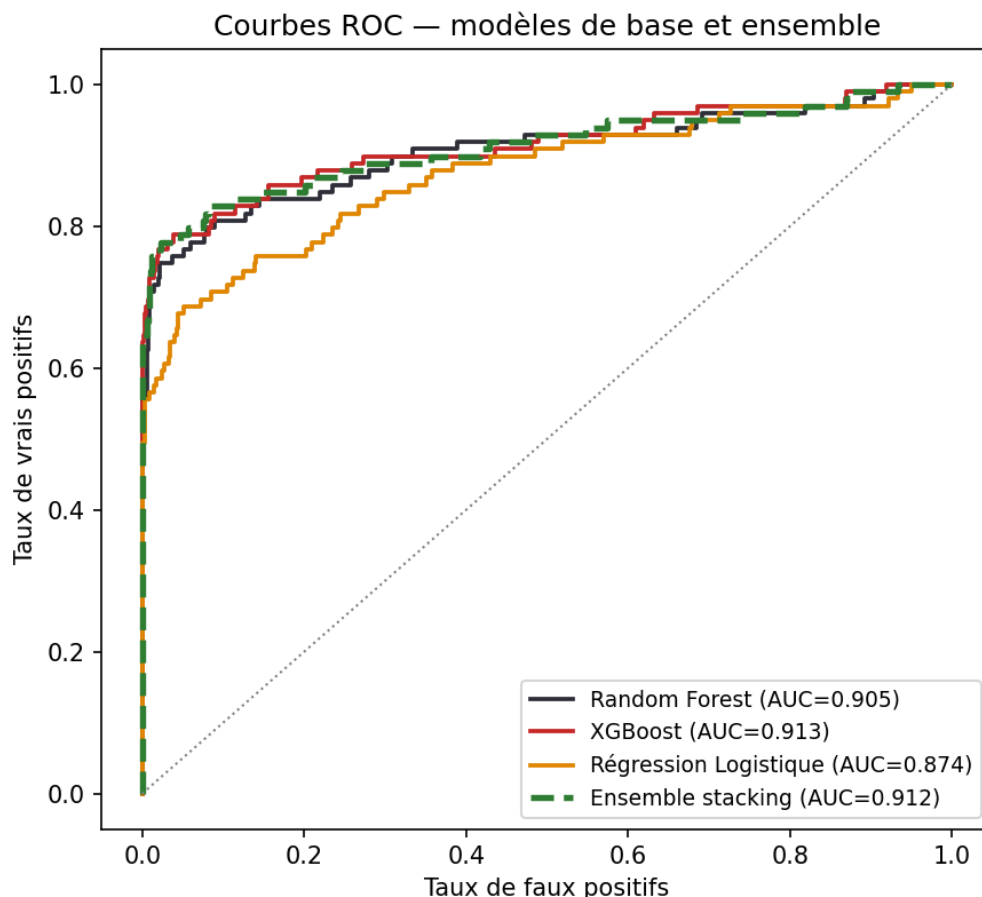


FIGURE 4.19 – Courbes ROC comparées des trois modèles de base et de l'ensemble

Aucun des trois modèles de base ne domine les autres sur tous les critères : XGBoost obtient le meilleur F1 et le meilleur rappel pris isolément, Random Forest la meilleure précision, et la Régression Logistique — bien que la plus faible en AUC — un rappel proche de celui de XGBoost. C'est précisément cette complémentarité que le méta-modèle exploite : en apprenant le poids à accorder à chacun plutôt qu'en imposant une combinaison fixe, l'ensemble atteint un rappel de 0,798, supérieur à celui de chacun des trois modèles pris seul, tout en conservant un AUC proche du meilleur modèle individuel. Un ajustement du seuil de décision (0,55 au lieu de 0,5, retrouvé par recherche sur les prédictions hors-échantillon de la validation croisée) porte le F1 final à 0,748.

En complément de ces trois modèles supervisés, un modèle **Isolation Forest** détecte les bénéficiaires dont le comportement est statistiquement atypique, sans avoir besoin d'exemples de fraude étiquetés : il isole chaque profil en le séparant aléatoirement des autres, les profils anormaux nécessitant nettement moins d'étapes de séparation que les profils courants. La Figure 4.20 compare la distribution de ce score d'anomalie entre bénéficiaires légitimes et fraudeurs : les fraudeurs se concentrent nettement vers les scores les plus élevés, même si un recouvrement subsiste dans la zone intermédiaire — ce chevauchement partiel est attendu, un modèle non supervisé n'ayant par construction aucune connaissance du label réel.

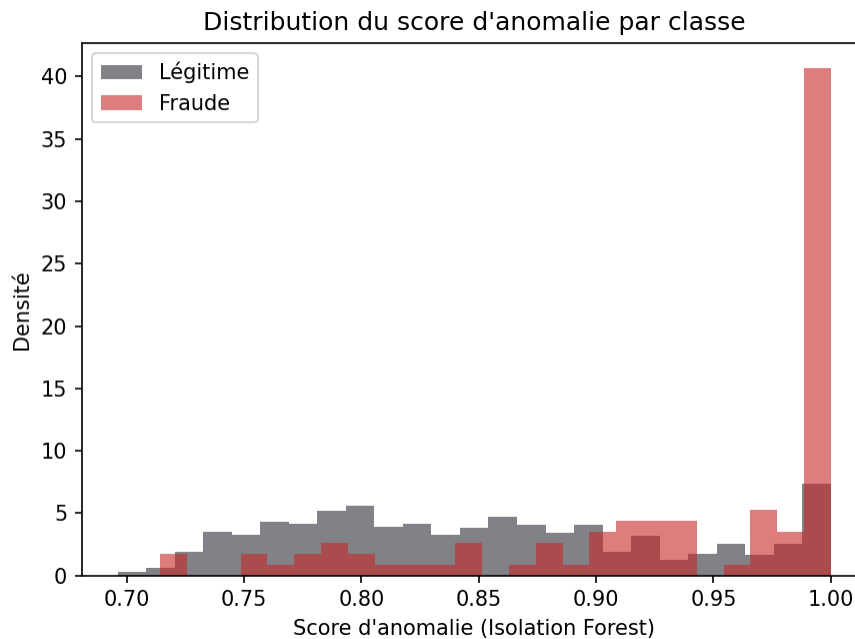


FIGURE 4.20 – Distribution du score d’anomalie (Isolation Forest) par classe réelle

Ce filet de sécurité non supervisé est calculé pour chaque bénéficiaire et exposé dans l’API du moteur, où il enrichit le contexte d’explicabilité ; son intégration au score pondéré final est discutée dans la sous-section 4.2.10.

Le moteur ML détecte efficacement les anomalies au niveau individuel, mais certaines fraudes ne se révèlent qu’au niveau collectif — lorsque plusieurs bénéficiaires partagent des ressources entre eux. C’est ce que l’analyse de réseau vient compléter.

4.2.9 Analyse de réseau

Les fraudes organisées, comme le partage de compte bancaire entre plusieurs bénéficiaires fictifs, ne sont pas toujours visibles au niveau individuel. L’analyse de réseau les révèle en construisant un graphe où les nœuds sont les bénéficiaires et les arêtes représentent des ressources partagées. La Figure 4.21 résume l’ensemble de ce traitement, depuis les données brutes d’OpenG2P jusqu’à la contribution du score réseau au score hybride final.

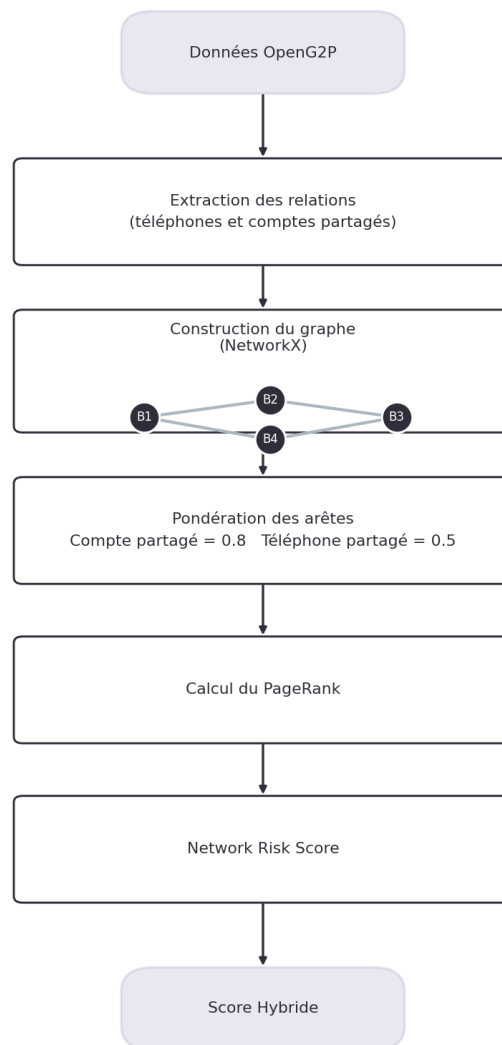


FIGURE 4.21 – Pipeline de l’analyse de réseau : extraction des relations, construction du graphe, pondération des arêtes et calcul du PageRank

Les comptes bancaires partagés se voient attribuer un poids double par rapport aux téléphones partagés (0,8 contre 0,5), car ils représentent un signal de fraude plus fort : partager un numéro de téléphone peut être légitime en contexte rural, mais partager un compte bancaire est très rarement justifiable. L’algorithme PageRank est ensuite appliqué sur ce graphe pour identifier les nœuds les plus centraux, donc les plus susceptibles d’appartenir à un réseau frauduleux organisé.

À ce stade, le système dispose de trois signaux indépendants — règles, ML, réseau — chacun capturant une facette différente du risque de fraude. Encore faut-il les combiner en une décision unique et exploitable : c’est le rôle de la fusion des scores.

4.2.10 Fusion des scores

Le moteur de détection combine trois sources d’information complémentaires : le moteur de règles, le modèle de Machine Learning et l’analyse de réseau. Cette approche s’inspire du principe du *stacking*, qui consiste à fusionner plusieurs prédictions afin d’améliorer la qualité

de la décision finale [4]. De manière générale, cette combinaison est exprimée par :

$$\hat{y} = \sum_{m=1}^M w_m f_m(x) \quad (4.1)$$

où $f_m(x)$ représente la prédiction du modèle m et w_m son poids dans la décision finale. Dans notre système, cette formule est adaptée pour combiner les trois scores produits par les différents modules :

$$\text{score_final} = 0,55 \times s_{\text{ML}} + 0,20 \times s_{\text{règles}} + 0,25 \times s_{\text{graphe}} \quad (4.2)$$

Le modèle de Machine Learning reçoit le poids le plus élevé (55 %), car il apprend automatiquement des relations complexes à partir des données. Le moteur de règles (20 %) apporte une détection simple et facilement interprétable des scénarios connus, tandis que l'analyse de réseau (25 %) permet d'identifier les fraudes organisées entre plusieurs bénéficiaires.

Ces poids ont été définis lors de la conception du système afin d'exploiter les points forts de chaque approche. Contrairement à l'étude de Noçairi et al., où les poids sont appris automatiquement à partir des données, ils sont ici fixés manuellement. L'apprentissage automatique de ces poids constitue une perspective d'amélioration.

Le score obtenu est ensuite converti en quatre niveaux de risque (**LOW, MEDIUM, HIGH et CRITICAL**), ce qui facilite la priorisation des dossiers à examiner.

Le score d'anomalie fourni par l'Isolation Forest est également calculé, mais il n'est pas encore intégré à cette formule. Il est actuellement utilisé comme un indicateur complémentaire pour l'explicabilité et le suivi des résultats.

Une fois le score de risque calculé, il reste à expliquer les raisons qui ont conduit à cette décision. Les deux modules suivants, SHAP et le modèle de langage Mistral, répondent à cet objectif en fournissant une explication claire des résultats.

4.2.11 Explicabilité SHAP

Un score seul n'est pas exploitable par un agent social : encore faut-il pouvoir justifier pourquoi un bénéficiaire donné a été signalé. Le moteur intelligent génère deux niveaux d'explication, dont le premier repose sur SHAP.

SHAP (*SHapley Additive exPlanations*) décompose la prédiction du modèle XGBoost en contributions individuelles de chaque feature, ce qui permet d'identifier les variables ayant le plus influencé le score attribué à un bénéficiaire donné. La Figure 4.22 illustre cette répartition de l'importance des variables sur l'ensemble des prédictions du jeu de test : chaque point représente un bénéficiaire, sa position horizontale son impact sur le score (positif ou négatif), et sa couleur la valeur de la feature pour ce bénéficiaire (rouge : valeur élevée, bleu : valeur faible).

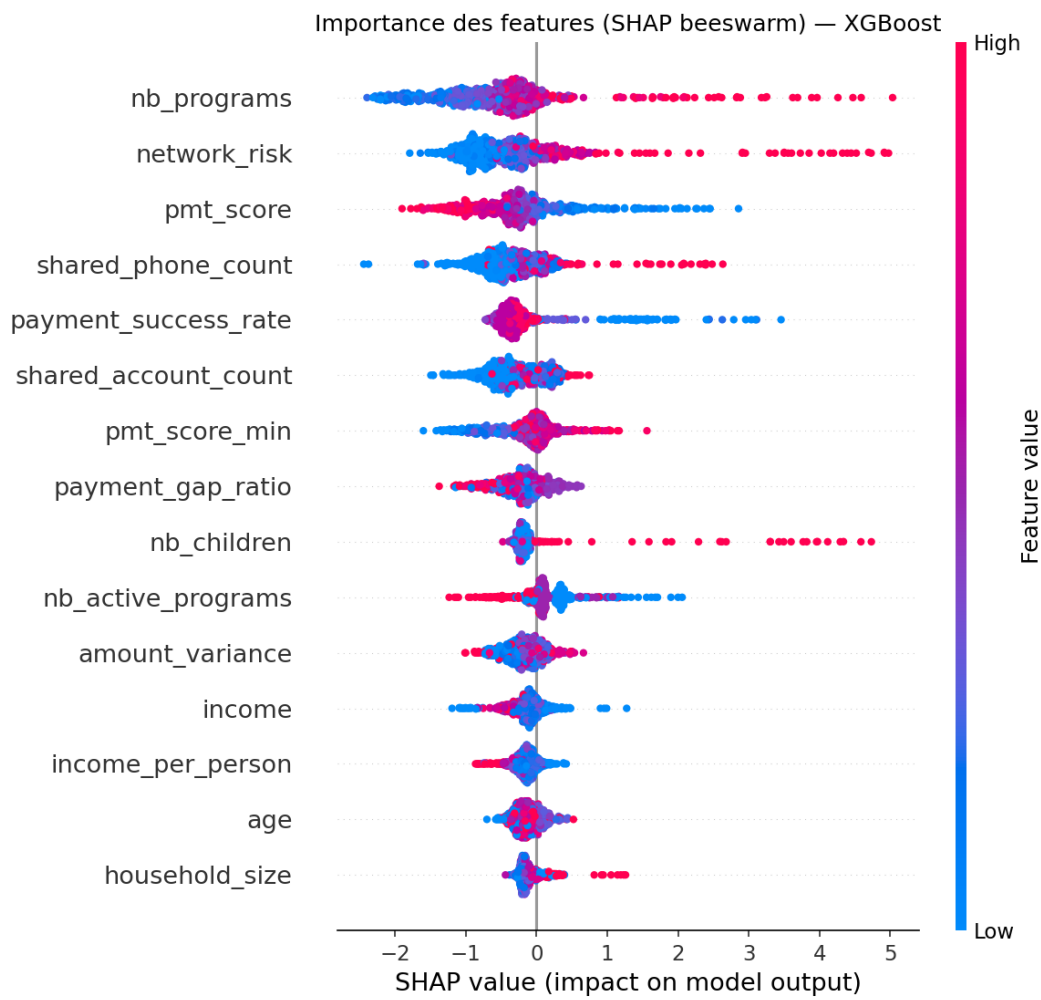


FIGURE 4.22 – Importance globale des features (SHAP beeswarm) sur le jeu de test

Le modèle XGBoost accorde principalement de l'importance au nombre de programmes, au risque du réseau et aux indicateurs de paiement. Les variables liées au partage de téléphone ou de compte contribuent également à la détection de comportements potentiellement frauduleux. À l'inverse, certaines caractéristiques socio-économiques telles que le revenu et l'âge ont une influence plus limitée.

On observe par exemple qu'un **nb_programs** élevé (points rouges à droite) pousse systématiquement le score vers la fraude, alors qu'un **pmt_score** élevé (points bleus à droite, rouges à gauche) le pousse plutôt vers un profil légitime — cohérent avec l'intuition métier : un score PMT élevé signale un ménage moins vulnérable, donc moins susceptible de frauder pour capter une aide.

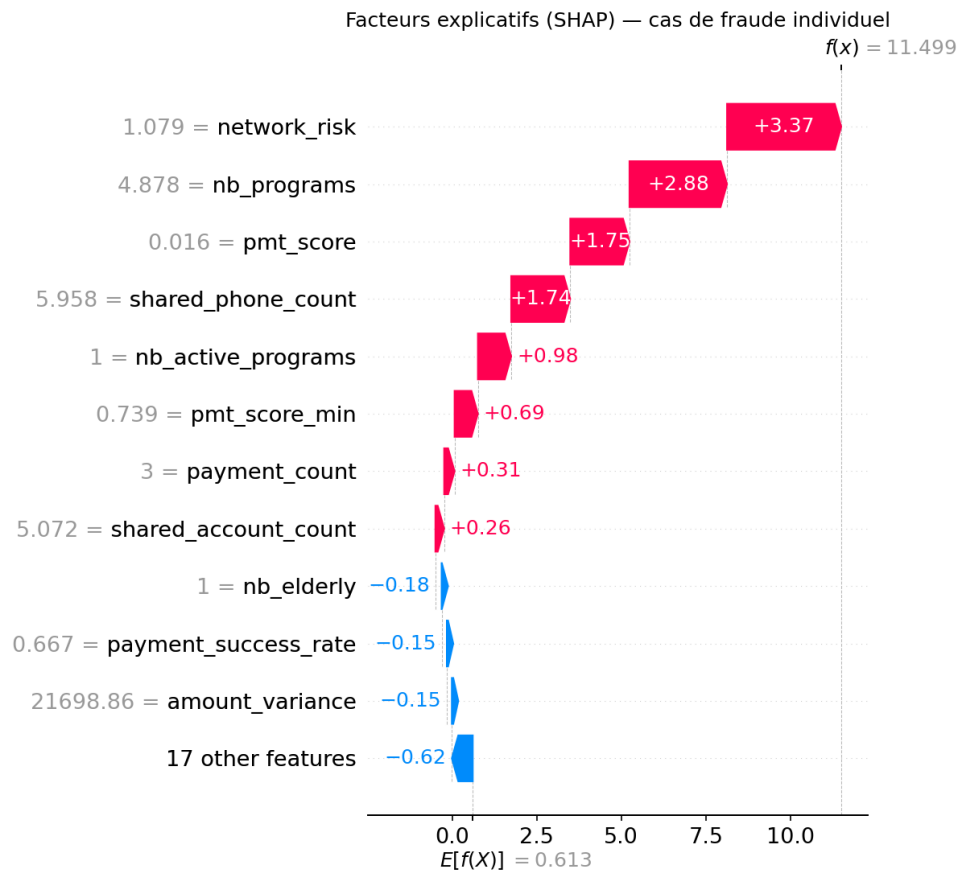


FIGURE 4.23 – Facteurs explicatifs SHAP pour un cas de fraude individuel

Cette décomposition technique, bien qu'indispensable pour un data scientist ou un auditeur, demeure difficile à interpréter directement par un agent social non spécialiste. La dernière couche du pipeline traduit donc ces facteurs SHAP en une explication rédigée en langage naturel.

4.2.12 Explication LLM

Le modèle de langage Mistral reçoit les facteurs SHAP, le score final et les signaux déclenchés, puis génère une explication lisible. Le module RAG (ChromaDB) enrichit ce contexte avec des cas de fraude similaires issus de la base de connaissance, ce qui permet à l'explication générée de s'appuyer sur des précédents comparables plutôt que sur les seules variables numériques.

Exemple de réponse générée : La Figure 4.24 vous montre un exemple d'explication générée pour un bénéficiaire réel du jeu de test. L'agent social peut ainsi comprendre immédiatement les raisons du signalement et décider d'une action appropriée.

« Getachew Daniel partage son compte bancaire avec Thomas Agyeman, un autre bénéficiaire du même programme. Il est également inscrit dans plusieurs autres programmes en même temps, ce qui constitue un risque très élevé. »

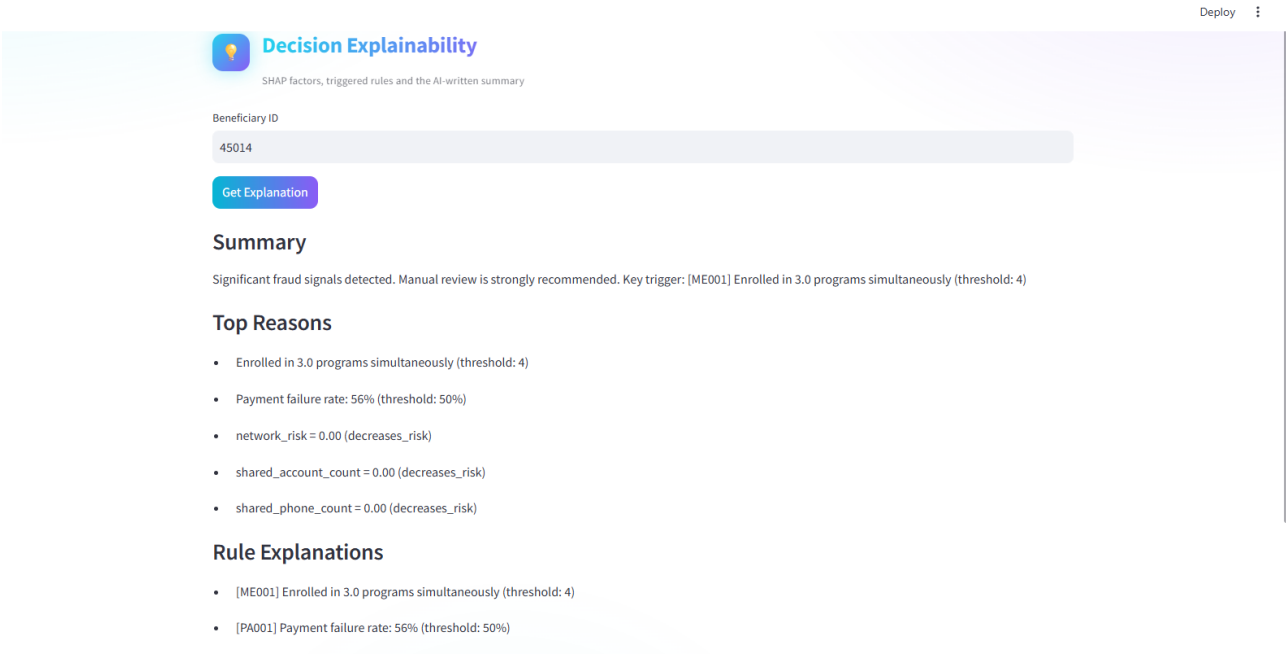


FIGURE 4.24 – Explication LLM détaillée générée pour un bénéficiaire intégrée dans le tableau de bord

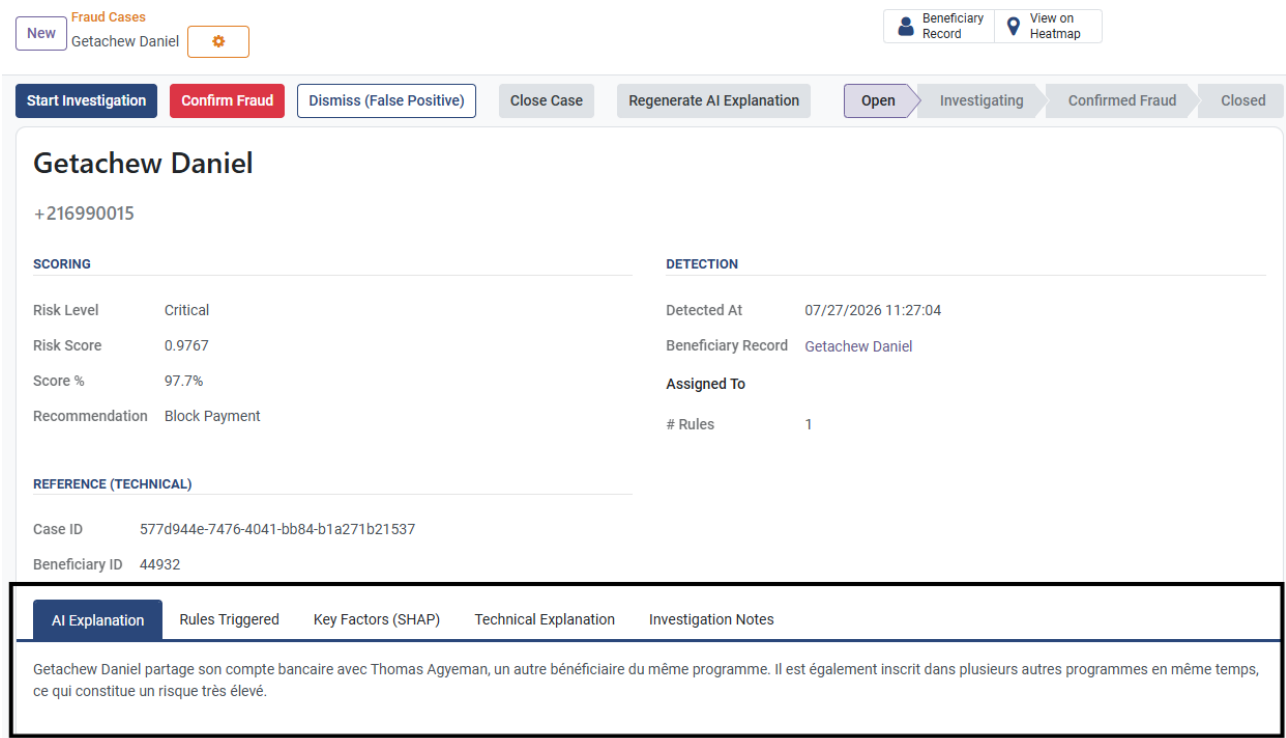


FIGURE 4.25 – Explication LLM simple générée pour un bénéficiaire intégrée dans le tableau de bord

Avec cette explication en langage naturel, le pipeline de scoring est complet : de l'extraction des données jusqu'à la restitution d'une décision justifiée. Reste à situer ce pipeline dans son environnement d'exploitation réel — comment il est exposé, synchronisé avec Odoo, restitué dans un tableau de bord, et déployé en production. C'est l'objet de la section suivante.

4.3 Intégration dans OpenG2P

Le moteur intelligent, aussi complet soit-il, n'a de valeur opérationnelle que s'il s'intègre naturellement dans le quotidien des agents sociaux qui utilisent OpenG2P. Cette section décrit les quatre mécanismes qui assurent cette intégration : l'exposition du pipeline sous forme de service, la synchronisation automatique avec Odoo, la restitution visuelle dans le tableau de bord, et le déploiement d'ensemble via Docker.

4.3.1 Synchronisation Odoo

Un module Odoo personnalisé synchronise périodiquement les cas de fraude détectés par le moteur intelligent et les affiche dans un menu dédié, sans quitter l'interface OpenG2P.

Un mécanisme de notification automatique permet également de déclencher un scoring en tâche de fond dès qu'un bénéficiaire est créé ou mis à jour dans OpenG2P, sans action manuelle de l'agent. Cette synchronisation garantit que la donnée reste fraîche côté Odoo sans imposer à l'agent social un aller-retour vers un outil distinct.

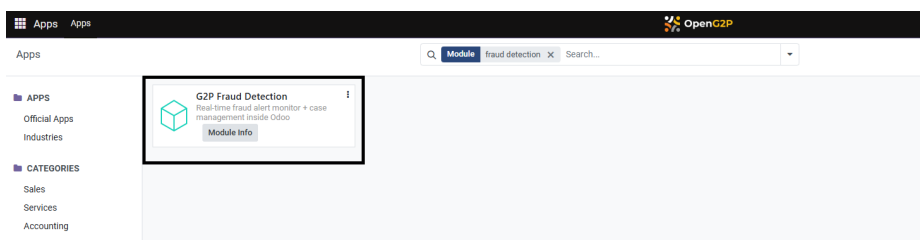


FIGURE 4.26 – Intégration du module de détection de fraude dans Odoo

La Figure 4.26 illustre l'intégration du module de détection de fraude dans Odoo.

Générateur de notification intégré a Odoo

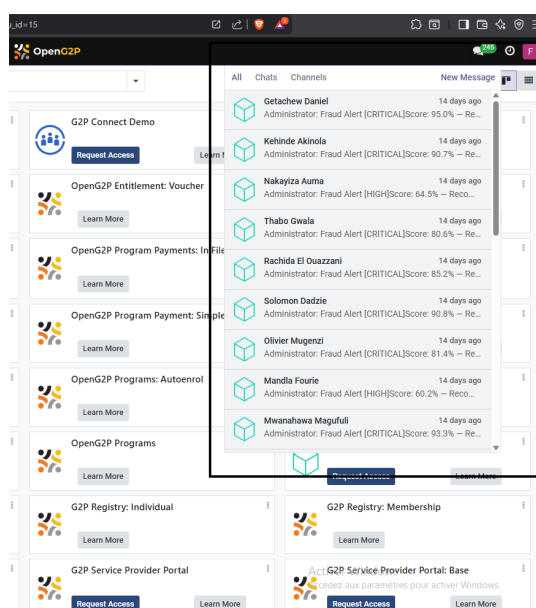


FIGURE 4.27 – Intégration du module de notification d'alertes synchronisées dans Odoo

La Figure 4.27 montre le module de notification d'alertes synchronisées dans Odoo, qui permet aux agents sociaux de recevoir des alertes en temps réel sur les bénéficiaires à risque.

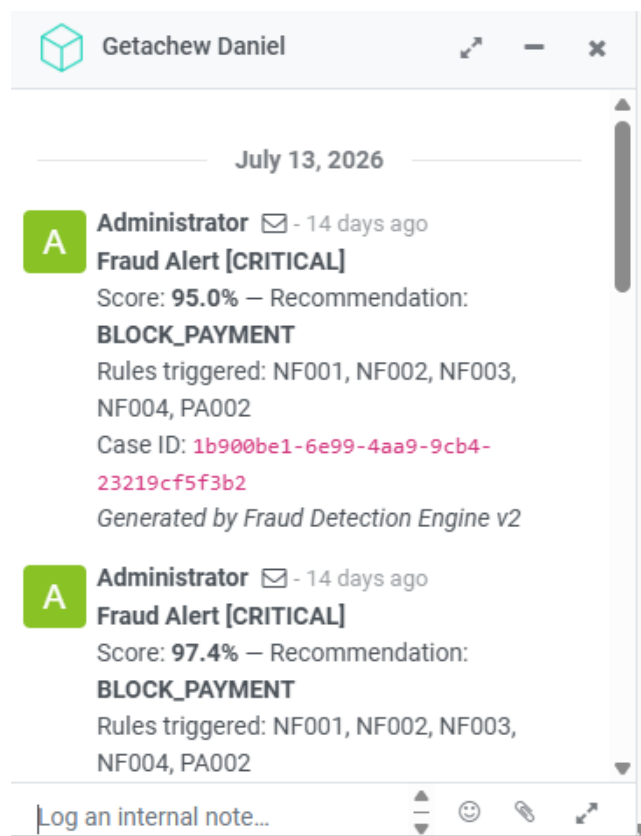


FIGURE 4.28 – Intégration du module de notification : détails d'une alerte Odoo

La Figure 4.28 montre les détails d'une alerte synchronisée dans Odoo comme le bénéficiaire concerné, le score de risque et les signaux déclenchés. L'agent social peut ainsi prendre connaissance de l'alerte et décider des actions à entreprendre directement depuis l'interface Odoo.

Vue kanban des cas de fraude

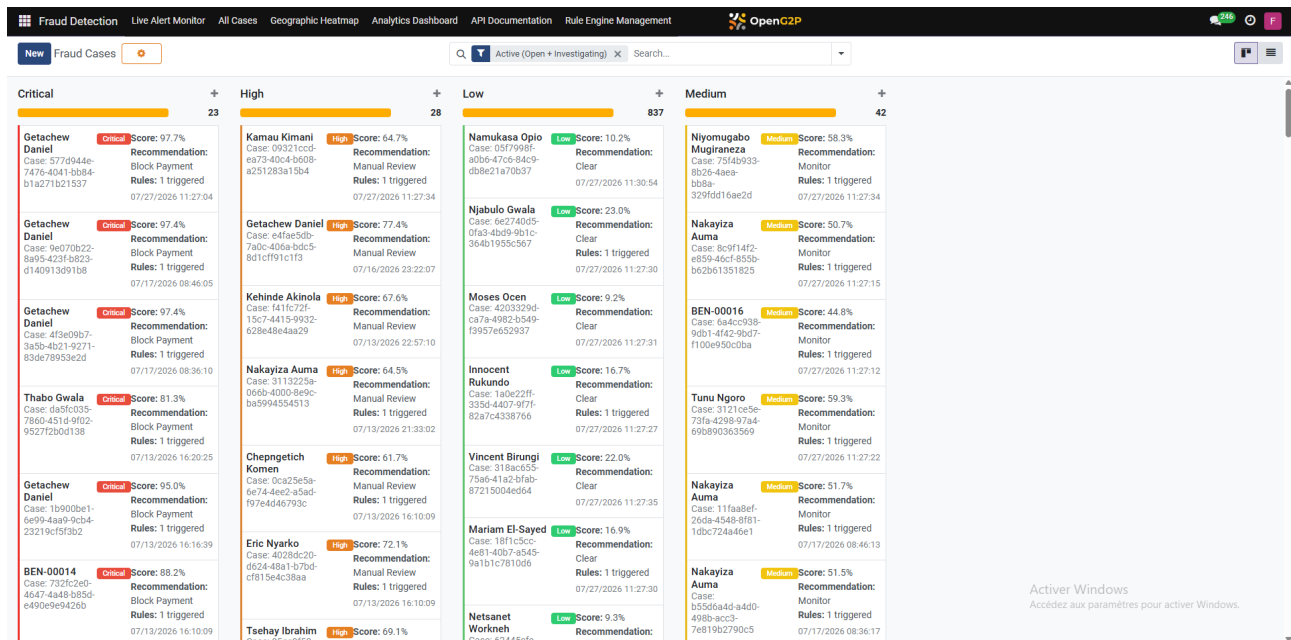


FIGURE 4.29 – Tableau de bord : vue des alertes actives avec niveaux de risque intégré dans Odoo

La Figure 4.29 montre le tableau de bord intégré dans Odoo, qui permet aux agents sociaux de visualiser les alertes actives avec les niveaux de risque associés. Les agents peuvent filtrer les alertes par programme, niveau de risque et date, et accéder aux détails de chaque cas pour prendre des décisions éclairées.

Suivi des alertes en temps réel

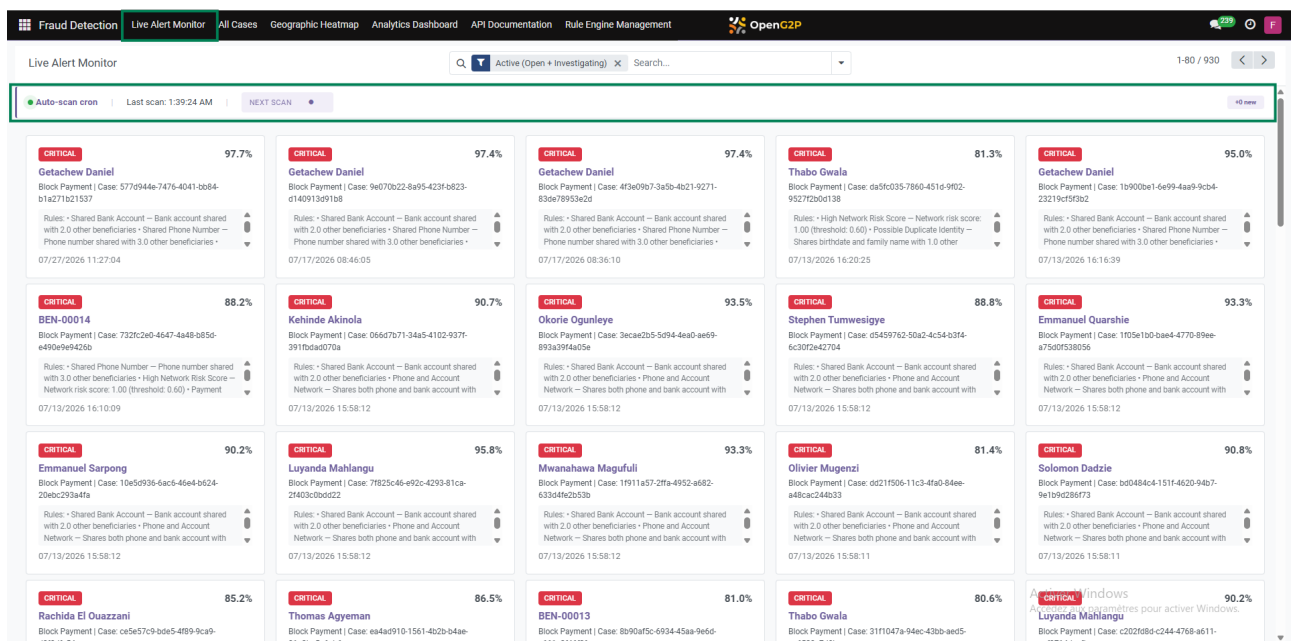


FIGURE 4.30 – Tableau de bord Odoo : vue du scanner en temps-réel de la base de données

La Figure 4.30 montre le tableau de bord du scanner en temps réel de la base de données, qui permet aux analystes fraude de suivre l'évolution des alertes et des cas de fraude détectés

par le moteur intelligent. Les agents peuvent visualiser les tendances, les statistiques et les indicateurs clés pour prendre des décisions stratégiques.

Traitement d'un cas dans Odoo

Une fois synchronisé dans OpenG2P, chaque cas de fraude s'ouvre dans une fiche dédiée (Figure 4.31) rassemblant toutes les informations nécessaires à la décision : identité et contact du bénéficiaire, niveau de risque, score et recommandation du moteur, ainsi que le nombre de règles déclenchées.

L'analyste dispose de quatre actions directement accessibles depuis cette fiche : **démarrer une investigation**, **confirmer la fraude**, **rejeter comme faux positif**, ou **clôturer le dossier**. Ce choix suit un cycle de statuts (Open → Investigating → Closed) qui trace l'historique de traitement de chaque cas. Un bouton permet également de régénérer l'explication IA si l'analyste souhaite une reformulation.

Pour appuyer sa décision, l'analyste consulte cinq onglets complémentaires : l'**explication en langage naturel** générée par le LLM, le détail des **règles déclenchées**, les **facteurs SHAP** ayant le plus influencé le score, une **explication technique** destinée à un profil plus averti, et un espace de **notes d'investigation** pour documenter ses observations.

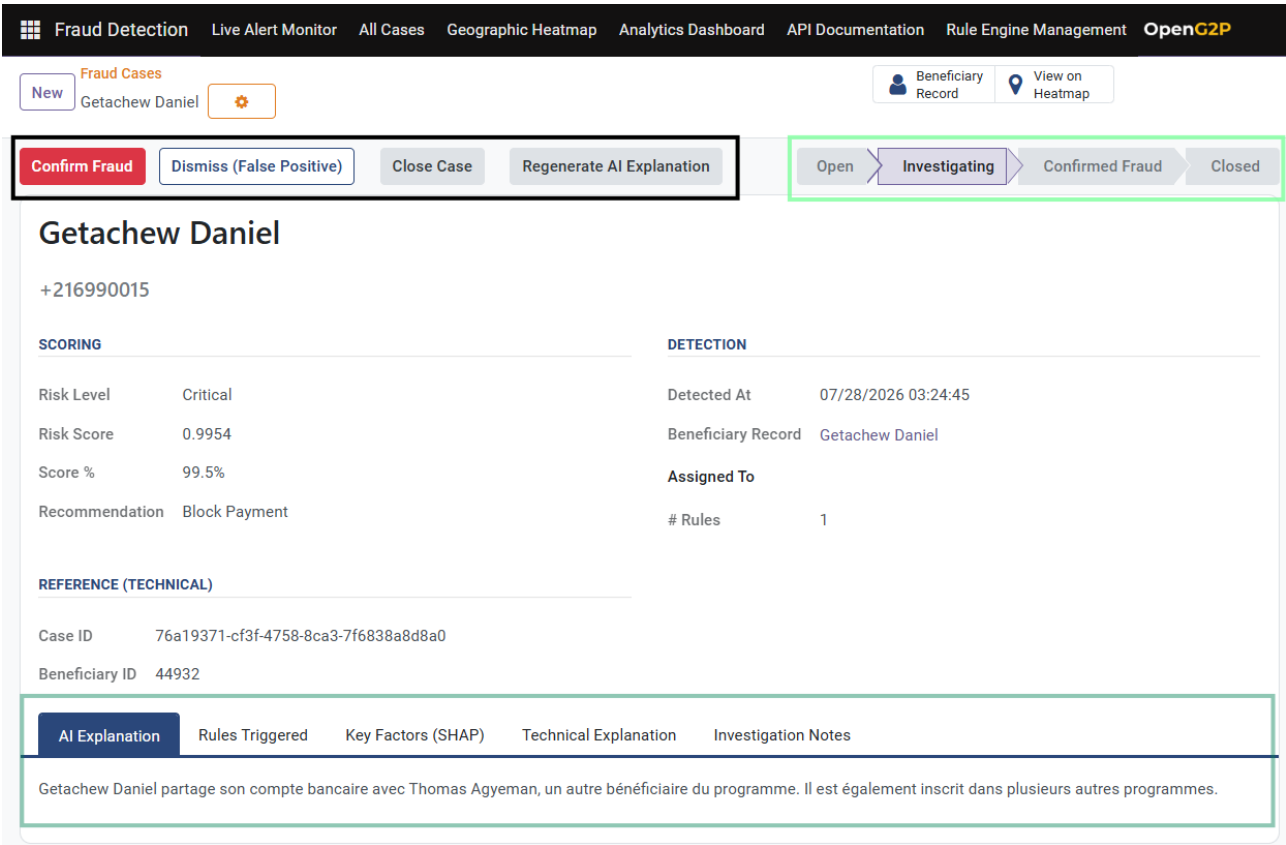


FIGURE 4.31 – Tableau de bord Odoo : outil de prise de décision

Chaque alerte critique est également enregistrée directement sur la fiche partenaire du bénéficiaire concerné (Figure 4.32), sous forme de note horodatée dans le journal Odoo : score, recommandation, règles déclenchées et identifiant du cas y figurent, avec la mention de l'engine ayant généré l'alerte. Cet historique cumulatif offre à l'analyste une vue immédiate de la récur-

rence des alertes pour un même bénéficiaire, sans quitter la fiche standard déjà utilisée pour la gestion des contacts dans OpenG2P.

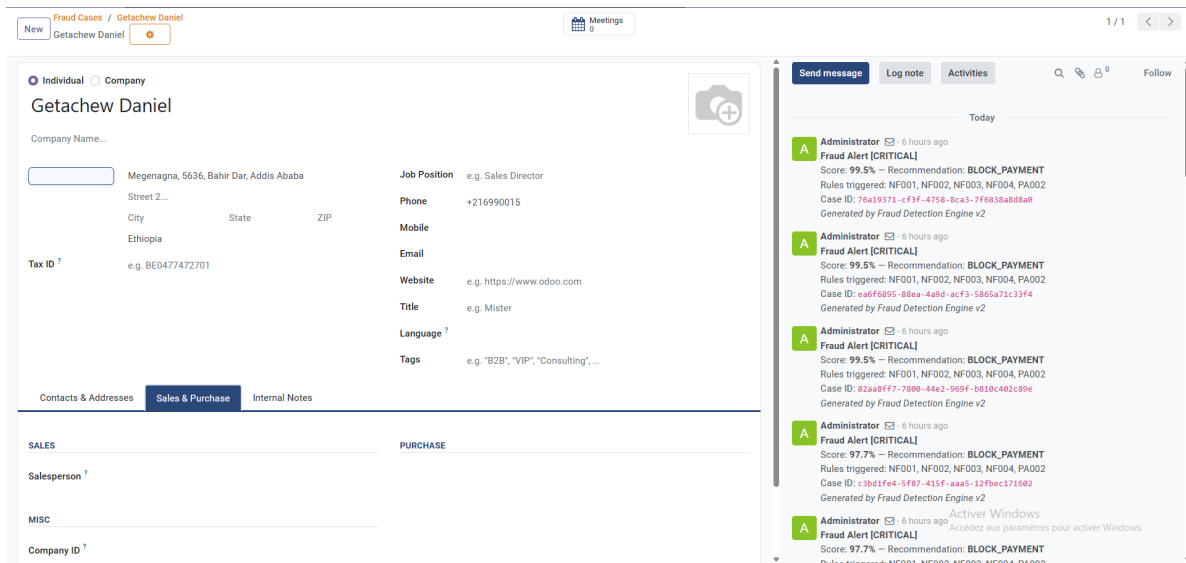


FIGURE 4.32 – Historique des alertes de fraude sur la fiche partenaire du bénéficiaire

4.3.2 Exposition via une API de services

L'ensemble du pipeline décrit précédemment est exposé sous forme de services accessibles à distance, ce qui permet à n'importe quel client — Odoo, tableau de bord, ou tout autre outil — de déclencher un scoring ou de consulter un cas sans connaître les détails d'implémentation du pipeline interne.

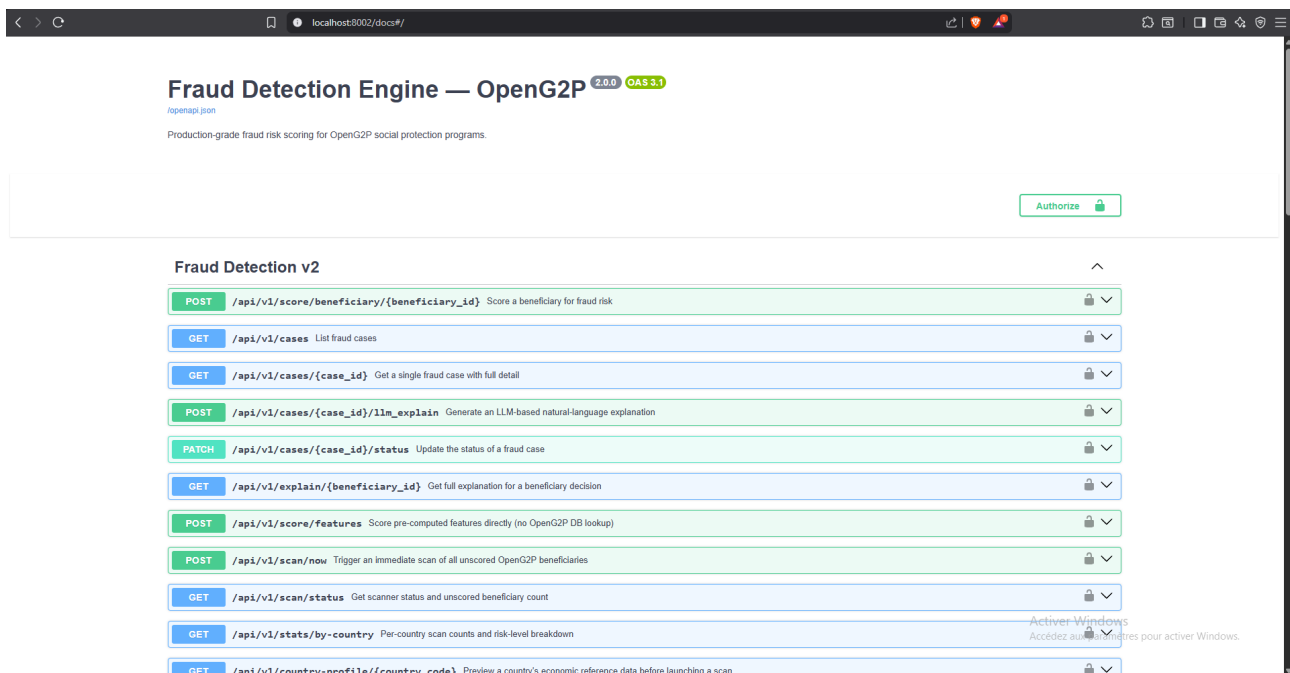


FIGURE 4.33 – Documentation Swagger de l'API exposant le pipeline de scoring

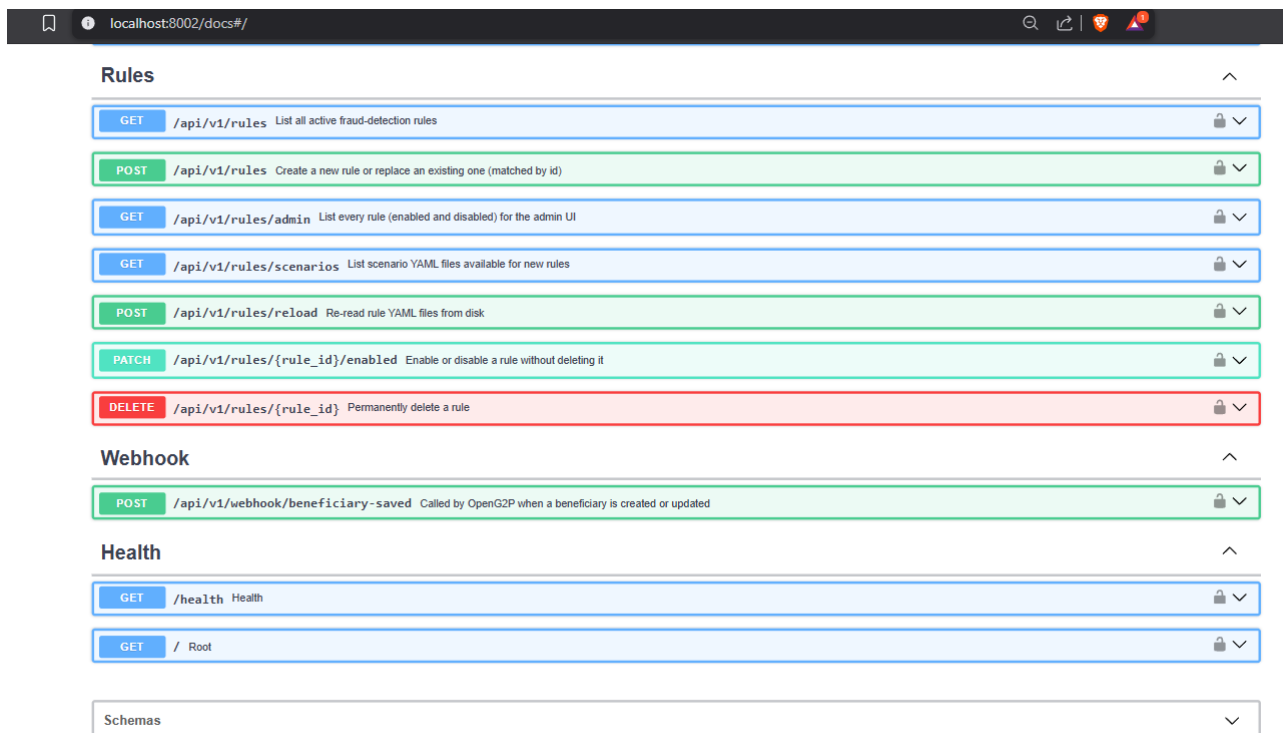


FIGURE 4.34 – Documentation Swagger de l'API exposant le pipeline de scoring

Les Figures 4.33 et 4.34 présente la documentation interactive de l'API, générée automatiquement à partir du code selon le standard **OpenAPI 3.1**. Cette documentation reste toujours synchronisée avec l'implémentation réelle et permet de tester chaque endpoint directement depuis le navigateur. Les endpoints couvrent quatre besoins : **scorer** un bénéficiaire (POST /score/beneficiary/{id}), **consulter** les cas de fraude (GET /cases), **générer une explication** (POST /cases/{id}/llm_explain, GET /explain/{beneficiary_id}), et **piloter** le scanner automatique (POST /scan/now).

Authentification et contrôle d'accès

L'accès à l'API est protégé par authentification (bouton *Authorize*, Figure 4.33), garantissant qu'aucun client non autorisé ne peut déclencher un scoring ou consulter un dossier de bénéficiaire. Ce contrôle d'accès s'appuie sur le système de groupes natif d'Odoo, avec trois rôles distincts : **Agent Social**, qui peut consulter le statut d'un bénéficiaire sans accéder aux détails de scoring ; **Analyste Fraude**, qui dispose des droits complets sur les cas (investigation, confirmation, rejet, consultation des explications) ; et **Admin**, responsable de la configuration du moteur (gestion des règles, paramètres du scanner).

TABLEAU 4.8 – Rôles et permissions d'accès au système

Rôle	Fonctionnalités accessibles
Agent Social	Consultation du statut d'un bénéficiaire, sans accès aux détails de scoring ni aux cas de fraude.
Analyste Fraude	Accès complet aux cas : investigation, confirmation, rejet, consultation des explications (SHAP, LLM), historique des alertes.
Admin	Configuration du moteur : gestion des règles métier, paramètres du scanner, supervision globale du système.

4.4 Tableau de bord de suivi - Streamlit

Le tableau de bord offre une vue consolidée des alertes en temps réel, complémentaire à l'intégration Odoo pour les usages d'analyse et de monitoring. Il comporte quatre volets :

- **Alertes actives** : liste des bénéficiaires à risque critique ou élevé, avec filtre par programme, niveau de risque et date.
- **Graphe de réseau (Heatmap)** : visualisation interactive du réseau de fraude, avec les communautés détectées mises en couleur.
- **Scoring Manuel d'un ou plusieurs bénéficiaires** : permet de scorer manuellement un ou plusieurs bénéficiaires en saisissant leurs informations, et d'obtenir le score de risque ainsi que la recommandation correspondante.
- **Générateur d'explication par bénéficiaire** : Générer des explications sur le niveau de risque d'un bénéficiaire donné.

4.4.1 Tableau de bord de suivi du moteur de détection de fraude

Le tableau de bord (Figure 4.35) permet aux analystes fraude de surveiller en temps réel les alertes générées par le moteur et de gérer les cas détectés. Les principales fonctionnalités sont résumées au Tableau 4.9.

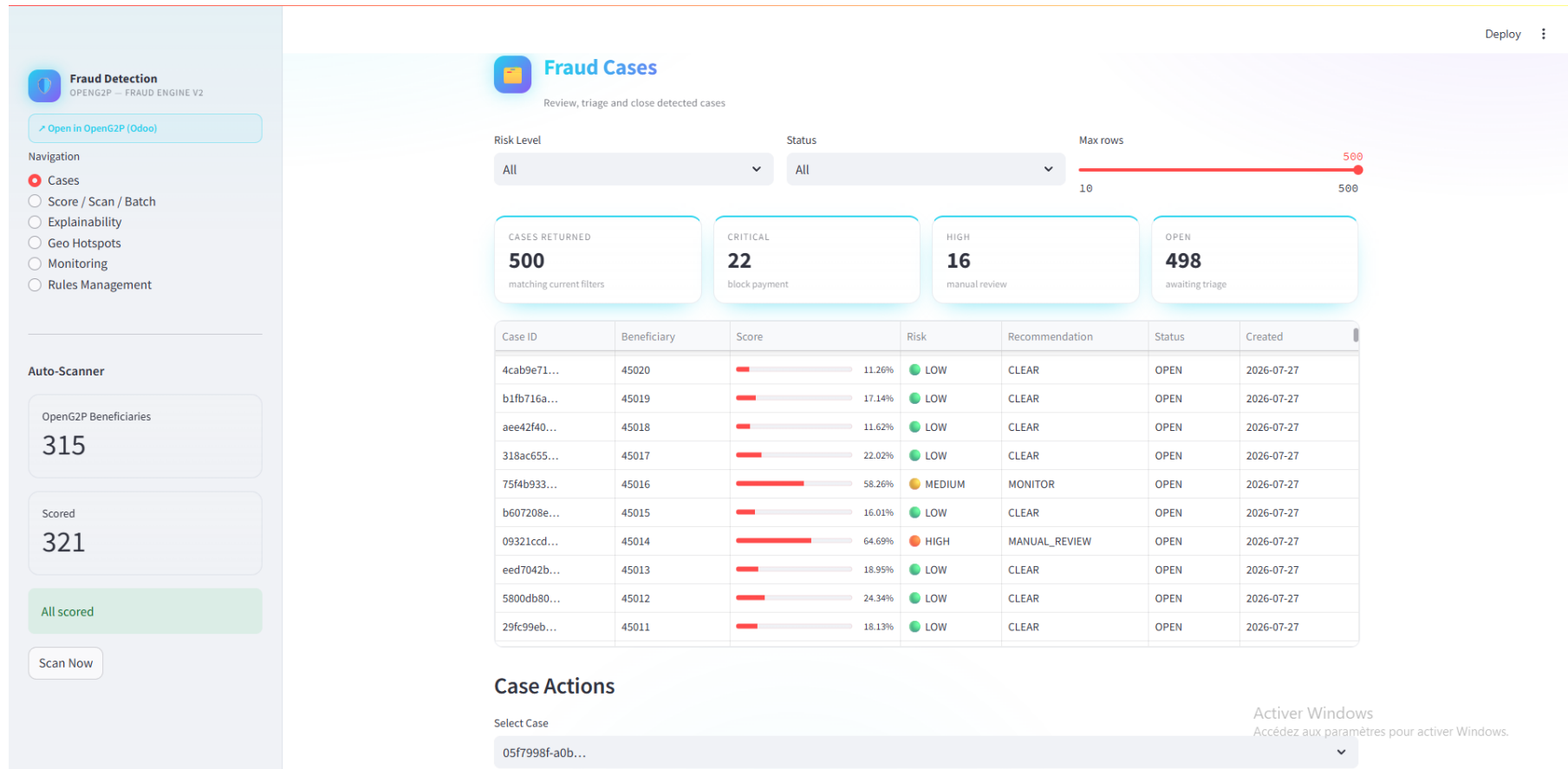


FIGURE 4.35 – Tableau de bord de suivi des cas de fraude

TABLEAU 4.9 – Fonctionnalités du tableau de bord pour l’analyste fraude

Section	Fonctionnalité
Fraud Cases	Liste complète des bénéficiaires signalés, filtrage par niveau de risque (LOW, MEDIUM, HIGH, CRITICAL) et statut (OPEN, IN_PROGRESS, RESOLVED). Affichage du score brut, de la recommandation (CLEAR, MONITOR, MANUAL_REVIEW, BLOCK) et de la date de détection.
Métriques	Quatre cartes synthétiques : cas retournés (matchant les filtres), cas CRITICAL (blocage paiement), cas HIGH (vérification manuelle), cas OPEN (en attente). Permet d’évaluer rapidement la charge de travail.
Auto-Scanner	Affichage du nombre de bénéficiaires traités et du nombre restant. Bouton pour déclencher un scan immédiat sans attendre le cycle automatique.
Case Actions	Sélection d’un cas pour examiner son détail : features brutes, facteurs SHAP, explication LLM. Mise à jour du statut et enregistrement de la décision finale de l’analyste.

4.4.2 Analyse des dossiers de fraude

Analyse manuel d’un dossier

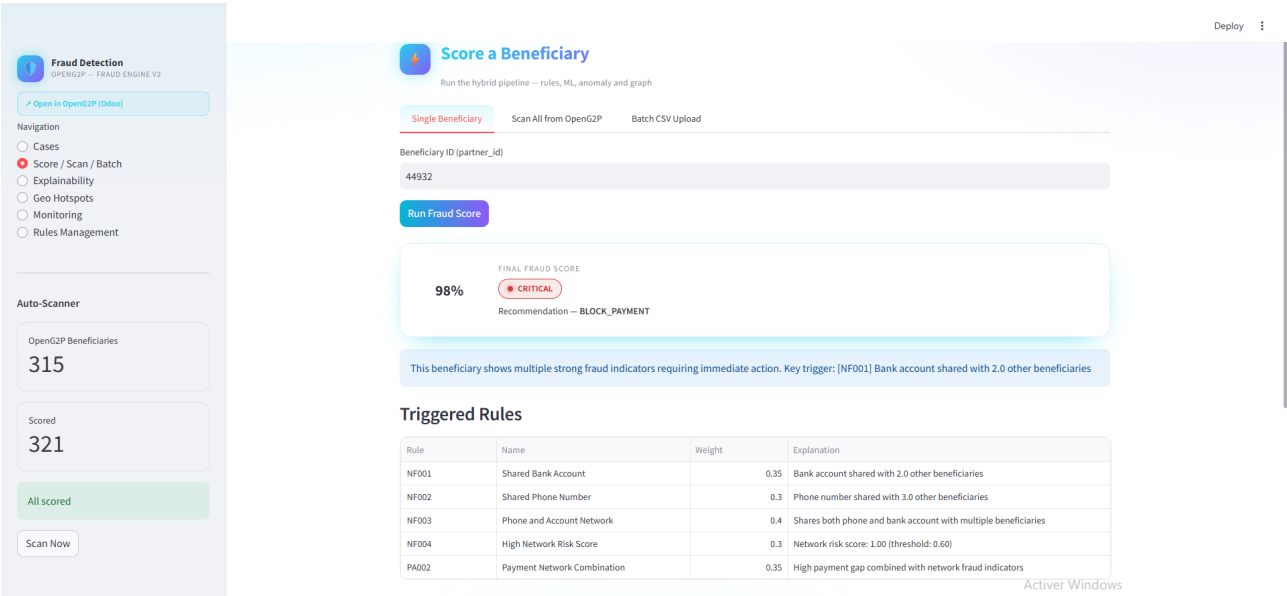


FIGURE 4.36 – Résultat du scoring d’un bénéficiaire

La Figure 4.36 illustre le résultat du scoring d’un bénéficiaire, avec le score final, les signaux déclenchés, les facteurs SHAP et l’explication générée par le modèle de langage. L’analyste peut ainsi comprendre rapidement les raisons du signalement et décider des actions à entreprendre.

Analyse d'un lot de dossier

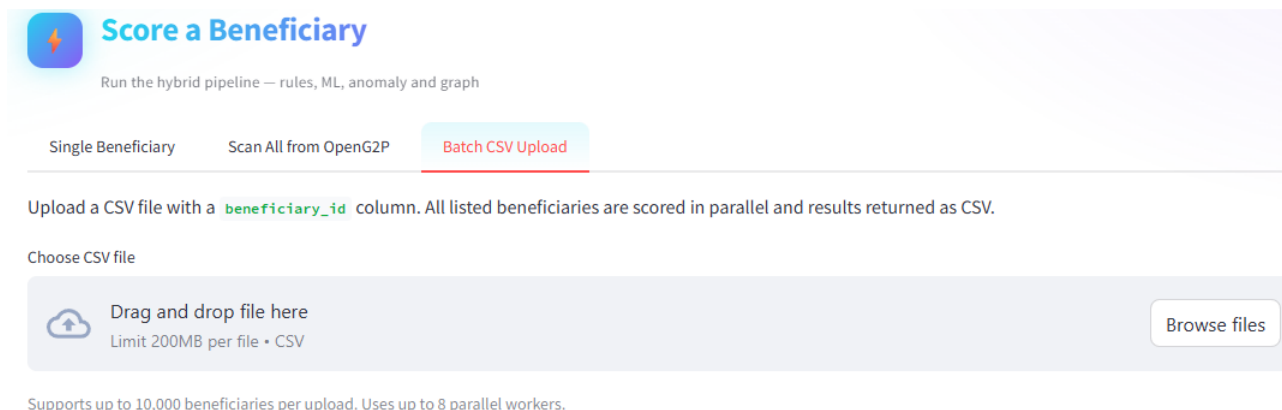


FIGURE 4.37 – Scoring des bénéficiaire au format CSV

La Figure 4.37 illustre le scoring des bénéficiaires au format CSV, avec le score final, les signaux déclenchés, les facteurs SHAP et l'explication générée par le modèle de langage. L'analyste peut ainsi comprendre rapidement les raisons du signalement et décider des actions à entreprendre.

Génération d'un rapport spécifique a un bénéficiaire

L'analyste fraude peut générer un rapport PDF détaillé pour chaque cas signalé (Figure 4.38). Le rapport synthétise en une seule page : l'identifiant du cas et du bénéficiaire, la date de génération, le statut de traitement, la recommandation du moteur (CLEAR, MONITOR, MANUAL_REVIEW, BLOCK), et le **score final avec son niveau de risque** (LOW, MEDIUM, HIGH, CRITICAL).

Le rapport inclut également un **diagramme de décomposition des scores** : chaque composant (Rule Score, ML Score, Graph Score) y est visualisé, montrant sa contribution au score final. Les **cinq features les plus influentes** (selon SHAP) sont listées avec leur impact relatif, permettant à l'analyste de comprendre rapidement les raisons de l'alerte. Un tableau récapitulatif (Feature Vector) complète le rapport avec les 28 valeurs numériques brutes du bénéficiaire, garantissant la traçabilité complète de la décision.

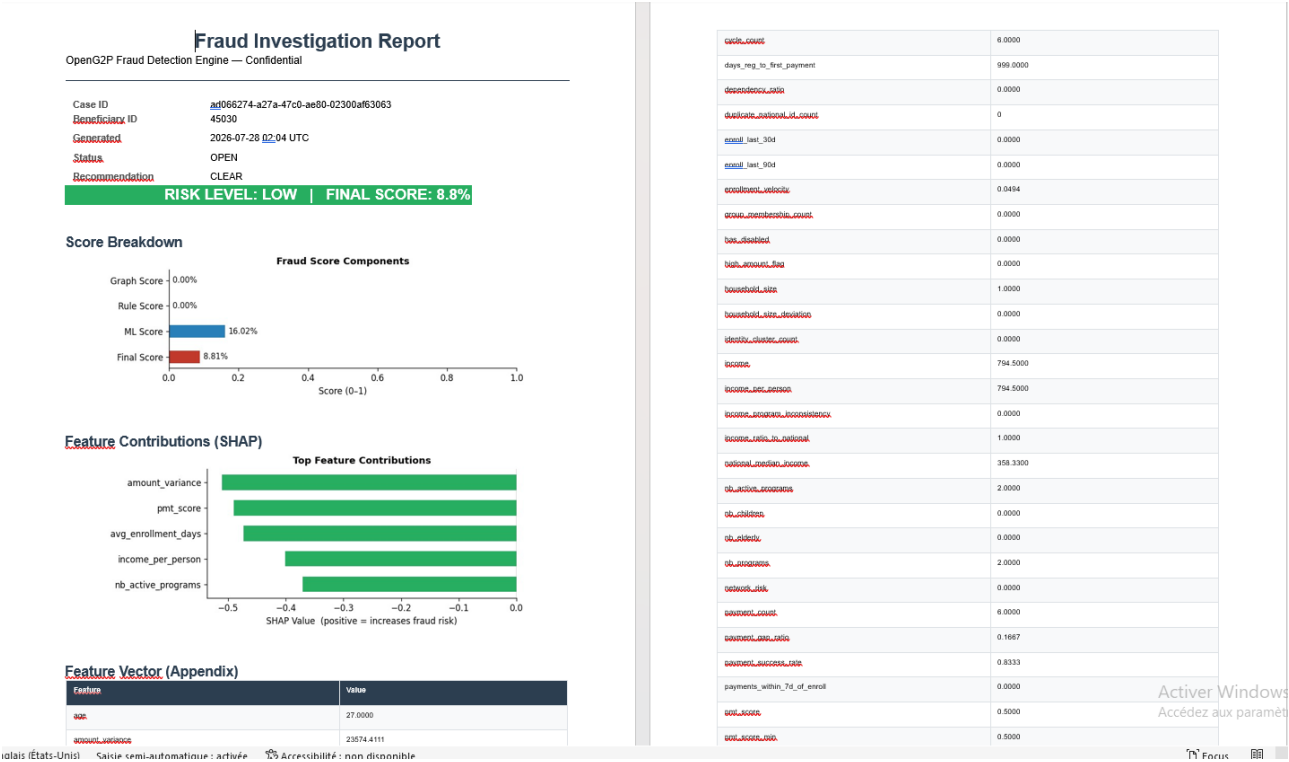


FIGURE 4.38 – Génération de rapport de fraude d’un bénéficiaire

Génération d’une explication détaillée

L’analyste fraude peut demander une explication complète sur chaque alerte : quelles règles ont déclenché, les scores de chaque agent (ML, réseau, règles), et comment ils se combinent pour l’alerte finale (Figure 4.39).

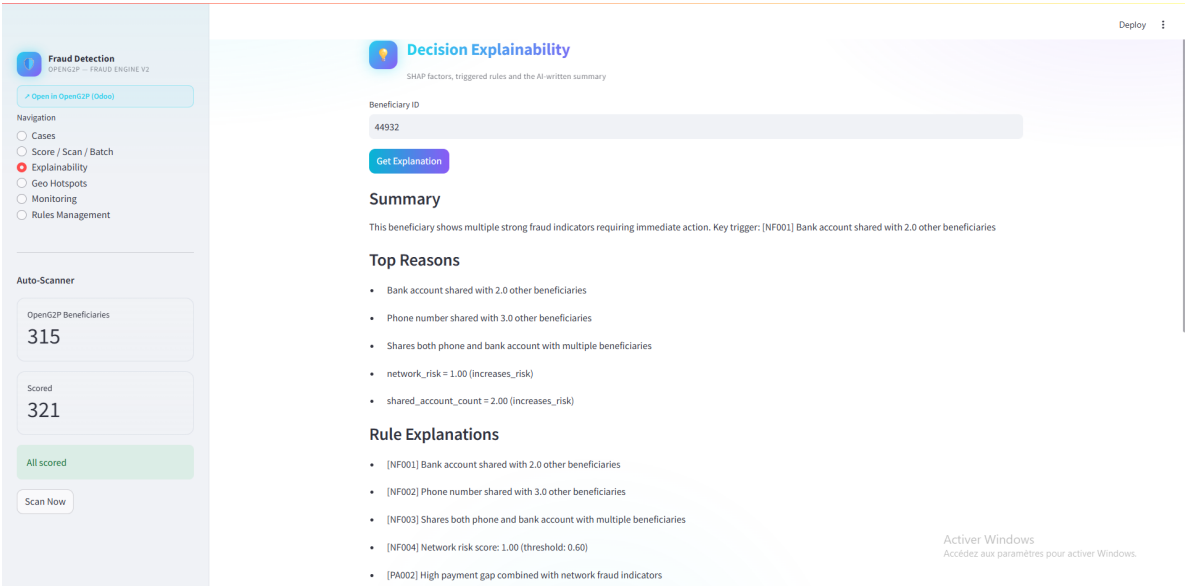


FIGURE 4.39 – Explication détaillée d’une décision de fraude

Analyse géospatiale des fraudes

Au-delà du scoring individuel, le système détecte également les **fraudes organisées par zone géographique**. L’algorithme DBSCAN regroupe les bénéficiaires spatialement proches(en fonction des adresses ou des zones fournies lors de l’inscription au programme social) dont les scores de fraude sont élevés, révélant des poches de fraude concentrées. La Figure 4.40 cartographie ces clusters : les zones en rouge et orange signalent une densité de fraude importante (West Africa notamment), tandis que les zones vertes indiquent des clusters isolés de faible densité. Cette vue permet aux équipes d’ajuster les contrôles par région : renforcer les vérifications manuelles dans les zones critiques, ou adapter les seuils de risque selon le contexte géographique du programme.

Le tableau joint synthétise les onze clusters détectés : pour chacun, on retrouve le centre géographique (latitude/longitude), le rayon englobant les bénéficiaires du cluster, le nombre total de cas, le nombre de cas HIGH ou CRITICAL, le taux de fraude moyen, et le score moyen. Le cluster 6 (Senegal/Guinea-Bissau) concentre 11 cas HIGH ou CRITICAL sur 27 bénéficiaires (taux de fraude de 40,7

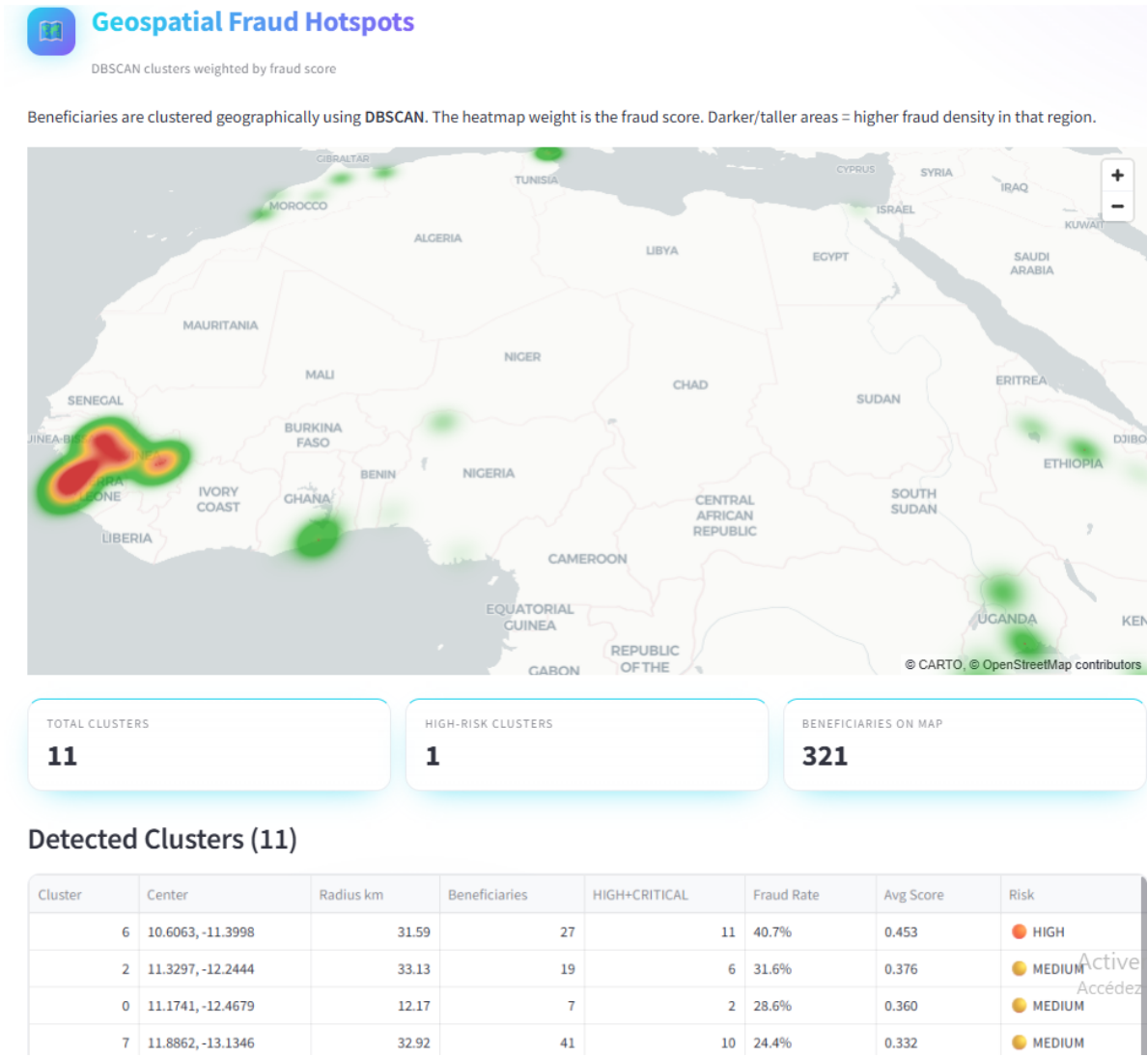


FIGURE 4.40 – Carte des clusters de fraude géographiques détectés par DBSCAN

Monitoring et amélioration continue du système

Un tableau de bord dédié (Figure 4.41) permet de superviser la santé du moteur en temps réel : statut du service, disponibilité des modèles ML, nombre de règles actives. La distribution des risques affiche le nombre de cas par niveau (LOW, MEDIUM, HIGH, CRITICAL) et par recommandation (CLEAR, MONITOR, MANUAL_REVIEW, BLOCK_PAYMENT), donnant une vue instantanée de la charge d'investigation.

Une section **Investigator Feedback** collecte les retours des analystes : nombre de verdicts confirmés, fraudes validées, faux positifs détectés. Ces données alimentent automatiquement un cycle de réentraînement hebdomadaire : les cas confirmés comme frauduleux et les faux positifs servent de labels pour affiner les modèles ML, garantissant que le système s'adapte aux patterns de fraude émergents et réduit progressivement ses erreurs.

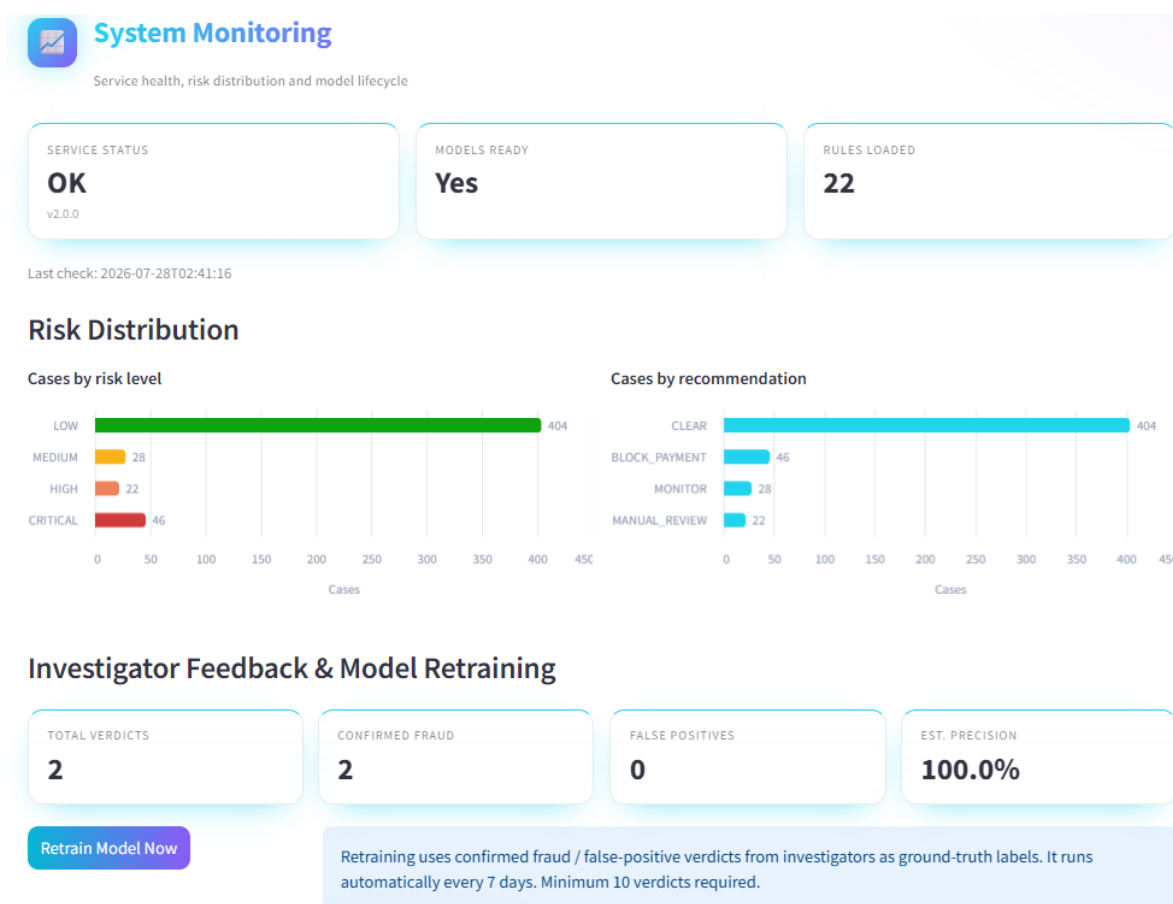


FIGURE 4.41 – Tableau de monitoring du système et feedback des analystes

Gestion des règles métier

Un interface dédié (Figure 4.42) permet aux administrateurs de créer, modifier ou désactiver les règles de détection sans redémarrer le service. Les règles sont stockées en YAML et rechargées automatiquement après chaque modification. Les conditions sont évaluées via un évaluateur AST restreint (opérations arithmétiques et comparaisons uniquement), ce qui garantit la sécurité : aucun code Python arbitraire ne peut être injecté.

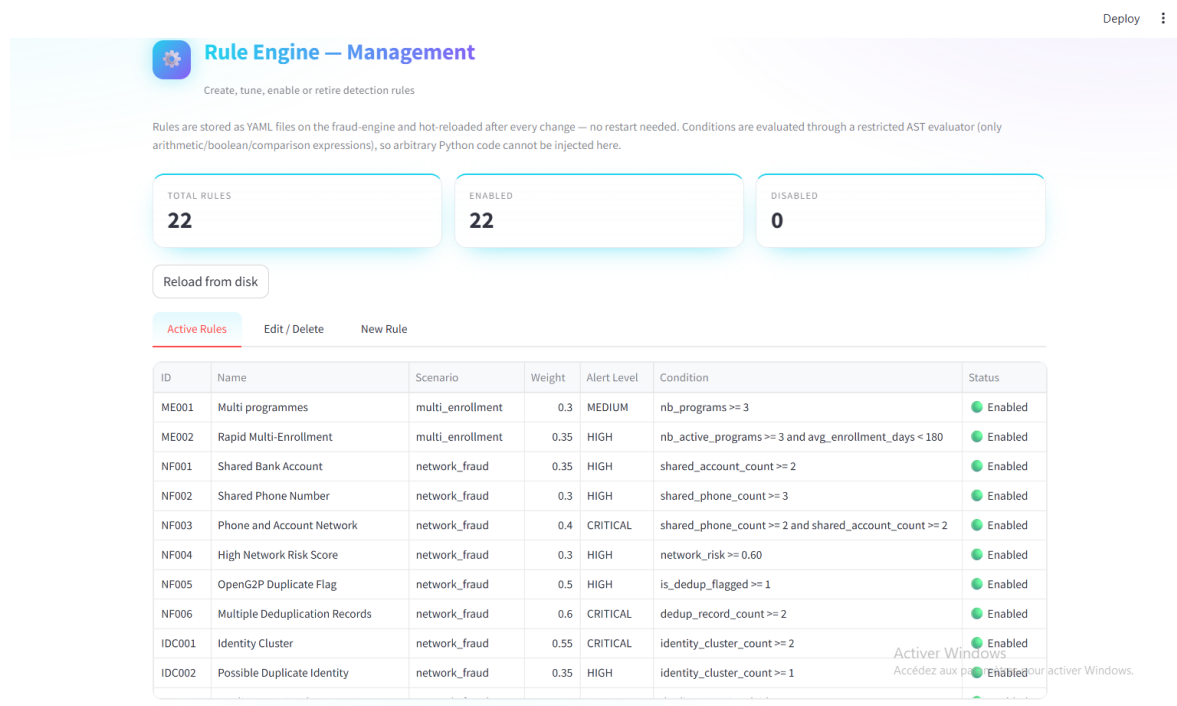


FIGURE 4.42 – Interface de gestion des règles de détection de fraude

Le tableau affiche les 22 règles actives avec leur identifiant, nom, scénario de fraude couvert (multi-enrollment, network_fraud, duplicate identity), poids dans le score final, niveau d'alerte déclenché, et condition logique. Chaque règle peut être activée, modifiée ou supprimée indépendamment, offrant une flexibilité maximale aux métiers pour adapter la détection sans intervention technique.

4.5 Conclusion partielle

Ce chapitre a présenté la mise en œuvre complète du moteur intelligent, depuis la génération des données d'entraînement jusqu'au déploiement de l'interface utilisateur. Toutes les fonctionnalités spécifiées au chapitre précédent ont été réalisées et intégrées dans un système cohérent et déployable via une seule commande Docker Compose.

Récapitulatif des fonctionnalités développées :

- Génération d'un dataset synthétique de 5 000 bénéficiaires (13,2 % de fraudes, 9 scénarios) avec injection PostgreSQL
- Extraction de 28 features SQL multi-tables depuis OpenG2P avec gestion défensive des valeurs manquantes

- Moteur de règles métier : 22 règles YAML réparties en 5 scénarios, évaluation sécurisée par AST, `rule_score` avec liste de signaux déclenchés
- Ensemble par stacking (Random Forest, XGBoost, Régression Logistique + méta-modèle) avec correction du déséquilibre par pondération de classe (`class_weight`, `scale_pos_weight`) : AUC 0,912, F1 0,748 au seuil retenu
- Isolation Forest pour la détection d'anomalies non supervisée, exposée dans l'explicabilité (intégration au score pondéré final : piste d'amélioration, voir dernier chapitre)
- Graphe NetworkX + PageRank pour l'identification des réseaux frauduleux
- Score hybride pondéré (20 % règles + 55 % ML + 25 % graphe) avec 4 niveaux de risque
- Explicabilité SHAP (importance globale et individuelle) + LLM Mistral via Ollama (explication en français naturel)
- Module Odoo `g2p_fraud_detection` avec synchronisation automatique et webhook de scoring en temps réel
- Tableau de bord de suivi : dossiers de fraude, métriques synthétiques, auto-scanner, actions sur les cas

Le chapitre suivant évalue la performance de ce système dans des conditions proches de la production : métriques ML approfondies, tests d'intégration avec OpenG2P, analyse des cas limites et mesure des temps de réponse.

Chapitre 5 Perspectives

Le moteur présenté dans ce mémoire est fonctionnel : il extrait les données d'OpenG2P, calcule un score hybride, l'explique et restitue une décision dans l'interface des agents sociaux. Il reste néanmoins un prototype validé en environnement de développement. Ce chapitre revient d'abord sur ses limites, puis expose les évolutions envisagées à court terme, les conditions d'un passage en production, et enfin les extensions possibles au-delà du cadre initial.

5.1 Limites du travail réalisé

La principale limite tient à la nature des données. Faute d'accès à un registre social réel — les données de bénéficiaires étant sensibles et protégées — l'entraînement repose sur un jeu synthétique généré à partir de distributions plausibles. Le recours au dataset PaySim a permis de vérifier que la chaîne d'entraînement généralise à des données réelles, mais rien ne garantit que les fraudeurs réels d'un programme de protection sociale se comportent exactement comme ceux du générateur. Les métriques obtenues doivent donc se lire comme une validation de la méthode, non comme une prédiction de performance en conditions réelles.

Deux autres limites méritent d'être signalées. Les poids de la formule hybride ont été fixés par raisonnement sur la nature de chaque signal, non appris sur des données de validation. Et le système n'a jamais été confronté à des analystes fraude en situation réelle : son ergonomie et la pertinence de ses explications restent à valider auprès des utilisateurs finaux.

5.2 Améliorations à court terme

5.2.1 Apprentissage des poids de fusion

Les poids 55/20/25 pourraient être estimés statistiquement plutôt que fixés a priori. La méthodologie de Noçairi et al. [?], déjà mobilisée pour justifier la combinaison pondérée, propose d'apprendre ces poids par régression logistique PLS sur les prédictions validées croisées — une approche qui garantit leur positivité et leur optimalité au sens d'un critère de performance mesurable. Cette évolution suppose de disposer d'un volume suffisant de verdicts d'analystes pour servir de vérité terrain.

5.2.2 Intégration du score d'anomalie

L'Isolation Forest est aujourd'hui calculé et affiché, mais exclu du score de décision. Son intégration avec un poids faible — de l'ordre de 5 % — donnerait au système un filet de sécurité contre les fraudes émergentes, celles dont aucun exemple ne figure encore dans les données d'entraînement. Ce poids devra être réévalué conjointement aux trois autres.

5.2.3 Calibration par pays

Les seuils des règles métier sont actuellement identiques pour tous les contextes. Or un revenu de 800 unités monétaires n'a pas la même signification selon le revenu médian national. Le raccordement à l'API de la Banque Mondiale permettrait de calibrer automatiquement ces seuils sur des indicateurs locaux (revenu médian, seuil de pauvreté, taille moyenne des ménages), avec une table de référence pré-alimentée pour garantir des évaluations déterministes et rapides.

5.2.4 Enrichissement des explications par RAG

Les explications générées par Mistral s'appuient aujourd'hui sur les seuls facteurs SHAP du cas courant. Une base vectorielle ChromaDB contenant des cas de fraude historiques documentés permettrait au modèle de langage de s'appuyer sur des précédents comparables, rendant l'explication plus concrète pour l'analyste. Cet usage reste strictement narratif : les seuils numériques continuent de provenir du moteur de règles, jamais de la base vectorielle.

5.3 Conditions d'un déploiement réel

5.3.1 Protection des données personnelles

Un déploiement sur données réelles impose un cadre juridique et technique que le prototype n'aborde pas. Le système manipule des données personnelles sensibles — revenus, composition du ménage, handicap, coordonnées bancaires — soumises aux réglementations nationales de protection des données. Trois chantiers en découlent : la minimisation des données extraites, la traçabilité des accès (qui a consulté quel dossier, quand), et le droit du bénéficiaire à contester une décision automatisée. Ce dernier point rejoint directement le travail sur l'explicabilité : une explication compréhensible n'est pas seulement un confort pour l'analyste, elle devient une obligation.

5.3.2 Supervision du modèle en production

Un modèle entraîné une fois se dégrade avec le temps : les profils de bénéficiaires évoluent, et les fraudeurs s'adaptent aux contrôles. La boucle de réentraînement déjà en place — alimentée par les verdicts des investigateurs — constitue une première réponse, mais elle suppose un volume régulier de retours et un suivi de la dérive des distributions. Un système d'alerte sur la dérive des features, comparant les distributions courantes à celles de l'entraînement, permettrait de déclencher un réentraînement avant que les performances ne se dégradent visiblement.

5.3.3 Montée en charge

Le prototype traite quelques centaines de bénéficiaires. Un programme national en compte des centaines de milliers, voire des millions. Deux composants demandent une attention particulière à cette échelle : la construction du graphe relationnel, dont le coût croît fortement avec le nombre de nœuds et d'arêtes, et la génération d'explications par le modèle de langage, coûteuse en temps de calcul. Une stratégie de traitement par lots, un calcul incrémental du graphe, et une génération d'explication à la demande — plutôt que systématique — constituent les pistes

les plus immédiates.

5.4 Extensions envisageables

Au-delà du programme initial, le moteur est conçu pour être réutilisable. Les règles métier étant externalisées en fichiers de configuration et modifiables sans redéploiement, l'adaptation à un autre programme social — pension de vieillesse, allocation scolaire, aide alimentaire — relève d'un travail de paramétrage plutôt que de développement. La même logique vaut pour un déploiement dans un autre pays : la calibration par indicateurs nationaux, décrite plus haut, en constitue le prérequis technique.

À plus long terme, le système peut être envisagé comme une brique d'infrastructure publique numérique, greffable sur n'importe quelle plateforme de transferts sociaux exposant ses données. L'architecture en services, découplée de l'implémentation interne du pipeline, a été pensée dans cette optique : un client qui consomme l'API n'a pas besoin de connaître les modèles utilisés, ni leur nombre.

Enfin, une dimension reste largement inexplorée : l'usage du moteur non pour sanctionner, mais pour corriger. Les mêmes signaux qui détectent la fraude peuvent révéler des erreurs d'inclusion involontaires — un ménage mal enregistré, un revenu saisi dans la mauvaise unité — et servir à améliorer la qualité du registre social plutôt qu'à exclure des bénéficiaires. Cette réorientation, plus qu'une évolution technique, relève d'un choix de conception que le système actuel rend déjà possible.

Bibliographie

- [1] Banque Mondiale. (2023). *World Development Indicators*. <https://data.worldbank.org/indicator>
- [2] OpenG2P Community. (2023). *OpenG2P Technical Documentation and Deployment Guide*. <https://openg2p.dev/docs/>
- [3] Banque Mondiale. (2020). *The Sourcebook of Social Protection Programmes*. Chapter 8 : Social Protection Finance and Risk. Washington, DC.
- [4] Noçairi, H., Gomes, C., Thomas, M., & Saporta, G. (2016). *Improving Stacking Methodology for Combining Classifiers : Applications to Cosmetic Industry*. Electronic Journal of Applied Statistical Analysis, 9(2), 340–361. <https://doi.org/10.1285/i20705948v9n2p340>